Минобрнауки России

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Волгоградский государственный технический университет»

Факультет электроники и вычислительной техники

Кафедра САПР и ПК

Согласовано

_____
(должность гл. специалиста предприятия)

_____ _____
(подпись) (инициалы, фамилия)

«_____» _____20___г

Утверждаю
Зав. кафедрой

_____ _____
(подпись) (инициалы, фамилия)

«_____» _____20___г

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к выпускной квалификационной работе бакалавра на тему
(наименование вида работы)
Разработка интеллектуального инструментария для
автоматизированного создания динамических игровых миров

Автор _____ Коваленко Кирилл Дмитриевич
(подпись и дата подписания) (фамилия, имя, отчество)

Обозначение _____
(код документа)

Группа ИВТ-465
(шифр группы)

Направление 09.03.01 Информатика и вычислительная техника
(код и наименование направления, наименование программы (профиля))

Профиль «Разработка игровых и XR приложений»

Руководитель работы _____ _____
(подпись и дата подписания) (инициалы и фамилия)

Консультанты по разделам:

_____ _____ _____
(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

_____ _____ _____
(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

_____ _____ _____
(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

Нормоконтролер _____ _____
(подпись, дата подписания) (инициалы и фамилия)

Волгоград 2027 г.

АННОТАЦИЯ

Объём пояснительной записки — 117 с., 18 рис., 16 табл., 41 источник, 4 приложения.

Выпускная квалификационная работа посвящена разработке подключаемого инструментария WorldWeaver для автоматизированного создания динамических игровых миров в Unity. Основной интерфейс версии 1.1 представляет собой редакторский чат с выбором модели и бюджета обработки. Инструментарий объединяет интерпретацию естественного языка, проверяемые операции сцены, процедурную генерацию Terrain, подготовку NPC с индивидуальными знаниями и исполнение высокоуровневых игровых действий после детерминированной проверки.

Практический результат представляет собой UPM-пакет с отдельными сборками Runtime и Editor, циклом EditorPlan → инструменты → фактическое состояние, демонстрационными компонентами и автоматическими проверками. Новый C# проходит отдельный просмотр, настоящую компиляцию Unity и подключение к указанному объекту; предусмотрено восстановление собственных файлов. Локальные вычисления выполняются через Ollama, для отдельной облачной конфигурации используется бесплатный Pollinations Classic API. Сохранённый QA-эксперимент версии 1.0 характеризует фиксированный корпус знаний и не переатрибутируется редакторскому агенту 1.1. Программные тесты, конкретные модельные сценарии и проверка кода оцениваются раздельно.

Ключевые слова: UNITY, РЕДАКТОРСКИЙ АГЕНТ, ПРОЦЕДУРНАЯ ГЕНЕРАЦИЯ, NPC, БОЛЬШАЯ ЯЗЫКОВАЯ МОДЕЛЬ, RAG, ОРКЕСТРАЦИЯ, ВАЛИДАЦИЯ ДЕЙСТВИЙ.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ..... 9
1.1 Процедурное создание игровых миров..... 9
1.2 Разделение авторинга и поведения во время игры..... 10
1.3 Языковые модели и извлечение знаний..... 11
1.4 Интеллектуальные неигровые персонажи..... 13
1.5 Сопоставление существующих решений..... 14
1.6 Постановка задачи и критерии качества..... 16
1.7 Сопоставление с университетскими исследованиями..... 18
2 ПРОЕКТИРОВАНИЕ ИНСТРУМЕНТАРИЯ..... 20
2.1 Требования и границы системы..... 20
2.2 Структура Unity Package..... 21
2.3 Маршрутизация модельных задач..... 23
2.4 Абстракция провайдеров и сетевой контракт..... 25
2.5 Архитектура знаний и памяти..... 26
2.6 Спецификация и генерация Terrain..... 29
2.7 Контракты редакторского агента и игровых действий..... 31
2.8 Диаграммы процессов и потоков данных..... 36
2.9 Выбор локальной среды и ограничения развёртывания..... 39
3 РЕАЛИЗАЦИЯ И ДЕМОНСТРАЦИЯ..... 43
3.1 Среда разработки и состав поставки..... 43
3.2 Единый чат в Unity Editor..... 44
3.3 Процедурный генератор рельефа..... 49
3.4 Локальные и бесплатные облачные запросы..... 52
3.5 Реализация retrieval и индексации..... 55
3.6 Обработка диалога и структурированного ответа..... 58
3.7 Передача предмета и автономное движение..... 60
3.8 Диагностика и ограничения безопасности..... 62
3.9 Демонстрационный сценарий..... 64
4 ТЕСТИРОВАНИЕ И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА..... 67
4.1 Цели и организация проверки..... 67
4.2 Модульные и сетевые испытания..... 68
4.3 Интеграция в Play Mode..... 70
4.4 Метрики качества ответов и retrieval..... 73
4.5 Локальное и облачное исполнение..... 76
4.6 Производительность и воспроизводимость Terrain..... 78
4.7 Формализация метрик и проверка воспроизводимости..... 79
4.8 Ограничения экспериментального вывода..... 81
4.9 Проверка редакторского чат-ассистента..... 82
ЗАКЛЮЧЕНИЕ..... 95
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ..... 100
ПРИЛОЖЕНИЕ А Техническое задание..... 105
ПРИЛОЖЕНИЕ Б Приемочные сценарии..... 106
ПРИЛОЖЕНИЕ В Корпус вопросов к индивидуальным знаниям..... 109
ПРИЛОЖЕНИЕ Г Порядок установки и повторения проверки..... 111

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

Таблица 1 — ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

Сокращение	Значение
API	Программный интерфейс взаимодействия компонентов
LLM	Большая языковая модель
NPC	Неигровой персонаж
RAG	Генерация ответа с извлечением внешних знаний
UPM	Unity Package Manager
JSON	Текстовый формат структурированных данных
fBm	Фрактальная сумма шумовых октав
NavMesh	Навигационная сетка для поиска пути
DTO	Объект передачи структурированных данных
UML	Унифицированный язык моделирования
DFD	Диаграмма потоков данных

ВВЕДЕНИЕ

Создание игрового мира включает несколько различных по природе задач. Рельеф и размещение объектов требуют управления геометрией и повторяемыми параметрами. Диалог персонажа требует интерпретации языка и согласования реплик с ролью. Игровые последствия разговора, например передача предмета или выбор цели перемещения, подчиняются строгим правилам состояния сцены. При раздельной реализации этих подсистем разработчик вынужден соединять несколько инструментов и самостоятельно поддерживать общий контекст.

Большие языковые модели позволяют формулировать намерения на естественном языке и получать содержательные ответы без предварительного описания всех диалоговых ветвей. Однако текстовый ответ сам по себе не обеспечивает корректного поведения игры. Модель может назвать отсутствующую цель, предложить передачу недоступного предмета либо использовать сведения, которые персонажу неизвестны. Поэтому практическая применимость LLM определяется не только качеством речи, но и устройством границы между вероятностным решением и детерминированным игровым кодом.

В этой работе рассматривается инструментальная задача: разработчик устанавливает пакет в существующий Unity-проект и ставит задачи в едином чате. Модель составляет план создания или изменения мира, а встроенный исполнитель проверяет команды, применяет их и возвращает прочитанное состояние сцены. В этом же процессе создаётся реальный компонент NPC с личностью, базой знаний и маршрутами моделей. Расширенный Inspector остаётся средством точного контроля. Результатом являются расширение среды разработки и runtime-компоненты; демонстрационная игра служит средством проверки инструментария.

Редакторский агент должен учитывать не только правильность отдельной команды, но и полноту исходного запроса. Поэтому после исполнения при наличии бюджета предусмотрен новый раунд с фактическим

результатом: модель может обнаружить отсутствующий параметр, исправить его или подтвердить завершение без повторного создания объектов. Сохранность проекта обеспечивается ограниченным набором встроенных инструментов, предконтролем плана, явными целями и отменой пакета. Для расширения поведения отдельно рассматривается путь просмотренного C#: компилятор проверяет программу, но не создаёт sandbox и не гарантирует правильность её будущего выполнения.

Актуальность задачи связана с распространением облачных и локальных LLM и потребностью контролировать стоимость, задержку и зависимость от поставщика. Современные продукты уже предоставляют редакторские чаты, инструменты изменения сцены, разговорных персонажей, retrieval и действия. Следовательно, обоснование разработки не строится на отсутствии аналогов. Исследовательский интерес представляет согласованное соединение редакторского цикла с воспроизводимым TerrainSpec, готовыми NPC с индивидуальными знаниями и наблюдаемым исполнением в переносимом пакете.

Цель работы — разработать и экспериментально оценить подключаемый инструментарий Unity, который преобразует естественно-языковые запросы в проверяемые редакторские планы, параметры игрового мира и контекстно-зависимые решения NPC, сохраняя детерминированный контроль встроенных операций изменения сцены.

Для достижения цели решаются следующие задачи: анализ актуальных средств Unity AI и интеллектуальных персонажей; формирование требований к переносимому пакету; проектирование редакторского цикла, маршрутизации моделей и границ доверия; разработка генератора рельефа и компоновщика окружения; реализация индивидуального RAG и памяти NPC; разработка редакторских инструментов, реестра игровых действий и проверок предусловий; создание чата на UI Toolkit и runtime-панели; реализация просмотра C#, компиляции и восстановления файлов; проведение раздельных программных, модельных и предметных испытаний.

Объект исследования — процессы автоматизации создания и изменения динамического игрового мира. Предмет исследования — архитектурные и алгоритмические способы объединения LLM, retrieval и процедурных компонентов Unity при ограниченном доступе модели к состоянию игры.

Исследование уточняется тремя вопросами. RQ1: как добавление индивидуального RAG изменяет совпадение известного факта в ответе одной и той же локальной модели? RQ2: какие полные и провайдерские задержки наблюдаются у выбранных локальных и бесплатной облачной конфигураций? RQ3: сохраняет ли детерминированный исполнительный слой проверяемые инварианты Unity при ошибочном или недопустимом решении модели? Первые два вопроса исследуются в сохранённом базовом QA-протоколе версии 1.0: сравнением без RAG и с RAG и измерением p50/p95. Третий охватывает программные и сквозные проверки runtime и нового редакторского исполнителя. Успешность выполнения свободной задачи агентом и компиляция просмотренного компонента требуют собственных протоколов; они не выводятся из метрик ответов на вопросы о лоре.

Методы работы включают анализ первичных технических источников, объектно-ориентированное проектирование, процедурную генерацию, векторное представление текста, поиск по косинусному сходству, контрактную валидацию и экспериментальное сравнение конфигураций. Программные свойства проверяются средствами Unity Test Framework, а влияние базы знаний оценивается на фиксированном авторском корпусе с известными ответами.

Практическая значимость состоит в возможности повторно использовать архитектуру в учебных и экспериментальных Unity-проектах. Единый чат связывает подготовку окружения, NPC, знаний и действий, а детальные настройки сохраняются в обычных компонентах и ассетах. Разработчик может заменить провайдера или добавить runtime-действие

через интерфейс. Основная проверенная среда нового редакторского процесса — Unity 6000.3.27f1; испытание базового ядра 1.0 в Unity 2022 не подтверждает полную совместимость нового интерфейса с этой версией. Работа не доказывает сокращение труда новичка, художественную убедительность мира или превосходство над всей экосистемой Unity.

Работа состоит из четырёх глав. В первой главе анализируются предметная область и аналоги. Во второй главе обосновывается архитектура. Третья глава описывает реализацию и демонстрационный сценарий. Четвёртая глава содержит методику испытаний, измерения и ограничения результатов. В приложениях приведены техническое задание, сценарии проверки и дополнительные диаграммы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Процедурное создание игровых миров

Процедурная генерация формирует игровое содержимое алгоритмически, используя параметры и начальные условия. Для рельефа таким содержимым выступает поле высот, для растительности — набор позиций и типов объектов, для правил игры — структура допустимых состояний. При одинаковых входных данных детерминированный алгоритм позволяет восстановить результат и исследовать влияние отдельного параметра [32].

Поле высот удобно для ландшафта, поверхность которого можно представить функцией h(x,z). Unity Terrain хранит такую поверхность в регулярной сетке и предоставляет операции обновления высот. Каждая ячейка содержит нормированное значение, а размер TerrainData определяет физический масштаб сцены [3]. У этого представления есть ограничения: нависающие скалы, пещеры и произвольная многослойная геометрия требуют дополнительных мешей. Поэтому генерация Terrain не равнозначна генерации любого трёхмерного мира.

Шумовые методы позволяют получать непрерывные неоднородные поверхности с небольшим числом параметров. Суммирование нескольких октав изменяет вклад крупных и мелких деталей. Амплитуда очередной октавы задаётся persistence, частота — lacunarity. Небольшое число октав удобно для интерактивного редактора, поскольку объём вычислений заранее ограничен разрешением heightmap и количеством проходов.

Одного шума недостаточно для выполнения содержательного описания, например требования о долине с озером. Необходимо ввести семантические преобразования поверхности: горный край, центральное понижение, направленный уклон. Эти преобразования связывают привычные пользователю слова с контролируемыми числовыми параметрами. В таком

устройстве генератор остаётся воспроизводимым, а естественный язык определяет спецификацию, а не каждую высоту.

Профессиональные средства, включая продукты Procedural Worlds, решают более широкий круг задач окружения: подготовку ландшафта, распределение объектов и художественную настройку [10]. Учебный инструментарий рассматривает ограниченный набор форм, достаточный для проверки архитектуры. Сравнение по количеству биомов или качеству готовой графики не является целью работы.

1.2 Разделение авторинга и поведения во время игры

В игровых инструментах необходимо различать помощь разработчику и интеллект персонажа в запущенной сцене. Помощник Editor может анализировать проект, создавать код или ассеты и управлять инструментами редактора. Runtime-персонаж должен реагировать на текущего игрока, учитывать игровые данные и выполнять действия в пределах возможностей, предоставленных разработчиком.

Это различие влияет на права доступа и требования к производительности. Во время разработки допустимы сравнительно продолжительные операции и ручная проверка изменений. Во время игры обработка запроса не должна блокировать кадр, а неправильный ответ должен приводить к контролируемому отказу. Исполнение произвольного сгенерированного кода не требуется ни для передачи предмета, ни для выбора конечной точки пути.

Современный Unity AI Assistant уже решает задачи редакторского агента: документация описывает режимы Ask, Plan и Agent, чтение контекста проекта, создание объектов и изменение компонентов. Следовательно, противопоставление «готовый ассистент только советует, а разрабатываемый инструмент изменяет сцену» некорректно. Генераторы ассетов также выходят за пределы текста и включают, в частности, трёхмерные объекты и материалы Terrain. Генерация Terrain Layer описывает материал поверхности;

сама по себе эта функция не является измерением качества генерации heightmap по естественному языку.

AI Gateway предусматривает подключение сторонних агентов [5]. При этом требования конкретного пути подключения нельзя переносить на весь продукт: текущие Unity CLI, MCP и плагины заявлены как бесплатные, а встроенный Assistant и генераторы имеют собственные условия доступа и расходования кредитов. Бесплатность интерфейса интеграции не означает бесплатность inference внешнего провайдера. Запуск процесса агента на компьютере также не доказывает локальное выполнение языковой модели: для этого требуется проверить фактический endpoint. В рассмотренных материалах не установлена универсальная возможность произвольно заменить endpoint встроенного Assistant на Ollama, но отсутствие такого подтверждения не считается доказательством невозможности.

Редакторская автоматизация и локальный runtime-inference представляют разные уровни системы. Sentis предназначен для исполнения моделей в приложении Unity; он не задаёт сам по себе индивидуальную базу знаний NPC, семантику передачи предмета или политику доступа к игровому состоянию. Наличие средств Assistant, Gateway или Sentis поэтому нельзя автоматически приравнивать к готовности конкретного игрового сценария.

Средства MCP для Unity также относятся прежде всего к автоматизации разработчика. Например, Coplay MCP предоставляет операции над сценами, ассетами, скриптами и испытаниями [31]. Такой инструмент полезен при создании проекта, однако архитектурно отличается от поставляемого игрового компонента, который должен работать после завершения процесса разработки.

1.3 Языковые модели и извлечение знаний

Языковая модель формирует ответ на основе параметров и переданного контекста. Её обучение не содержит авторский лор нового игрового мира, если этот лор не был включён в запрос или специально использован при

адаптации модели. Поэтому свободный диалог о конкретной деревне, её жителях и предметах требует внешнего источника знаний.

Подход RAG соединяет генеративную модель с извлекаемой внешней памятью. В исходной работе Lewis и соавторов рассматривается сочетание параметрического генератора и непараметрического компонента извлечения для задач, требующих знаний [14]. В игровом инструментарии этот принцип реализуется как поиск нескольких релевантных фрагментов авторского корпуса и добавление их в контекст NPC.

Типовой pipeline включает загрузку документа, разбиение текста, получение embedding-вектора, сохранение фрагмента вместе с метаданными, представление запроса тем же embedding-пространством и поиск ближайших векторов. После retrieval генератор получает исходную реплику, сведения о персонаже и найденные фрагменты. Наличие RAG не отменяет возможности ошибки: неверно найденный фрагмент или неправильная интерпретация могут привести к недостоверному ответу.

Важен выбор границы фрагмента. Слишком короткий chunk теряет контекст факта; слишком длинный увеличивает объём нерелевантного текста. Перекрытие соседних окон снижает риск разрыва факта на границе, но создаёт частично дублирующиеся результаты. Поэтому метрики на уровне фрагментов и документов должны различаться: присутствие правильного документа в top k не означает, что все k фрагментов полезны.

Индивидуальность персонажа требует ограничения корпуса. Если торговец не знает пароль стражи, один только высокий cosine score не должен открывать ему служебный документ. Фильтрация по Scope должна предшествовать ранжированию. Передача всех найденных документов модели с инструкцией «не использовать чужие знания» оставляет ограничение на вероятностном уровне и не обеспечивает изоляцию данных.

Историю разговора также нецелесообразно полностью добавлять в каждый запрос. Её размер растёт с продолжительностью игры, а большая часть старых реплик может быть не связана с текущим вопросом. Извлечение

релевантной памяти позволяет ограничивать контекст, но требует решения о сроке хранения и достоверности события. Запись «модель предложила передачу» отличается от записи «передача исполнена». В этой работе эпизодическое событие создаётся после результата валидатора.

1.4 Интеллектуальные неигровые персонажи

Для традиционного NPC поведение обычно задаётся конечным автоматом, деревом поведения или другими заранее определёнными правилами. Такие методы хорошо контролируются и подходят для реакций с жёсткими временными ограничениями. Однако описание всех возможных формулировок свободного разговора становится трудоёмким, особенно при индивидуальных характерах и знаниях.

Генеративный агент расширяет эту модель за счёт языковой интерпретации, памяти и планирования. Park и соавторы показывают архитектуру, в которой наблюдения, извлечение памяти и планирование участвуют в формировании поведения [15]. Для рассматриваемого проекта полезен сам принцип разделения памяти и решения, но результаты социальной симуляции из этой работы не считаются доказательством качества разработанного NPC.

Работа ReAct связывает рассуждение языковой модели с действиями во внешней среде [16]. Применительно к Unity это означает выбор дискретного высокоуровневого действия, например MoveTo или GiveItem. Низкоуровневое движение остаётся задачей NavMeshAgent, а состояние предметов — задачей Inventory. Модель не должна определять координату Transform каждый кадр: это увеличивает задержку, стоимость и нестабильность управления.

Личность персонажа влияет на выбор среди допустимых решений. Торговец может отказать, попросить оплату или согласиться помочь. Но ограничения игры нельзя выражать только характеристикой «осторожный». Если NPC не должен отдавать больше одного предмета, это правило задаётся

числовым лимитом и проверяется кодом. Текстовые установки личности остаются мягкими предпочтениями, а не доказанной политикой доступа.

Существенный риск связан с расхождением речи и действия. Реплика «я отдал кристалл» не означает, что предмет действительно переместился. Поэтому ответ модели должен содержать отдельное поле речи и массив структурированных действий. Пользовательский интерфейс отображает подтверждённое состояние инвентаря, а ошибочное действие фиксируется с причиной отказа.

1.5 Сопоставление существующих решений

Сравнение выполнено по официальной документации и репозиториям на дату исследования. Оно характеризует опубликованные возможности, но не заменяет установку и нагрузочное испытание всех коммерческих SDK. Для неподтверждённой функции используется формулировка «не установлено», поскольку отсутствие сведений в одном разделе не доказывает отсутствие возможности во всём продукте.

Таблица 2 — Сопоставление существующих решений

Решение	**Основная подтверждённая возможность**	**Существенное отличие от задачи работы**
Unity AI Assistant и Gateway	Режимы Ask/Plan/Agent, редакторские инструменты, генераторы и сторонние агенты	Возможности описаны документацией; одинаковый сравнительный прогон не выполнялся
Convai Unity SDK	Персонажи, знания, действия и цели	Готовая специализированная экосистема персонажей
Inworld Runtime	LLM, речь, embeddings и knowledge	Набор runtime-модулей с собственной архитектурой
NVIDIA ACE	Agent, Chat, RAG и on-device inference	C/C++ и собственные требования inference-стека
Promethean AI	Поиск и компоновка 3D-ассетов	Работа с окружением, не только с heightmap
Gaia	Процедурное Unity-окружение	Более широкие художественные средства Terrain
LLMUnity	Локальные модели, диалог и RAG	Уже решает существенную часть локального NPC

Решение	Основная подтверждённая возможность	Существенное отличие от задачи работы
WorldWeaver	Редакторский чат, TerrainSpec, scoped RAG и проверяемые инструменты	Специализированный набор для мира и NPC; не охватывает весь Unity Editor

Актуальные сведения Unity уточнены по документации Assistant 2.21, доступной на дату исследования. Наличие выбора модели, режимов обработки, истории и видимых вызовов инструментов у развитого аналога исключает представление этих элементов интерфейса как самостоятельной научной новизны. У Unity также описаны разрешения на операции и checkpoints, поэтому один механизм Undo не даёт основания утверждать превосходство WorldWeaver в восстановлении. Unity Assistant в исследовательской среде не устанавливался и не испытывался: для него фиксируются документированные возможности, а время, успешность и субъективное удобство остаются неизмеренными.

Convai предоставляет средства конфигурирования доступных персонажу действий, объектов и целей, а также локального исполнения [6]. Следовательно, исполнение действий NPC не является самостоятельной новизной проекта. Официальное сообщение Inworld указывает на вывод прежнего Character Studio из эксплуатации и перенос соответствующих возможностей в Agent Runtime SDK и Unity-шаблоны с LLM, речевыми компонентами, knowledge и memory [7]. Поэтому legacy Studio не рассматривается как действующий продукт. Эти продукты необходимо сравнивать по составу и интеграции, а не представлять как обычные чат-боты.

NVIDIA ACE Game Agent SDK описывает Agent, Chat и RAG API, а также отдельные средства локального inference [8]. На используемом Apple M4 не выполнялась проверка NVIDIA inference-стека, поэтому работа не делает утверждения о его производительности на данной машине. Promethean

AI ориентирован на использование и организацию ассетов; его сайт называет Unreal основной интеграцией и описывает плагины других редакторов [9].

LLMUnity особенно важен для сопоставления, поскольку предоставляет локальную LLM-интеграцию и RAG непосредственно в Unity [11]. Разработка собственного компактного backend в настоящей работе мотивируется исследованием контролируемого pipeline и переносимости HTTP-провайдеров. Это инженерный выбор, а не вывод о непригодности готовой библиотеки.

Общие RAG-frameworks LangChain и LlamaIndex дают развитые абстракции загрузчиков, индексов и vector store [12, 13]. Их подключение через отдельный сервис расширило бы возможности системы, но добавило бы процесс и язык исполнения. Для малого демонстрационного корпуса выбран C#-индекс с интерфейсом замены backend. При увеличении объёма данных этот выбор должен быть пересмотрен.

1.6 Постановка задачи и критерии качества

По результатам анализа задача сформулирована как создание переносимого инструментария с ограниченной и измеряемой областью ответственности. Главным интерфейсом версии 1.1 становится единый редакторский чат: пользователь формулирует задачу, модель предлагает структурированный план, инструменты изменяют сцену, а последующий ответ опирается на прочитанное состояние Unity. Через этот процесс доступны создание и изменение Terrain, объектов и освещения, подготовка NPC, настройка индивидуальных знаний и подключение встроенных поведений. Расширенный Inspector сохраняется для точного ручного контроля.

Сквозное требование состоит в различении намерения, исполнения и подтверждения. Текст «мир создан» должен сопровождаться реальными TerrainData, объектами окружения и назначенными компонентами. Неизвестные параметры и недопустимые цели отклоняются до изменений;

результат выполнения возвращается агенту для продолжения многошаговой задачи. Предлагаемый C# проходит отдельный просмотр и настоящую компиляцию Unity. Такой путь расширяет набор задач, но не превращает исполнение произвольной программы в изолированную среду.

Интеграционная специализация работы состоит в сочетании этого редакторского процесса с типизированной генерацией рельефа и готовым runtime-контуром индивидуальных знаний и действий NPC. Она не предполагает создания универсального аналога всех инструментов Unity. Качество определяется воспроизводимой установкой, фактическим состоянием сцены и выполнением ограниченного сценария.

Функциональная корректность включает воспроизведение Terrain при одном seed и спецификации, отсутствие изменений сцены при отклонённом редакторском плане, сохранение посторонних объектов и возврат подтверждённых результатов инструментов. Для runtime отдельно проверяются отсутствие изменений инвентаря при неверной команде, получение ответа через реальную модель, изоляция знаний и исполнение движения через NavMesh в подготовленной области. Надёжность включает отмену ожидания, timeout, ограниченный retry и продолжение работы редактора после контролируемого отказа. Отмена до исполнения и остановка после завершённого шага являются разными исходами; второй случай должен сохраняться в журнале как уже произошедшее изменение.

Исследовательские вопросы RQ1–RQ3 связывают индивидуальные знания с совпадением факта, модельную конфигурацию с наблюдаемой задержкой и исполнительный слой с сохранением инвариантов. Для каждого вопроса требуется собственный источник доказательств: QA-корпус, временной протокол или тест состояния. Базовый QA-эксперимент версии 1.0 сохраняет свой исходный протокол и не служит оценкой нового редакторского чата. Для версии 1.1 программные проверки контрактов и сценарные испытания модели должны учитываться отдельно: корректный отказ валидатора подтверждает инвариант, но не означает успешного

выполнения пользовательской задачи. Исследование не доказывает общую интеллектуальность NPC, удобство для новичков или удовольствие игрока от диалога.

1.7 Сопоставление с университетскими исследованиями

Опубликованные выпускные исследования полезны для выбора метода оценки, но не задают единый шаблон оформления и не являются источником готового авторского результата. В бакалаврской работе O. Kyzr о процедурной генерации внешних сцен отдельно рассмотрены реализация, производительность и качественное воспроизведение сцен [35]. Для настоящей работы существенно такое разграничение: измерение функции heightmap не заменяет оценки времени подготовки полного окружения.

Магистерская работа F. Marcher посвящена инструменту генерации Terrain в Unity и включает отдельное исследование использования интерфейса участниками [36]. Из этого следует методическое требование различать функциональную проверку Editor и оценку удобства. В WorldWeaver проведён проход основных операций разработчиком и проверена чистая установка. Эти сведения подтверждают исполнимость сценария; на их основании не выводятся удобство для новичков и процент сокращения труда.

Бакалаврская работа O. R. J. Wakeford исследует генерацию рельефа с фиксированными точками [37]. Она показывает необходимость явно описывать геометрические ограничения и разные виды оценки. Для WorldWeaver аналогичное требование состоит в точном определении семейства TerrainSpec: радиальный край и долина не означают поддержку любого описанного мира. Численные показатели разных работ не сравниваются напрямую, поскольку различаются алгоритмы, устройства и измерительные границы.

Исследование P. Aslan о LLM-персонажах отдельно рассматривает несколько сторон RAG, включая работу при отсутствии знаний и

противоречивой информации [38]. Наш набор из 30 вопросов с известными ответами покрывает только более узкую задачу. Он не содержит полноценного отрицательного корпуса для оценки отказа и контрфактической устойчивости. Эти направления остаются за пределами подтверждённых метрик.

Сопоставление показывает, что полноту ВКР определяет связь постановки задачи, реализации и проверяемого результата. Вклад WorldWeaver является интеграционным: редакторский цикл с подтверждёнными результатами инструментов, типизированные спецификации, scope знаний, маршрутизация по задачам, реестр действий и единый процесс работы через UPM. RAG, fBm и NavMesh используются как известные методы, а не объявляются изобретёнными в настоящей работе.

Вывод первой главы состоит в необходимости разделить генерацию намерения и исполнение последствий. Современные аналоги подтверждают практическую применимость LLM и RAG в играх, но также задают требования к честному сопоставлению. Для дальнейшей разработки выбрана архитектура, в которой модель заменяема, знания имеют область видимости, а состояние Unity меняется только после проверки контрактов.

2 ПРОЕКТИРОВАНИЕ ИНСТРУМЕНТАРИЯ

2.1 Требования и границы системы

Исходное техническое задание определяет инструментарий как подключаемый продукт для Unity 6+, а не отдельную игру. Для реализации выбран UPM-пакет WorldWeaver с демонстрационным проектом. Пакет должен предоставлять авторинг в Editor и компоненты, которые продолжают выполнять свои функции в Play Mode.

Разработчик ставит задачу в редакторском чате, выбирает модель и предел глубины обработки, затем проверяет план и результаты изменений. Описание рельефа, личность NPC, документы знаний и разрешённые действия могут задаваться естественным языком с последующим переводом в проверяемые параметры. Игрок в Play Mode обращается к уже созданному персонажу и наблюдает речь и последствия его решения. Внешние системы предоставляют только inference: текстовый completion или embedding-вектор. Их вывод интерпретируется C#-слоем; непосредственного сетевого доступа к объектам Unity им не предоставляется.

Обязательные функции ограничены текстовым взаимодействием, редакторскими инструментами сцены, Terrain, retrieval, действиями NPC и диагностикой. Для небольших дополнительных компонентов предусмотрен отдельный просмотр C# и отчёт о компиляции. Голосовые интерфейсы и извлечение текста из PDF не включены в проверенный сценарий. Это ограничение не скрывается под общим утверждением о поддержке любых источников. Импорт знаний принимает TXT, Markdown и JSON с установленным форматом.

Таблица 3 — Требования и границы системы

Группа требований	Проектное решение	Способ проверки
Переносимость	Отдельный UPM-пакет, сборки Editor и Runtime	Установка и компиляция в тестовом проекте
Редакторский диалог	История, EditorPlan и ограниченный цикл инструментов	Несколько шагов с фактическим чтением результата

Группа требований	Проектное решение	Способ проверки
Сохранность сцены	Предконтроль плана, ownership, Undo и учёт новых ассетов	Отказ до записи и откат ошибочного пакета
Управляемость Terrain	TerrainSpec и детерминированный C#	Повторная генерация по seed и спецификации
Индивидуальные знания	Scope перед поиском	Тест исключения документа другого NPC
Управляемые последствия	NpcDecision и ActionDispatcher	Неверные amount, target и permissions
Несколько моделей	ModelTask и OrchestratorProfile	Проверка разных назначений маршрутов
Отказоустойчивость	Timeout, cancellation и bounded retry	Локальный HTTP-сервер ошибок
Диагностика	Результаты инструментов, PipelineTrace и Debug	Сопоставление плана, проверки и состояния Unity
Расширение C#	Просмотр исходника, импорт и события компилятора	Отдельный статус записи, компиляции и подключения

Нефункциональные требования связаны с сохранностью сцены и контролем ресурсов. Редакторский исполнитель действует только в активной загруженной сцене и не создаёт новую сцену вместо неё. Новые объекты помещаются в собственную группу WorldWeaverGenerated и получают маркер GeneratedSceneObject. Само совпадение имени с этой группой не подтверждает владение. Изменение существующего объекта требует явной цели; удаление разрешено только для собственных объектов без посторонних потомков. Повторная генерация Terrain дополнительно использует GeneratedTerrain. Локальные модели хранятся вне репозитория, а количество октав, размер ответа, число фрагментов и создаваемых объектов имеют явные пределы.

2.2 Структура Unity Package

Пакет размещён в каталоге Packages/com.worldweaver.toolkit. Файл package.json определяет имя, версию и зависимость от официального пакета Newtonsoft JSON. Сборка WorldWeaver.Runtime содержит модели данных, providers, RAG, действия, компоненты NPC и Terrain. Сборка

WorldWeaver.Editor содержит чат на UI Toolkit, контроллер сеанса, исполнитель инструментов, компоновщик окружения, расширенное окно настройки и средства исследования.

Разделение сборок не позволяет runtime-коду зависеть от UnityEditor. Это важно для дальнейшего использования игровых компонентов за пределами редактора. Средства измерений, создание ассетов и графический авторинг остаются в Editor. Тесты вынесены в отдельные сборки EditMode и PlayMode и подключаются через testables демонстрационного проекта.

Unity Package Manager поддерживает Git-зависимости и выбор подкаталога пакета через параметр path [2]. Такая структура позволяет держать программный продукт, демонстрацию и документы в одном репозитории, сохраняя независимую точку установки. Для локальной разработки используется относительная file-зависимость; она не должна ошибочно публиковаться как универсальный сетевой адрес.

Редакторский слой версии 1.1 разделён на представление WorldWeaverChatWindow, ChatController, SceneToolExecutor, WorldComposer, ChatHistoryStore и ScriptDrafts. Представление не исполняет команды самостоятельно. Контроллер управляет запросом и историей; исполнитель проверяет и применяет EditorPlan; компоновщик создаёт геометрию и ассеты; отдельный обработчик кода отслеживает компиляцию и восстановление файлов. Такое разделение позволяет подменять ILLMProvider в тестах без подмены реальных операций над сценой.

На рисунке показаны редакторский чат, контроллер, исполнитель инструментов, обработчик C# и компоненты генерации и runtime. Связи отражают разделение редакторского плана EditorPlan и игрового решения NpcDecision; подробный порядок раундов раскрывается далее в описании контрактов. Во время игры IntelligentNPC передаёт запрос в NpcPipeline, использующий retrieval, router и dispatcher независимо от конкретного поставщика inference.

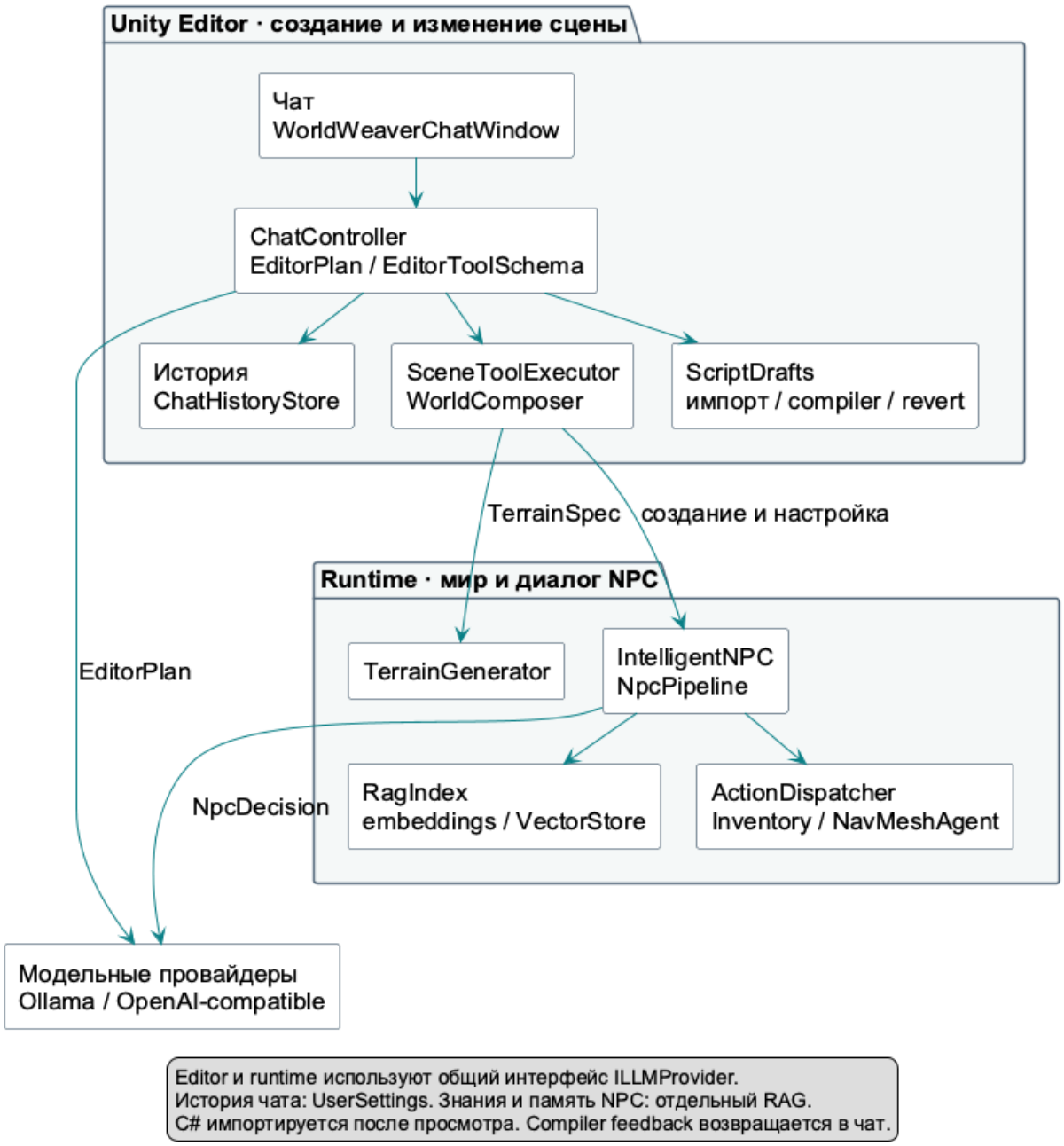


Рисунок 1 — Компоненты пакета WorldWeaver

2.3 Маршрутизация модельных задач

Редакторский агент использует выбранную для беседы модель. Настройка мощности изменяет бюджет обработки: число модельных раундов, лимит вывода, контекст и время ожидания. Режимы «Быстро», «Баланс» и «Глубоко» допускают соответственно один, три и шесть раундов. Это пределы ресурсов, а не доказанное ранжирование моделей по интеллекту или скорости. Цикл ограничен также общим числом инструментов на запрос.

После каждого применённого пакета следующий раунд получает фактический результат и свежий контекст сцены.

От этой настройки отделены маршруты игрового pipeline. В системе выделены три текстовые задачи: Terrain, Dialogue и Planning. Для каждой OrchestratorProfile содержит отдельную ссылку на ModelProfile. Метод Resolve проверяет наличие маршрута и возвращает явно назначенный профиль. Такая организация допускает разные модели и endpoints без изменения компонента NPC.

Terrain требует короткой интерпретации параметров. Dialogue формирует русскоязычную речь с учётом знаний и личности. Planning выбирает высокоуровневое действие на основе компактного состояния мира. Embeddings выделены в отдельный IEmbeddingProvider, поскольку их результат имеет другой тип и не должен обрабатываться как текстовый completion.

В смешанной демонстрационной конфигурации диалог выполняется бесплатной облачной моделью, а Terrain и Planning — локально. Причина связана с ограничением частоты анонимного API: отправлять в облако все вспомогательные задачи означало бы занимать редкие доступные интервалы запросов. Детерминированная проверка остаётся обычным C#-кодом и не требует ещё одной модели.

Такой router не является системой совместного обсуждения нескольких агентов. Он не осуществляет голосование или автоматическую оценку «умности» моделей. Его назначение — разделить задачи и сделать выбранный маршрут наблюдаемым. В текущей версии отказ провайдера не вызывает скрытого перехода на fixture или mock, поскольку это исказило бы демонстрацию и измерения.

Кэш экземпляров провайдеров внутри ModelRouter уменьшает число создаваемых HTTP-клиентов. Освобождение pipeline закрывает принадлежащие ему сетевые ресурсы. Перед сменой профиля

демонстрационный интерфейс освобождает прежние services; смена модели становится явным действием разработчика или пользователя demo.

2.4 Абстракция провайдеров и сетевой контракт

Интерфейс ILLMProvider содержит CompleteAsync, HealthAsync и Dispose. CompletionRequest включает сообщения и JSON Schema ожидаемого объекта. CompletionResult содержит текст, фактический provider/model, задержку и доступные счётчики токенов. При отсутствии метрики она не восстанавливается по предположению.

LocalProvider использует native API Ollama. Схема передаётся в format, streaming выключен, а для выбранной локальной модели отключается дополнительный thinking-вывод. Native API позволяет получить prompt_eval_count, eval_count и eval_duration. Эти данные нужны для измерения локальной скорости вывода и не подменяются скоростью сети [17].

OpenAICompatibleProvider реализует общий формат chat completions с отдельными особенностями Groq и Pollinations. Совместимость означает поддержку проверенного подмножества полей. Она не гарантирует идентичность всех API: путь получения моделей, имя параметра ограничения токенов и поддержка строгой схемы могут отличаться [21, 22].

ModelProfile хранит endpoint, model ID, timeout, число повторов, temperature и лимит токенов. Если нужен ключ, сериализуется только имя переменной окружения. Конструктор проверяет, что адрес использует HTTPS или локальный loopback HTTP, не содержит credentials, query и fragment. Автоматические redirects отключены, чтобы заголовок Authorization не передавался неожиданному хосту.

JsonTransport ограничивает объём ответа двумя мебибайтами. Чтение тела остаётся под действием timeout и cancellation token; иначе timeout только на получение заголовков не защитил бы от зависшего потока. Ошибки 400 и недоступная модель не должны повторяться как временная проблема. Для

429, 408 и серверных ошибок предусмотрены ограниченные повторы. Для HTTP 402 временным пределом считается только код FREE_LIMIT: без Retry-After ожидаются 46 секунд; платёжный механизм не вызывается.

Важна разница между транспортным успехом и прикладным. HTTP 200 подтверждает получение тела, но не валидность NpcDecision. EditorPlan и NpcDecision проверяются ещё раз после десериализации соответствующими прикладными валидаторами. При JSON Object Mode схема дополнительно описывается в system-сообщении, поскольку сам этот режим гарантирует только синтаксическую форму JSON, а не точные поля.

2.5 Архитектура знаний и памяти

KnowledgeBase представляет редактируемый ассет со списком KnowledgeDocument. Документ имеет идентификатор, Scope, заголовок, текст и MemoryKind. Chunk сохраняет идентификатор исходного документа, смещение, область видимости, текст и вектор. Такая структура обеспечивает связь найденного результата с авторским источником.

TextChunker использует окна по 500 символов с перекрытием 80 символов. Размер задан в символах, а не в токенах; это осознанное упрощение для короткого русского лора. Идентификатор chunk вычисляется через SHA-256 от scope, документа, смещения и содержимого. Повторная индексация одинакового текста даёт тот же идентификатор, а перенос текста другому персонажу меняет его.

Для получения векторов используется EmbeddingGemma. Модель предназначена для многоязычных текстовых представлений; в проверенном локальном запуске получены векторы размерности 768 [19]. Запрос и документ имеют разные префиксы, что соответствует поисковому назначению embedding-модели [20]. Документы не интерпретируются как инструкции разработчика.

Для векторов a и b косинусное сходство определяется как отношение скалярного произведения к произведению норм. Значение зависит от

направления векторов, а не от их абсолютной длины. Нулевые векторы обрабатываются отдельно, а несовместимая размерность и нечисловые значения отклоняются. Изменение embedding-модели требует перестроения индекса, поскольку сравнение в разных пространствах не имеет корректной интерпретации.

InMemoryVectorStore сначала ограничивает множество документов по Scope и MemoryKind, затем вычисляет сходство и выбирает top k. При равных score порядок стабилизируется идентификатором, чтобы исключить случайную перестановку результатов. Для N фрагментов размерности d стоимость одного полного поиска составляет O(Nd). Для демонстрационного корпуса эта сложность приемлема и проще для проверки, чем приближённый индекс.

RagIndex различает статические знания, эпизодические события и разговорную память. Для основной базы применяется настраиваемый top k, дополнительно извлекаются до двух эпизодических и двух разговорных фрагментов. При сборке контекста общий текст retrieved ограничивается 4000 символами. Это ограничение не является точным токеновым budget, поэтому максимальный размер всей prompt также зависит от описания личности и состояния мира.

Память ограничена 32 последними документами текущего сеанса. При превышении предела старый документ и соответствующие ему chunks удаляются. Индекс не является системой постоянного сохранения игры: завершение процесса уничтожает его содержимое. Статическая KnowledgeBase остаётся ассетом и может быть проиндексирована повторно.

Постоянная история редакторского чата хранится отдельно от памяти персонажа. ChatHistoryStore сохраняет беседы и настройки в UserSettings проекта, а в очередной запрос включается ограниченная часть последних сообщений и результатов инструментов. Эта история помогает продолжать авторинг; она не индексируется автоматически как личное знание каждого

NPC. Такое разделение предотвращает смешение разработческой переписки и игрового лора.

Перестроение статического индекса выполняется через временное хранилище. Новый backend становится активным после успешной обработки всех документов и проверки cancellation token. Это защищает предыдущий индекс от частичной замены при отмене. Ошибка индексации памяти после уже исполненного действия фиксируется отдельно и не должна превращать произошедшую передачу предмета в фиктивный откат.

Для поиска используется косинусная мера

$$c(q,d) = \frac{\sum_{i=1}^{D} q_i d_i}{\sqrt{\sum_{i=1}^{D} q_i^2} \cdot \sqrt{\sum_{i=1}^{D} d_i^2}}$$

Здесь q — вектор запроса, d — вектор фрагмента, D = 768 — размерность представления проверенной модели. После фильтрации результат представляет упорядоченный список пар «фрагмент, score». Значение score не является вероятностью истинности текста. Даже высокий показатель означает близость представлений, поэтому предметная достоверность должна оцениваться отдельно по ответу и источнику.

При N фрагментах и d координатах память самих векторов имеет порядок 4Nd байт для float32, без учёта строк и объектов среды выполнения. Например, для 1000 фрагментов размерности 768 числовая часть занимает примерно 3,07 МБ. Это расчёт объёма представления, а не измерение полного потребления процесса. Такая оценка объясняет выбор простого точного поиска для небольшого корпуса и необходимость другого backend при существенно большем N.

Граница Scope относится к доступу к данным, а не к качеству ранжирования. Если запрос совпадает со служебным паролем другого персонажа, этот документ всё равно исключается до вычисления сходства. Проверка scope входит как в формирование индекса NPC, так и в контракт

поиска. Двойное ограничение защищает от случайного смешивания документов при дальнейшем расширении хранилища.

2.6 Спецификация и генерация Terrain

TerrainSpec содержит seed, физические размеры, разрешение, параметры fBm, силу горного края, глубину долины, уровень воды, южное понижение и число проходов сглаживания. Текущая версия использует один тип шума и не предоставляет произвольные карты биомов или гидравлическую эрозию. Выбор ограниченного набора параметров уменьшает неоднозначность JSON-контракта.

Таблица 4 — Спецификация и генерация Terrain

Поле	Смысл	Допустимая область
seed	Начальное условие шума	Целое число
width и length	Размер поверхности	От 32 до 2048 единиц Unity
height	Масштаб высоты	От 4 до 512
resolution	Размер регулярной сетки	33, 65, 129, 257 или 513
octaves	Число шумовых октав	От 1 до 8
mountainStrength	Выраженность горного края	От 0 до 1
valleyDepth	Центральное понижение	От 0 до 0,8
waterLevel	Нормированная высота воды	От 0 до 0,8
smoothingPasses	Проходы сглаживания	От 0 до 8

Валидация проверяет числовые типы и диапазоны до выделения карты. Например, дробное resolution не округляется в допустимое целое, а считается нарушением контракта. Это важно, поскольку автоматическое преобразование типов могло бы скрыть ошибку модели и затруднить сопоставление запроса с фактическим результатом.

Шумовая функция использует целочисленный hash координат и seed, сглаженную интерполяцию соседних значений и ограниченное число октав. После базового fBm к поверхности добавляется радиальный горный край и

вычитается центральная долина. Множитель southSlope понижает южную часть края. Результат нормируется в диапазон, допустимый для TerrainData.

Сглаживание выполняется синхронно: каждый проход читает предыдущую карту и пишет новую. Это исключает зависимость результата от порядка обхода обновляемых ячеек. Используемый фильтр не моделирует перенос воды или осадка, поэтому в документации он не называется физической эрозией.

Для чата инструмент create_world принимает частичную спецификацию, дополняет отсутствующие значения установленными значениями по умолчанию и передаёт результат тому же TerrainGenerator. WorldComposer добавляет слои поверхности, воду, растительность, дома, освещение и камеру. Пресеты alpine, forest и desert задают параметры и цветовую палитру, а не обещают физически достоверные биомы. Распределение декора использует seed, высоту Terrain, ограничения уклона, воды и расстояния между объектами. Если запрошенное число объектов невозможно разместить в пределах попыток, операция завершается ошибкой с откатом, а не объявляет неполный набор успешным.

Инструмент update_world изменяет только собственный мир или Terrain: копирует TerrainData в новый ассет, применяет частичные параметры и обновляет высоты, слои, collider и воду. Помеченные объекты окружения привязываются к новой поверхности. Прежний TerrainData не изменяется, что защищает другие ссылки на него. Отдельный Regenerate расширенного инспектора сохраняет механизм Undo для GeneratedTerrain [25]. В обоих путях отмена сцены после успешной генерации может оставлять новые ассеты в Project; файловый результат не следует считать полностью отменённым только по исчезновению GameObject.

Расчёт нормированного шума при O октавах имеет вид

$$F(u,v) = \frac{\sum_{o=0}^{O-1} a_o N(f_o u, f_o v, s_o)}{\sum_{o=0}^{O-1} a_o}$$

Амплитуда начинается с единицы и умножается на persistence после каждой октавы, частота начинается со scale и умножается на lacunarity. Seed октавы равен seed + 1013o с переполнением в установленной целочисленной арифметике. Такая конкретизация нужна для воспроизведения алгоритма, поскольку одно название fBm не определяет реализацию шума.

Финальная высота до ограничения диапазона получается суммой базового уровня 0,20, шумового вклада 0,24F, горного края с южным коэффициентом и отрицательного вклада долины. После этого применяется Clamp01. WaterLevel не изменяет поле высот: он определяет высоту дополнительной визуальной плоскости. Поэтому изменение уровня воды и изменение формы дна являются разными операциями.

Для внутренних ячеек проход сглаживания p выражается формулой

$$h^{p+1} = 0{,}65\,h^{p} + 0{,}35\,avg_4\left(h^{p}\right)$$

Здесь avg_4 обозначает среднее четырёх соседей. Край массива в проходе сохраняется. Алгоритм является локальным усреднением и не моделирует перенос массы или потоки воды. При описании результатов термин «эрозия» для этой операции не используется.

2.7 Контракты редакторского агента и игровых действий

Редакторский план EditorPlan содержит reply, complete и массив actions. Поле reply предназначено для объяснения, complete обозначает завершение задачи на текущем шаге. Команда EditorCommand имеет tool, name, target и args. Перечень имён инструментов закрыт; типы, диапазоны и допустимые ключи args проверяются отдельно для каждой операции. Лишние поля и повторённые JSON-свойства отклоняются. План с пустым actions допускает объяснение без изменения сцены.

Таблица 5 — Контракты редакторского агента и игровых действий

Группа	Инструменты	Изменяемый или читаемый результат
Окружение	create_world, update_world, scatter_objects	Terrain, вода и собственный декор

Группа	Инструменты	Изменяемый или читаемый результат
Объекты сцены	create_object, set_transform, set_material, delete_object	Компоненты и параметры явно указанных целей
Свет и обзор	set_lighting, create_camera	Источники света и камеры
Персонажи	create_npc, update_npc, attach_behavior	Runtime-компоненты, знания и встроенное поведение
Наблюдение	inspect_object, search_project, read_script	Сведения без изменения сцены
Сохранение	save_scene	Копия сцены по разрешённому уникальному пути

Контекст ограничен и содержит идентификаторы и параметры не более 80 объектов, выделение, настройки Terrain и NPC, существующие собственные группы и доступные инструменты. Цель вида id:число относится к объекту текущего сеанса Editor; это не постоянный идентификатор после перезапуска проекта. Ссылка @Name разрешает обратиться к более раннему созданию того же плана. Валидатор строит виртуальный набор целей, поэтому корректная последовательность «создать — настроить» допускается без предварительной мутации. Ссылка на будущее или уже удалённое создание отклоняется.

До исполнения проверяется весь пакет. В один план допускается не более 24 команд, в одно пользовательское обращение — не более 48 вызовов инструментов. Распределение ограничено 300 размещениями; полный бюджет учитывает дочерние GameObject и составляет 600 новых объектов на пакет и 2000 собственных объектов активной сцены. Дерево, дом и персонаж поэтому учитываются по составу, а не как одна строка в плане. Числовые параметры должны быть конечными; пути не допускают выхода из разрешённого каталога.

При изменении сцены создаётся одна группа Undo. Исполнитель ведёт список вновь созданных ассетов и возвращает идентификаторы объектов и прочитанные параметры. При ошибке исполнения отменяется группа сцены и

удаляются только файлы, зарегистрированные текущим пакетом; ошибка очистки также включается в результат. Операции inspect_object, search_project и read_script не создают Undo-группу. Успешные несколько раундов образуют несколько завершённых пакетов, поэтому отмена последнего пакета не равна откату всей беседы. Остановка ожидания после уже завершённого раунда не отменяет этот раунд автоматически.

Изменение материала создаёт отдельный Material, сохраняя чужой sharedMaterial. Новые ассеты записываются в уникальный подкаталог Assets/WorldWeaverGenerated/Assistant. Сохранение сцены выполняется только явной последней командой save_scene по уникальному пути внутри Assets/WorldWeaverGenerated; существующие файлы и список сборки не перезаписываются. Успешный Undo относится к объектам сцены: оставшиеся после него ассеты требуют отдельного учёта. Таким образом, откат ошибочного пакета и отмена ранее успешного действия имеют различную файловую семантику.

Генерация C# вынесена в отдельный контракт. draft_script предлагает полный исходник MonoBehaviour, который всегда показывается для просмотра перед импортом, даже при включённом автоматическом применении обычных инструментов. Допускается создание или исправление только собственных помеченных файлов Assets/WorldWeaverGenerated/Scripts. После импорта фиксируются реальные события компилятора Unity. Только отдельная команда attach_script может подключить уже скомпилированный компонент к явно указанному объекту через Undo. План кода не смешивается с пакетом обычных изменений сцены, поскольку компиляция и перезагрузка домена имеют иной жизненный цикл.

Предварительный фильтр ограничивает имена, размер исходника, Editor-only конструкции и известные шаблоны credentials. Он не является формальным анализатором безопасности. Компилятор проверяет синтаксис, типы и ссылки, но не доказывает правильность игрового поведения и не создаёт sandbox. Сгенерированный компонент после просмотра исполняется

с правами процесса Unity. Поэтому этот путь требует содержательной проверки кода; его нельзя описывать как безопасное исполнение произвольной программы. Каждый импорт получает отдельную постоянную запись журнала, связанную с беседой и сообщением: версии собственных файлов до и после импорта, контрольные суммы и исход компиляции. Второй импорт B сохраняет запись первого импорта A. Кнопка возврата кода у соответствующего сообщения восстанавливает файлы только при совпадении текущего содержимого с версией после этого импорта. Для последовательных изменений одного файла сначала возвращается B, затем A; попытка возврата A поверх B отклоняется. Такая проверка также сохраняет последующие ручные правки пользователя. Возврат файлов проходит повторную компиляцию и учитывается отдельно от обычного Undo сцены.

Игровой контракт отделён от редакторского. Ответ NPC имеет два обязательных поля: speech и actions. Каждая команда содержит type, target, item и amount. Даже для действия без предмета поля сохраняются: item и target при необходимости равны пустой строке, amount равен нулю. Единый DTO упрощает совместимость разных JSON-режимов.

NpcDecision.Parse отклоняет неизвестные и дублирующиеся поля, неправильные типы, чрезмерно длинные строки и массивы более четырёх действий. Структурная схема полезна для генерации, но не заменяет прикладные проверки. JSON Schema задаёт форму объекта, а право на изменение инвентаря определяется текущим миром [26].

ActionDispatcher хранит реестр IGameAction. Для встроенных операций используются TransferAction и WorldAction. Добавление нового обработчика через Register не требует изменения router. Пользовательский обработчик обязан выполнять Validate без изменения сцены; нарушение этого контракта делает общую предварительную проверку бессмысленной.

Перед исполнением решения dispatcher формирует словарь явно известных NPC сущностей. Неизвестный target и повторяющийся entity ID приводят к отказу. Для передачи предметов создаётся проекция каждого

затронутого инвентаря. Она резервирует количество и вместимость до реального изменения объектов. Так два действия по шесть кристаллов не могут оба пройти при исходном остатке десять.

Количество передаваемых предметов ограничивается не только остатком, но и общим MaxTransferPerDecision. Проверяются дальность взаимодействия и разрешение TakeItem. Текстовое ограничение «не отдавай много» не считается заменой этих полей. Для перемещения проверяется наличие агента на NavMesh и полный путь к достижимой точке.

После успешного предконтроля действия выполняются последовательно без await в главном потоке. Это предотвращает изменение состояния между проверкой и записью из-за другого продолжения async-метода. Для встроенных инвентарных операций такой порядок обеспечивает согласованную передачу. Полный rollback произвольных пользовательских действий во внешних системах текущий интерфейс не гарантирует.

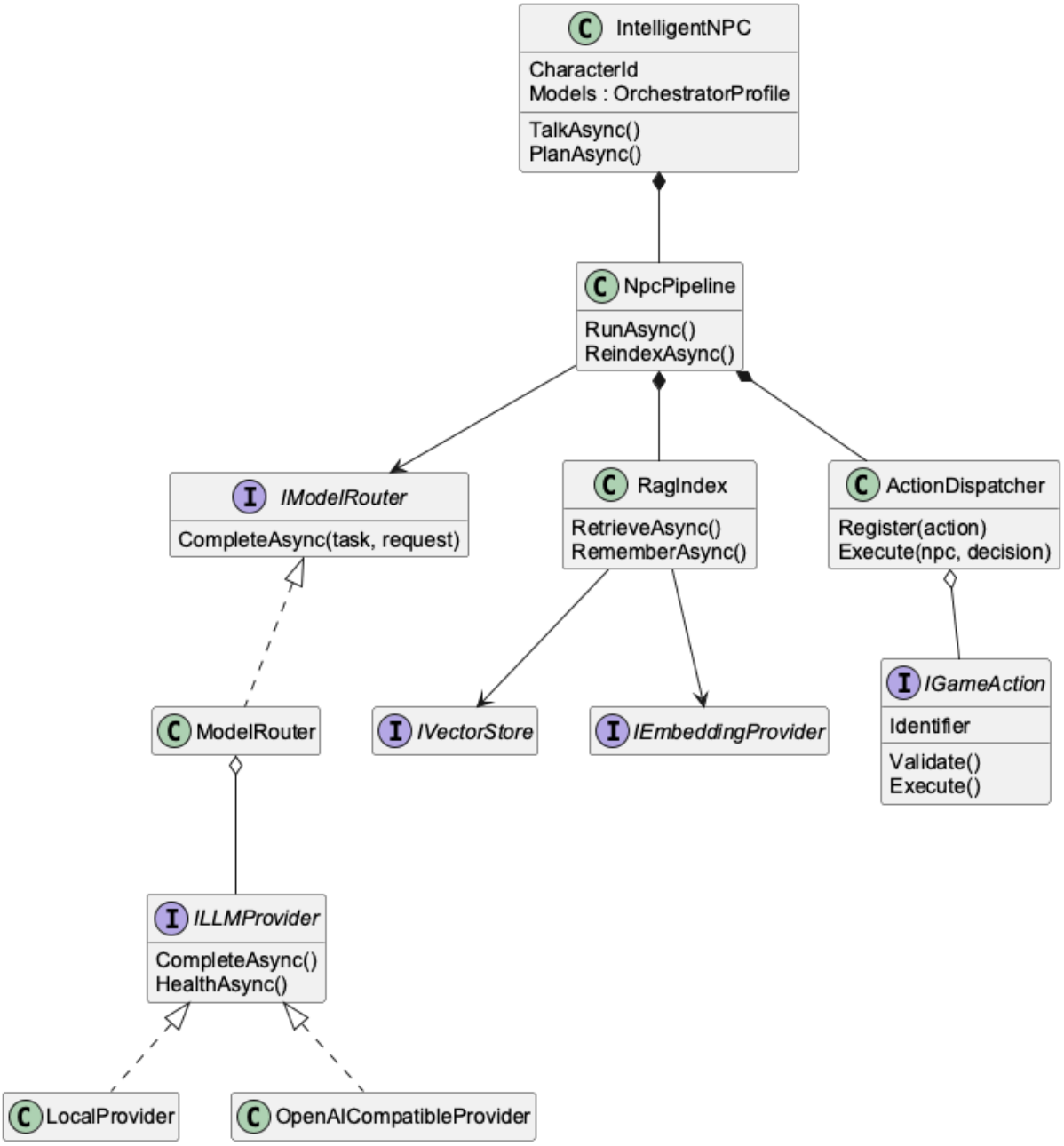


Рисунок 2 — Классы runtime и точки расширения

2.8 Диаграммы процессов и потоков данных

Следующие диаграммы описывают базовый контур Terrain и NPC, используемый также новой версией; полный цикл редакторского чата рассмотрен текстовым контрактом выше. Диаграмма прецедентов разделяет действия разработчика и игрока. Разработчик настраивает пакет и исследует traces; игрок участвует в диалоге. Автономное планирование использует тот же контур проверки действий, поэтому отдельный неконтролируемый путь изменения мира не создаётся.

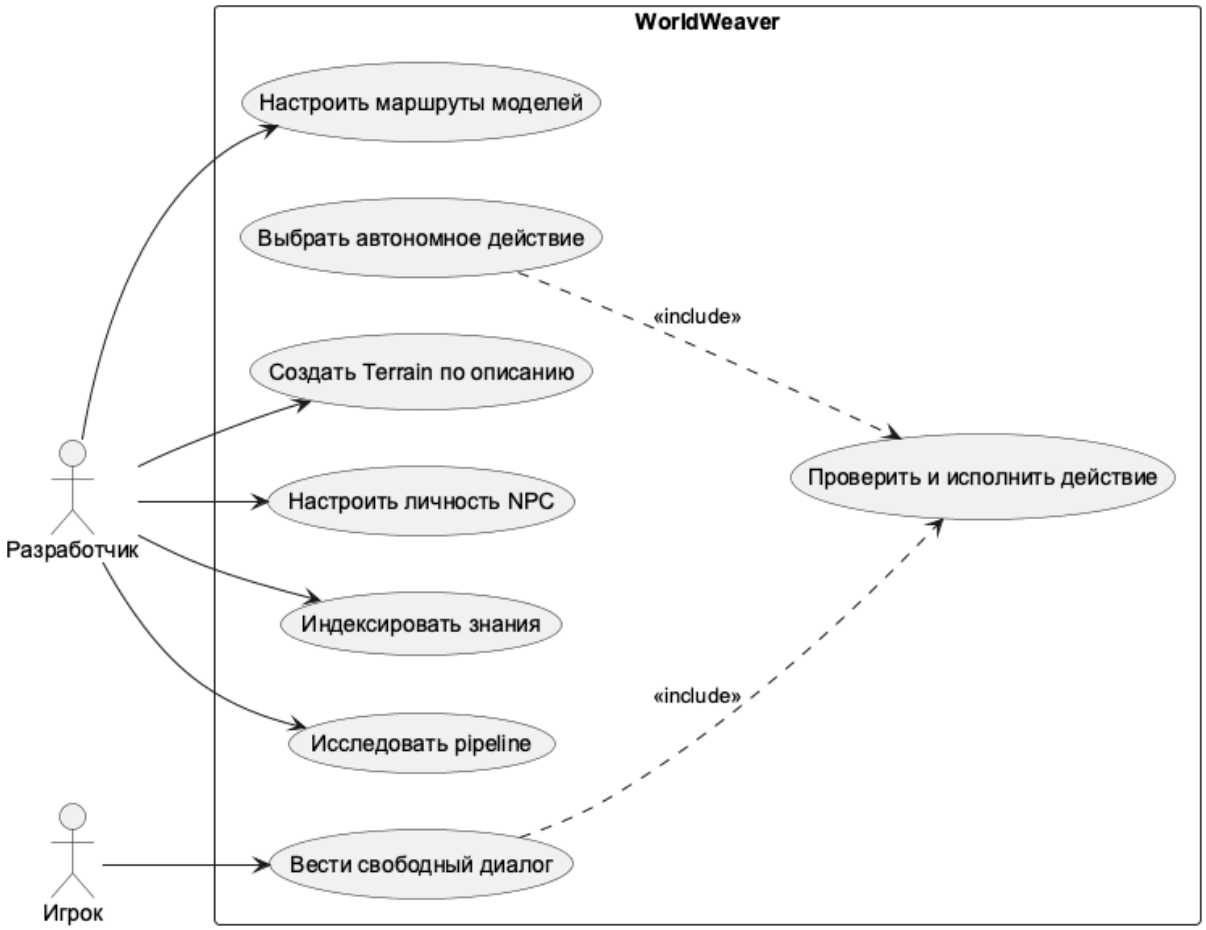


Рисунок 3 — Прецеденты использования инструментария

На диаграмме последовательности показан разговор с передачей предмета. Retrieval предшествует запросу модели, а инвентарь изменяется после разбора и валидации. Запись памяти содержит результат исполнения, а не одно лишь намерение модели.

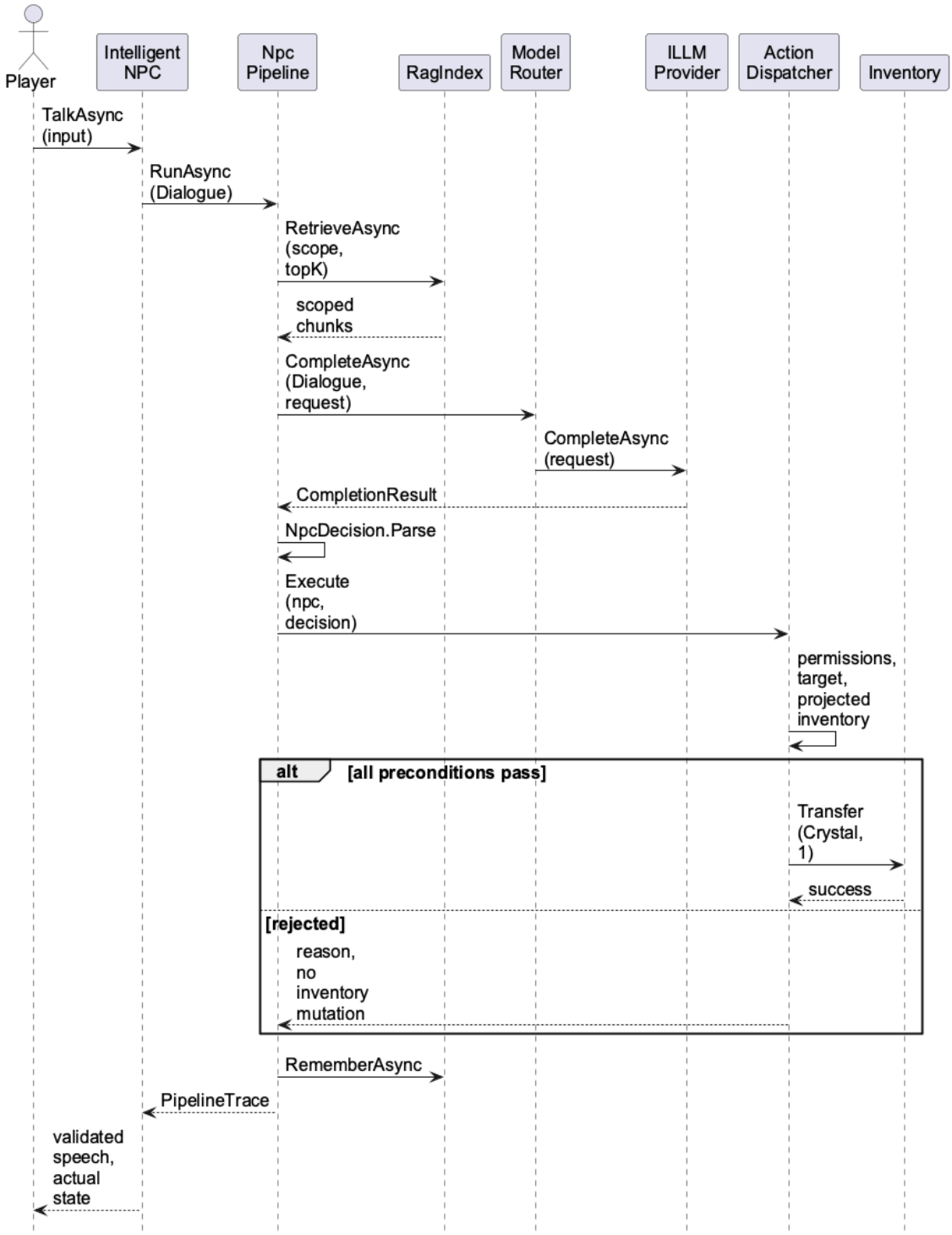


Рисунок 4 — Последовательность обработки диалога

Диаграмма потоков данных выделяет внешние inference-сервисы, хранилище знаний, состояние сцены и журнал. Через сетевую границу проходят текстовые данные и структурированные результаты. Внешний сервис не записывает Inventory или TerrainData непосредственно.

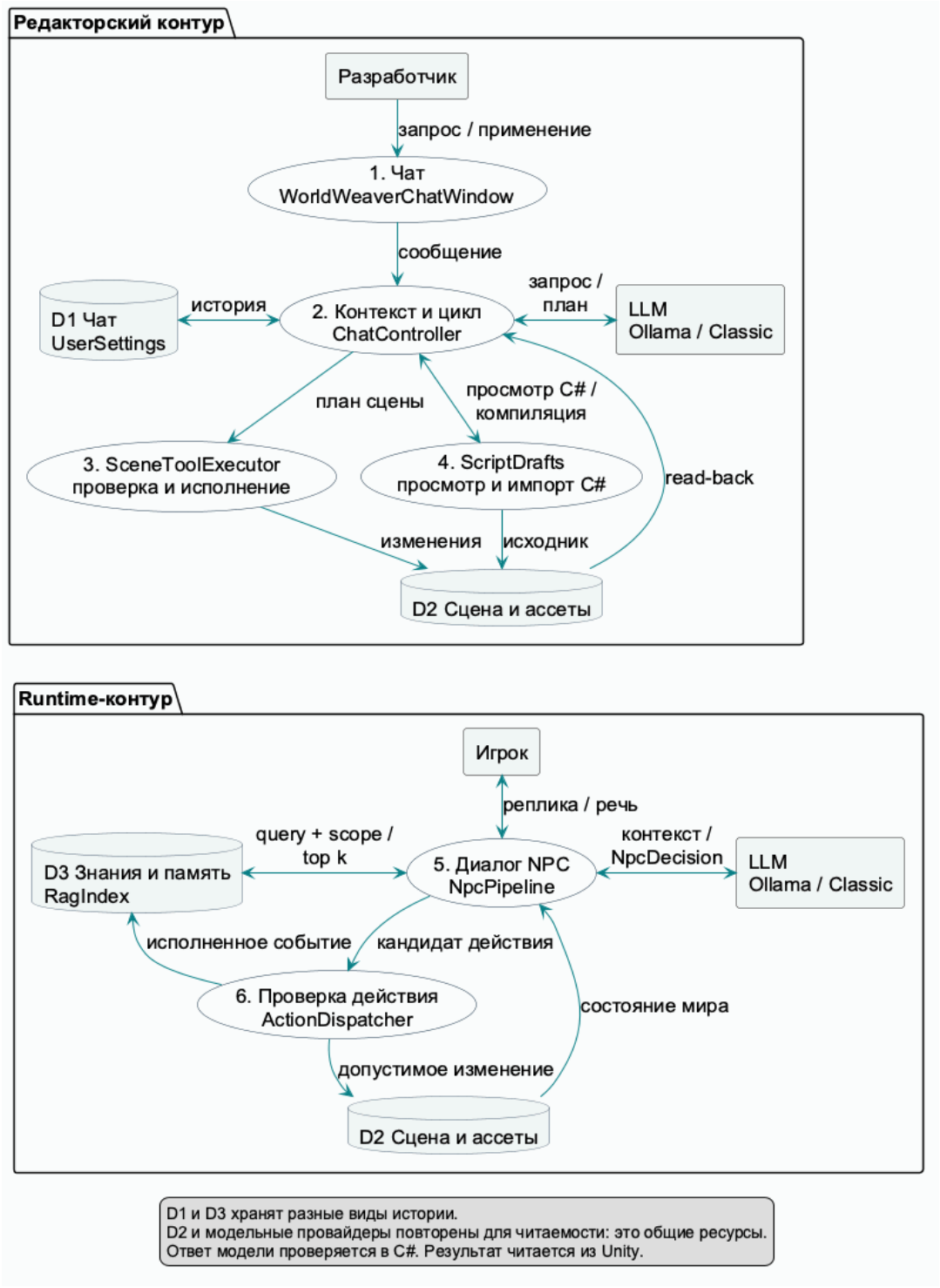


Рисунок 5 — Потоки данных WorldWeaver

2.9 Выбор локальной среды и ограничения развёртывания

Для локального inference рассмотрены Ollama, LM Studio и llama.cpp. Все три подхода позволяют организовать локальное исполнение, однако отличаются управлением моделями и API [17, 33, 34]. На машине уже был установлен Ollama, поэтому его использование уменьшило объём изменений

среды и позволило проверить реальные запросы через один локальный daemon.

Apple M4 имеет объединённую память, которую одновременно используют Unity, операционная система и inference. Выбор моделей для базовой runtime-конфигурации версии 1.0 учитывал это ограничение. Установка модели на 120 миллиардов параметров не соответствует доступным 16 ГБ. Поэтому для коротких запросов Terrain и QA выбрана небольшая Qwen3.5 2B и отдельная embedding-модель; для итогового Dialogue и Planning дополнительно используется квантованная qwen2.5-coder 7,6B. Выбор основан на возможности совместного запуска, а не на попытке воспроизвести cloud-модель на любом оборудовании [18].

Бесплатный Pollinations API не требует ключа, но ограничивает частоту и не гарантирует доступность [23]. Это делает его пригодным для демонстрации и сравнительного опыта при соблюдении интервалов, но не позволяет обещать фиксированную задержку в коммерческой игре. Локальная конфигурация остаётся обязательным воспроизводимым вариантом.

Рисунок показывает физическое развёртывание редакции 1.1. Unity содержит редакторский чат и runtime-компоненты; Ollama выполняется отдельным локальным процессом и вызывается по loopback HTTP. Начальная модель чата — Qwen3.5 9B, остальные установленные текстовые модели доступны для выбора, а EmbeddingGemma формирует векторы для retrieval. Перечисление моделей не предполагает одновременную загрузку всех весов.

История бесед и постоянный журнал импортов C# с предыдущими версиями файлов хранятся в UserSettings проекта. Созданные Terrain, материалы, сцены, знания NPC и просмотренные скрипты записываются в Assets/WorldWeaverGenerated. Необязательный маршрут Pollinations Classic использует HTTPS: текущий серверный предел составляет один ответ за 45 с на IP, клиентский интервал — 46 с, max_tokens — 1500 [39]. Исходное QA-исследование версии 1.0 сохраняет собственные модели и интервал 31 с в

протоколе четвёртой главы; эти исторические условия не обозначают текущую квоту сервиса.

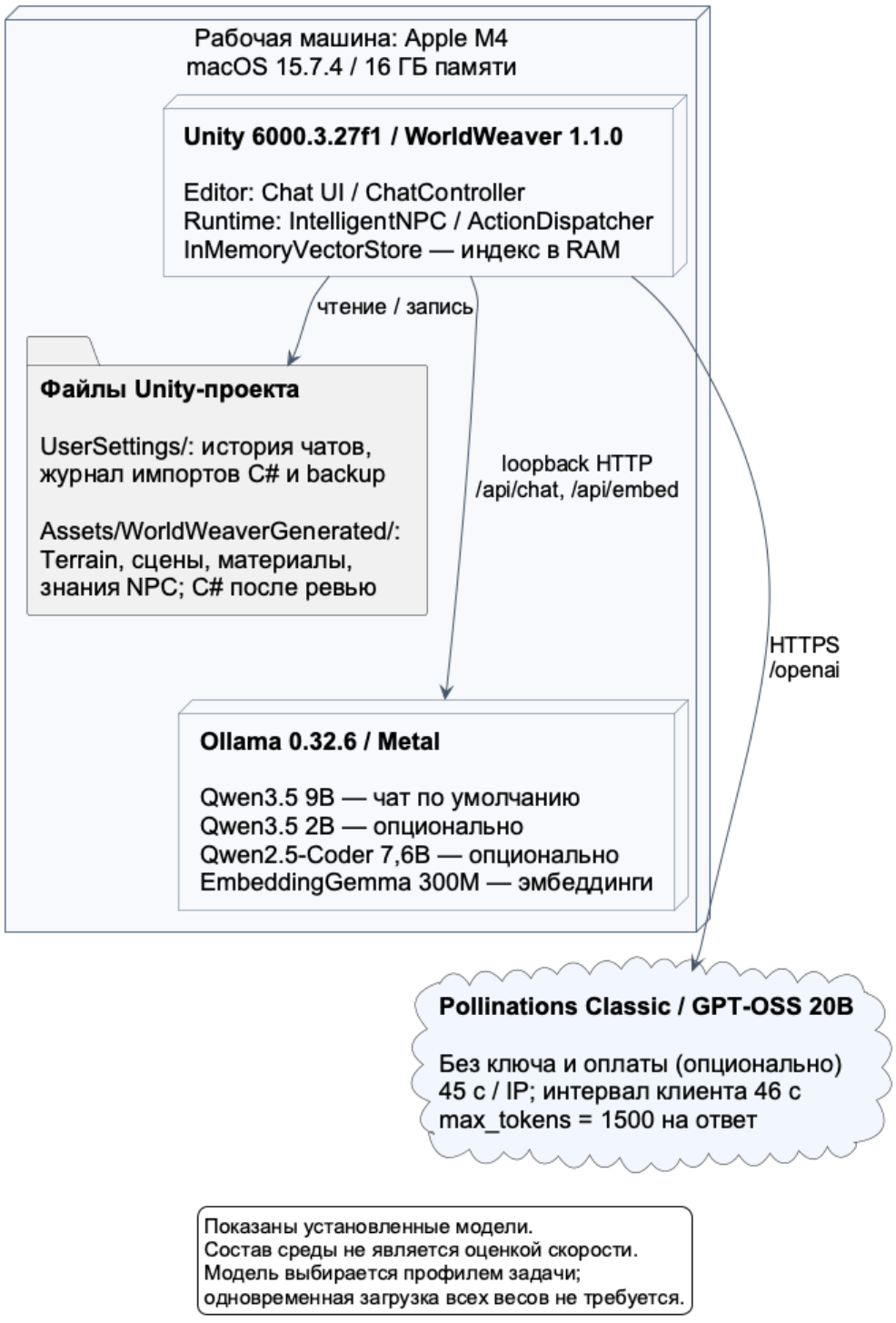


Рисунок 6 — Развёртывание редакторских, игровых и модельных компонентов WorldWeaver 1.1

Проектная архитектура задаёт наблюдаемую границу ответственности: редакторская модель предлагает EditorPlan, исполнитель возвращает фактическое состояние и результат; runtime-retrieval выбирает доступные персонажу факты, а ActionDispatcher проверяет игровые последствия.

Отдельный путь просмотренного C# расширяет возможности продукта и требует отличающихся правил проверки и восстановления. Следующая глава описывает, как эти границы реализованы в программных компонентах.

3 РЕАЛИЗАЦИЯ И ДЕМОНСТРАЦИЯ

3.1 Среда разработки и состав поставки

Основная реализация версии 1.1 выполнена на C# для Unity 6000.3.27f1. Эта LTS-версия установлена из официального архива с учётом требования Unity 6+ в ранее подготовленном техническом задании [1]. Изначально на компьютере присутствовала Unity 2022.3.62f3; проверенные в ней компиляция и EditMode-тесты относились к базовому ядру 1.0. Новый чат, его инструменты и процесс импорта C# проверяются в Unity 6000.3; полная совместимость 1.1 с Unity 2022 и другими платформами из прежних испытаний не выводится.

Аппаратная среда включает Apple M4 и 16 ГБ объединённой памяти, операционная система — macOS 15.7.4. Локальный daemon Ollama имеет версию 0.32.6. Эти сведения нужны для интерпретации времени ответа: локальная производительность зависит от ресурсов, одновременно занятых редактором, моделей в памяти и прогрева inference.

Runtime-пакет разделён на каталоги Core, Providers, Orchestration, RAG, NPC, Actions, Dialogue, Terrain и Assistant. Последний содержит маркер сгенерированных объектов, готовые поведения и панель проверки персонажей в Play Mode. В Editor/Assistant находятся чат, контроллер, исполнитель инструментов, компоновщик мира, история и обработчик просмотренного C#. WorldWeaverWindow, TerrainAuthoring и DemoBuilder сохраняются как расширенные и демонстрационные средства. Исходники диаграмм, экспериментальные данные и текст работы хранятся рядом с кодом, но не входят в устанавливаемую runtime-сборку.

Версия 1.1 добавляет редакторский агент поверх ранее реализованного ядра. Исходный QA-протокол и результаты версии 1.0 сохранены: изменение интерфейса не превращает прежние ответы на вопросы о лоре в испытание качества нового чата. Общие методы retrieval, генерации высот и runtime-валидации продолжают использоваться в обоих процессах.

Для десериализации применяется официальный Unity-пакет Newtonsoft JSON. Он выбран из-за необходимости проверять произвольное JSON-дерево, типы токенов и повторение имён полей. Простое отображение JSON в сериализуемый объект не обеспечило бы всех проверок, поскольку неизвестные или ошибочно приведённые поля могли бы остаться незамеченными.

Ключи поставщиков не копируются в репозиторий. Файл .env.example перечисляет имена переменных без значений. Настоящий .env остаётся вне проекта, а launcher при необходимости передаёт окружение процессу Unity. Проверенная бесплатная демонстрация не требует ключей для локального Ollama и Pollinations.

3.2 Единый чат в Unity Editor

Основной интерфейс версии 1.1 — WorldWeaverChatWindow на UI Toolkit, открываемый через Tools → WorldWeaver → Чат. Пользователь работает в одной беседе: задаёт вопрос о сцене, просит создать мир или персонажа, уточняет параметры и видит исполненные инструменты рядом с ответом. Ручное копирование JSON между разделами не требуется. Точное редактирование параметров остаётся доступным через обычные компоненты Unity и расширенный инспектор.

В заголовке размещены выбор модели и режимы «Быстро», «Баланс» и «Глубоко». Каталог локальных моделей обновляется через Ollama; embedding-модели исключаются из списка текстовой генерации. Выбор мощности управляет бюджетом раундов, объёмом контекста сцены и ожиданием. Для локального маршрута также увеличивается лимит вывода; бесплатный облачный маршрут сохраняет предел max_tokens=1500 во всех режимах. Увеличение бюджета даёт агенту больше возможностей прочитать результат и исправить план, но само по себе не подтверждает более высокое качество ответа. Статус доступности модели отделён от результата конкретного inference-запроса.

Кнопка «+» у поля ввода предлагает задачи «Мир», «Персонаж», «Изменить сцену», «Написать скрипт» и прикрепление контекста выделения. Эти действия подготавливают обычное сообщение в том же чате. Боковая панель содержит историю и создание новой беседы; для открытых бесед сохраняются черновики и положение прокрутки. Enter отправляет сообщение, Shift+Enter вводит новую строку. Во время ожидания доступны остановка запроса и обновляемый таймер прошедшего времени рядом с текущим этапом. Таймер показывает длительность ожидания, а не прогноз завершения.

ChatController собирает ограниченный контекст сцены и последнюю часть переписки, вызывает ILLMProvider и разбирает EditorPlan. После предконтроля SceneToolExecutor применяет пакет и возвращает результаты: фактические цели, параметры и созданные ассеты. При наличии бюджета следующий модельный раунд получает новое состояние для проверки исходного задания, даже если прежний план содержал complete:true. Это поле обозначает намерение завершить задачу и не считается доказательством результата. Модель может применить недостающую корректировку или завершить цикл пустым actions с complete:true. Если последним доступным раундом был исполнен инструмент, статус «Шаг выполнен» сообщает об исчерпании бюджета до дополнительной модельной проверки. В режиме «Быстро» с одним раундом второй проверочный вызов не обещается; работу можно продолжить следующим сообщением.

Неверный план не исполняется частично ради сохранения видимости успеха. Контроллер сообщает модели причину отказа и при наличии бюджета запрашивает исправление. Текст ответа рассматривается как объяснение или намерение; подтверждением изменения служит результат инструмента. Поэтому «план отклонён», «шаг выполнен» и «задача завершена» представлены различными состояниями.

Переключатель «Применять изменения автоматически» определяет, исполняются ли обычные допустимые планы сразу. Если он выключен, план

остаётся в истории до явного применения. Сгенерированный C# и подключение такого компонента требуют просмотра независимо от этого переключателя. Кнопка «Отменить действие» вызывает стандартную отмену Unity, которая может затронуть последнее ручное действие. Для кода предусмотрена отдельная кнопка «Вернуть предыдущий код» в сообщении, которому принадлежит доступный сохранённый импорт; она восстанавливает файлы и запускает повторную компиляцию. Остановка запроса и Undo не объединены: уже выполненные шаги остаются перечисленными, даже когда пользователь прервал дальнейшее ожидание.

После применения или возврата C# сообщение сохраняет снимок кода для чтения и копирования из принятой попытки истории. Этот снимок может отличаться от текущего файла проекта. Кнопка применения относится только к действующему PendingPlan; просмотр исторического исходника не импортирует его повторно.

Для всех провайдеров явно распознанный запрос «только объясни» допускает лишь inspect_object, search_project и read_script. Команды изменения, draft_script и attach_script отклоняются до исполнения, не заменяя прежний ожидающий код. Распознавание учитывает прямую формулировку текущей просьбы, исключает слова внутри кавычек и смешанные задания с явным изменением; оно не является универсальным классификатором намерений. Повтор после отклонения получает исходное актуальное задание, а не только прошлую ошибку.

Представление подписывается на события контроллера, но обновляет визуальное дерево через EditorApplication.update. Асинхронное продолжение лишь отмечает необходимость обновления; оно не изменяет UI Toolkit из произвольного потока. При закрытии окна обработчики снимаются, а текущий контроллер освобождается.

WorldWeaverWindow сохранён как Tools → WorldWeaver → Advanced Inspector. Его разделы WORLD, NPC, KNOWLEDGE, MODELS и DEBUG позволяют вручную проверить TerrainSpec, личность и числовые

ограничения, назначенные маршруты, исходные документы и PipelineTrace. При отсутствующем обязательном маршруте показывается предупреждение. Импорт знаний принимает TXT, Markdown и установленный JSON, ограничивает размер файла одним мебибайтом и назначает Scope выбранного NPC; PDF не выдаётся за проиндексированный документ. Regenerate ограничен GeneratedTerrain и использует Undo. Это дополнительный диагностический интерфейс, а не обязательная последовательность экранов для нового пользователя.

Для локального декодера порядок свойств также является инженерным ограничением. Формальная JSON Schema не задаёт порядок ключей объекта, однако её преобразование в грамматику llama.cpp сохраняет объявленную последовательность и ставит обязательные поля перед необязательными [40, 41]. Поэтому порядок аргументов согласован с примерами, а модель получает короткую подсказку о допустимой последовательности. Иначе после style декодер мог уже не разрешить возвращение к width. Это не изменение семантики полного контракта: те же поля и границы проверяются в C#.

Числовые ограничения грамматики также неполны. Документация llama.cpp отдельно указывает, что minimum и maximum поддерживаются для integer, но не для number [40]. Следовательно, типизированный вывод ещё не доказывает, что нормализованный waterLevel находится в пределах от 0 до 0,8. Дробные параметры, бюджеты и ссылки повторно проверяются перед изменением сцены. Результат контролируемого запроса до и после согласования порядка рассматривается отдельно от приёмки всего агента.

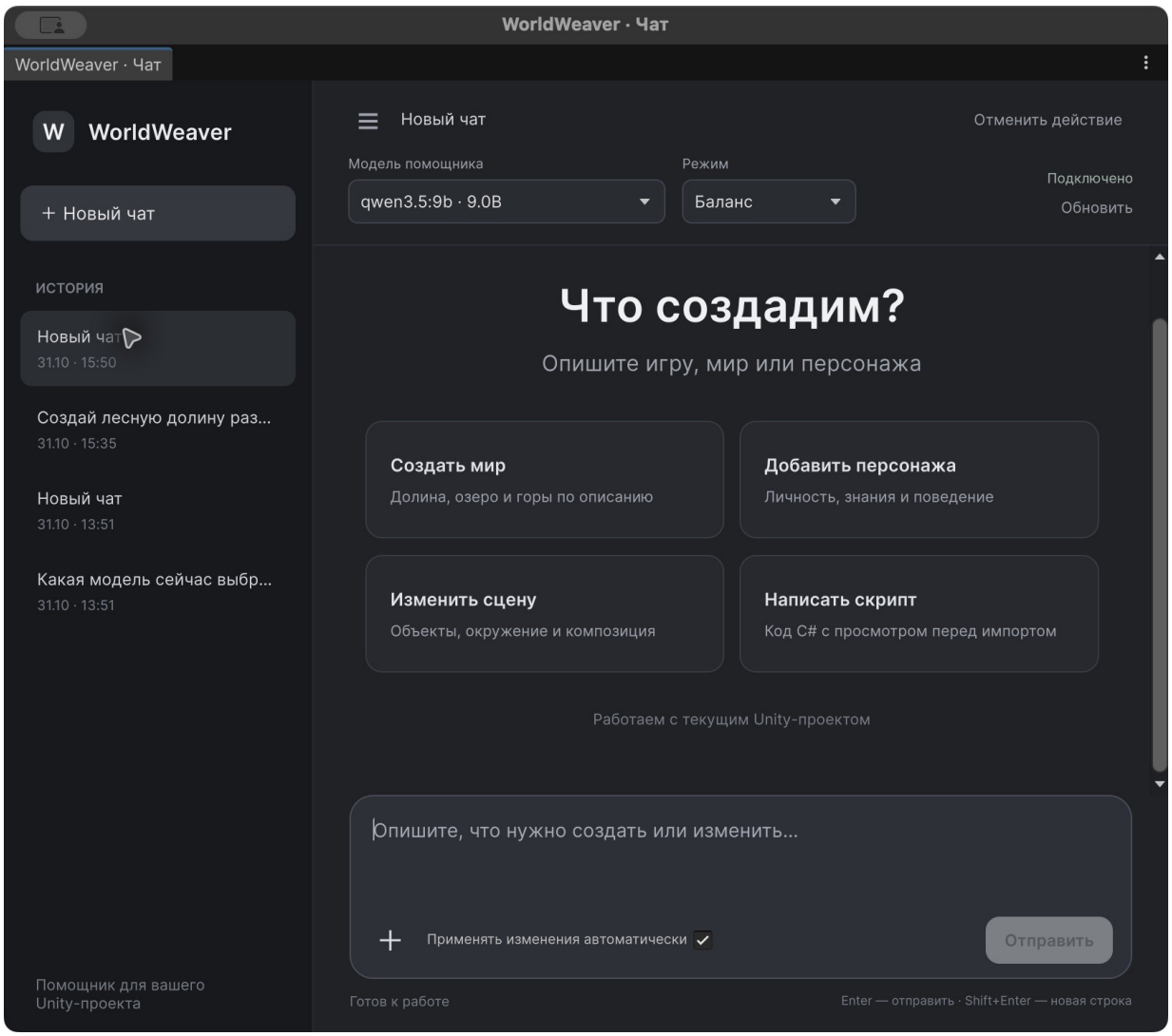


Рисунок 7 — Основной интерфейс WorldWeaver с выбором модели и свободным вводом

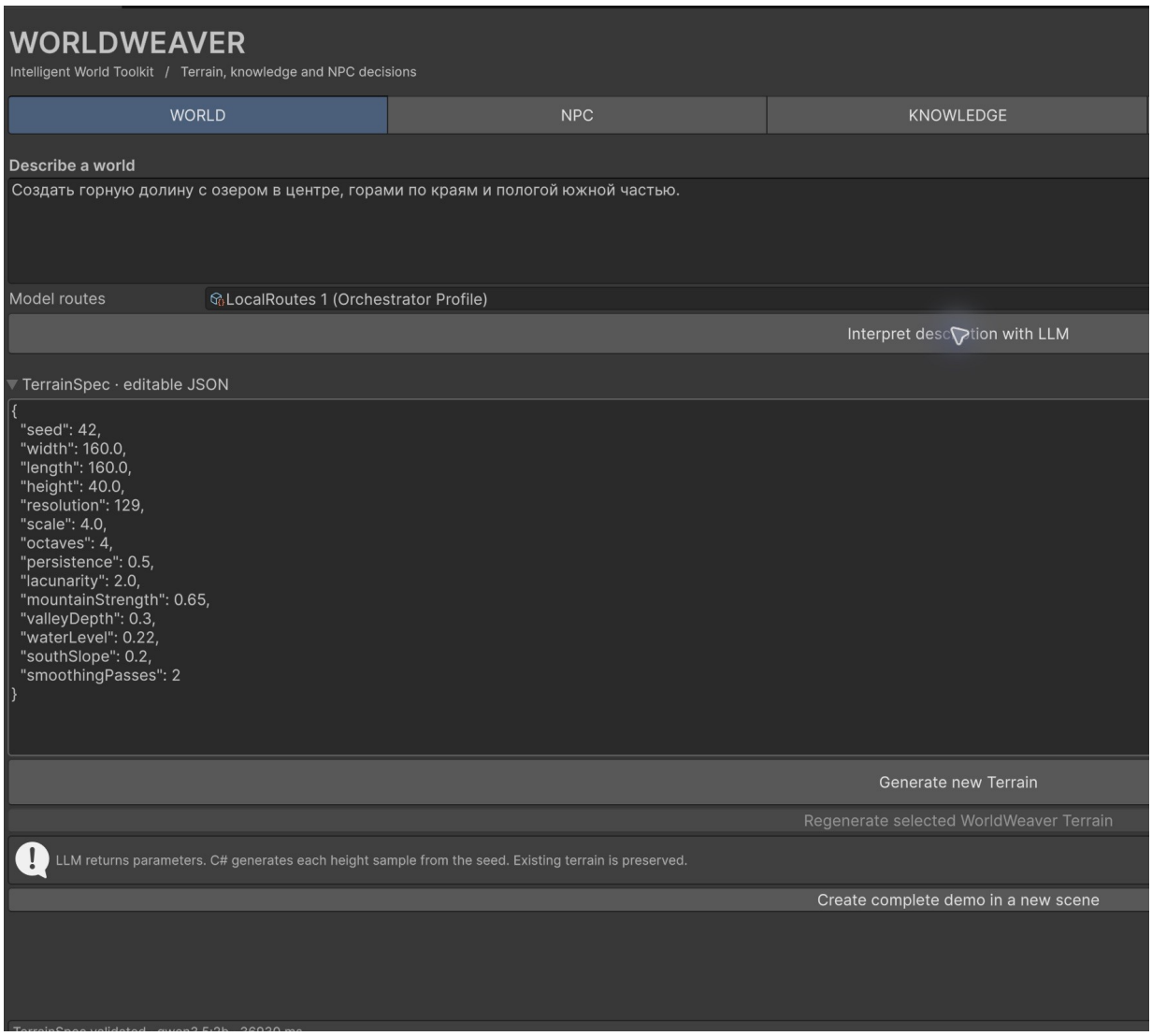


Рисунок 8 — Расширенный инспектор: раздел WORLD, сохранённый в версии 1.1

3.3 Процедурный генератор рельефа

TerrainGenerator не обращается к сети и не зависит от модели. На вход он получает проверенный TerrainSpec, на выход возвращает двумерный массив высот. Вычисление каждой точки включает сумму шумовых октав и геометрические преобразования поверхности. Все параметры ограничены до начала генерации.

В качестве основы применяется value noise с целочисленным смешиванием координат и seed. Интерполяция использует сглаженный коэффициент $t^2(3-2t)$, поэтому соседние клетки переходят друг в друга непрерывно. Для каждой октавы изменяются частота, амплитуда и seed, после чего сумма нормируется на суммарную амплитуду.

Горный край определяется нормированным расстоянием от центра. Центральная долина задаётся экспоненциальным понижением с различным масштабом по двум осям. Южное понижение изменяет вклад края в зависимости от координаты z. Эти операции дают контролируемое семейство ландшафтов, а не произвольную семантическую реконструкцию любого описания.

Сглаживание использует соседей по четырём направлениям. Новый массив создаётся на каждом проходе, поэтому ранняя запись одной ячейки не влияет на вычисление следующей в том же проходе. Для разрешения n и o октав вычислительная сложность основы составляет $O(n^2o)$, дополнительного сглаживания с p проходами — $O(n^2p)$.

TerrainAuthoring формирует TerrainData и назначает его созданному Terrain GameObject [3]. Этот базовый путь оставлен для повторения первоначальной демонстрации: горизонтальная плоскость визуализирует воду, а три одноцветные текстуры помогают различать высотные зоны.

Редакторская команда create_world использует WorldComposer, который подготавливает полное простое окружение в активной сцене. К рельефу добавляются процедурные текстуры трёх слоёв с небольшой цветовой неоднородностью; смешивание учитывает высоту и уклон. Пресеты alpine, forest и desert меняют палитру и начальные параметры. Выбор шейдера соответствует активному render pipeline: одного присутствия URP-пакета недостаточно для назначения URP Lit проекту со стандартным рендерером.

Окружение строится из примитивов и небольших программно сформированных мешей. Дерево имеет ствол и два яруса конической кроны, дом — основание, стены, двускатную крышу, дверь и окна, камень представляет масштабированную сферу. Это собственная стилизованная геометрия, а не результат генерации импортированного или анимированного 3D-ассета. Количество объектов и seed задаются аргументами инструмента. Детерминированное размещение проверяет высоту, уклон, сухой участок и

расстояние до уже размещённых объектов. Ограниченный поиск завершается ошибкой, если нужное количество разместить невозможно.

Новые Point- и Spot-источники при стандартном on_terrain поднимаются по Y как минимум на 3 м над Terrain и сгенерированной водой под той же точкой x/z; более высокая заданная позиция сохраняется. Без Terrain под точкой положение не меняется. Для существующего источника такая коррекция применяется только при явном on_terrain=true; false сохраняет точные координаты, а Directional не перемещается. Это однократное размещение: последующее изменение высот мира не поддерживает постоянную привязку света. Дальность range задаётся отдельно и должна достигать земли с итоговой высоты.

Мир получает собственный источник света и камеру; существующие объекты не удаляются. Новая камера активируется с учётом уже доступной камеры сцены. Команда update_world позволяет, например, уменьшить mountainStrength, не создавая второй мир: обновляются TerrainData, слои, collider и вода, а помеченные объекты привязываются к изменённой поверхности. Такой повтор использует исходную спецификацию с частичными изменениями и не требует заново перечислять все параметры.

В текущей версии нет высотных пещер, симуляции гидрологии, ручного рисования зон или физической эрозии. Отсутствие этих функций не мешает основному исследовательскому сценарию, поскольку проверяется переход от описания к контролируемым параметрам и повторяемому результату.

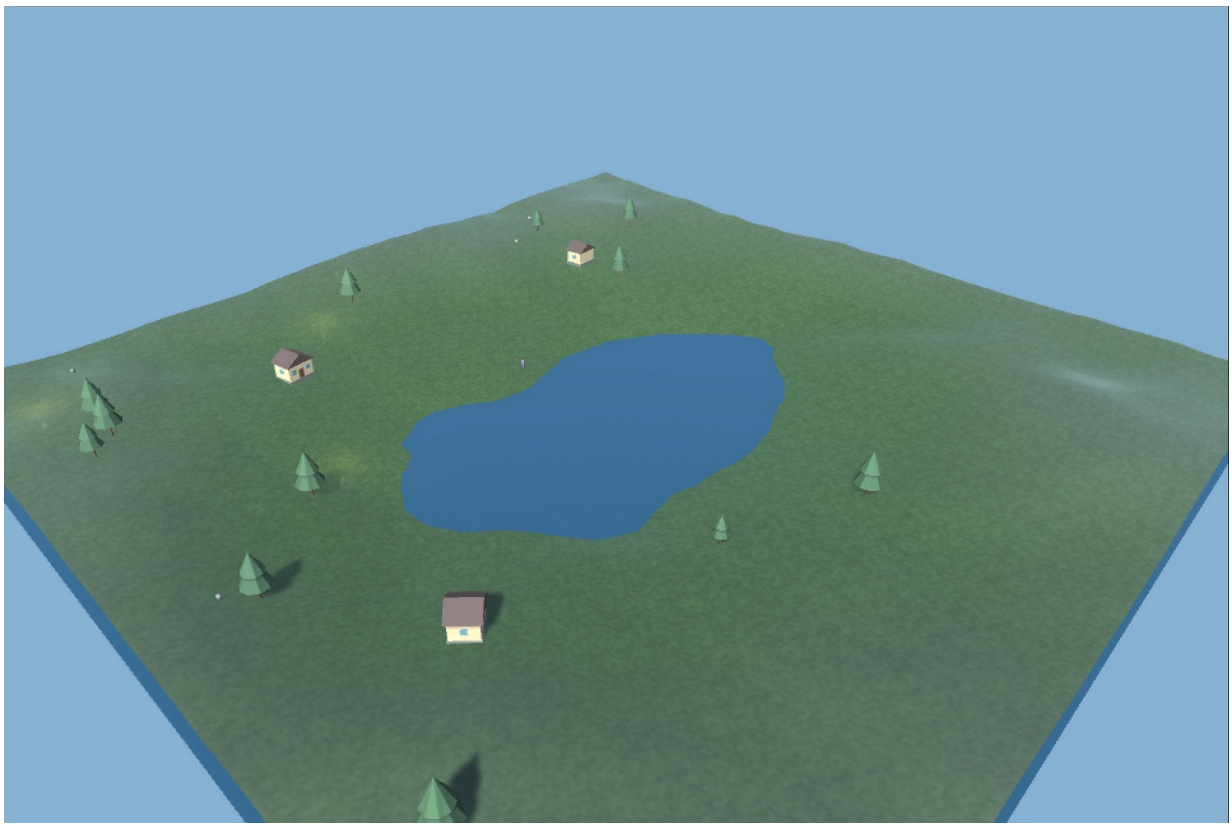


Рисунок 9 — Фрагмент Game View сцены, созданной редакторским агентом в локальной серии 9B

3.4 Локальные и бесплатные облачные запросы

В базовом протоколе версии 1.0 для Terrain и сравнительного QA используется Qwen3.5 2B, а в сквозной runtime-демонстрации Dialogue и Planning — qwen2.5-coder:latest. Обе работают через Ollama на loopback-адресе; embeddings получает EmbeddingGemma. В локальном хранилище Qwen3.5 имеет 1,9 млрд параметров и квантование Q8_0, а qwen2.5-coder — 7,6 млрд и Q4_K_M. Различие моделей между QA и демонстрацией учитывается при выводах: качество диалога 7,6B не выводится из метрик 2B. Модели установлены в стандартном хранилище Ollama за пределами Git. При проблеме TLS штатного загрузчика использован HTTPS-загрузчик официальных manifests и blobs с обязательной проверкой SHA-256 каждого файла до регистрации manifest. Это обход технической ошибки загрузки, а не отключение проверки сертификатов.

Обычный запрос к local chat был проверен отдельно от Unity. Затем тот же provider вызывается из C#-pipeline. В исходном runtime-протоколе задаются schema, ограничение числа выводимых токенов, контекст 4096 и

отключение thinking для выбранной модели. Результат содержит JSON-объект решения, который независимо разбирается приложением.

В версии 1.1 модель редакторского чата выбирается отдельно; начальное назначение — qwen3.5:9b. Меньшие установленные модели доступны в том же каталоге, но их исторические результаты не переносятся на 9B. Это назначение настройки, а не доказательство общей успешности модели. Пресет мощности ограничивает один, три или шесть раундов; temperature устанавливается в ноль, но это не считается гарантией детерминизма языкового ответа. Контекст и лимит вывода отличаются от исходного QA-протокола, поскольку включают описания инструментов и состояние сцены. При создании NPC контроллер формирует или повторно использует сохраняемые профили маршрутов, чтобы игровые компоненты не ссылались на временный уничтожаемый объект настроек чата. В дальнейшем назначения доступны для изменения через Advanced Inspector.

Первоначально предоставленный model ID openai/gpt-oss-120b проверялся через официальный Groq API: документация подтверждает такую модель в списке structured outputs, однако фактический endpoint из текущей сети возвратил HTTP 403 [22]. OpenAI позволил получить каталог моделей, но генерация вернула ошибку исчерпанного баланса. Доступность каталога оказалась недостаточным условием работоспособности inference.

В исследуемой конфигурации платные запросы исключены. Для проверенной cloud-конфигурации выбран Pollinations Classic API, который официально предоставляет анонимный OpenAI-compatible endpoint и GPT-OSS 20B [23]. Положительный ответ реального запроса сохранён отдельно. В runtime используется модельный псевдоним openai-fast, а trace сохраняет фактическое имя модели из ответа.

Актуальный на 31.10.2026 официальный Classic adapter задаёт один бесплатный текстовый ответ за 45 секунд на IP и MAX_TOKENS=1500 [39]. В provider установлен интервал начала запросов 46 секунд и последовательный доступ через semaphore; лимит max_tokens ограничен 1500

независимо от выбранной мощности. Поэтому более глубокий режим увеличивает доступные раунды и объём отобранного контекста сцены, сохраняя предел отдельного облачного ответа. Ожидание сервиса входит в общий бюджет операции. Среди HTTP 402 повторяется только распознанное ограничение error.code=FREE_LIMIT, с учётом Retry-After либо разрешённого бесплатного интервала и числа повторов профиля. Обычные платёжные ошибки и HTTP 400 не повторяются автоматически. Покупка токенов и платёжные действия отсутствуют. Интервал 31 секунда относится к исходному QA-протоколу версии 1.0 и остаётся его историческим параметром. На сверенной странице APIDOCS ещё указаны 30 секунд [23]; для текущей конфигурации использованы значения первичного adapter [39] и фактическое сообщение HTTP FREE_LIMIT, поэтому расхождение документации сохранено явно.

В исходном QA-эксперименте бесплатный API использовал JSON Object Mode с текстовым описанием схемы; этот протокол сохранён без пересчёта. Для текущего Classic отдельно исследован сбой последующей проверки EditorPlan. Повторный запрос с тем же контекстом дал повреждённое начало reply в двух JSON-mode вариантах A/C; native schema в варианте B2 вернула HTTP 400 с сообщением Unimplemented keys: propertyNames. Вариант D с обычным текстовым ответом и явным образцом JSON-экземпляра вернул валидный проверочный результат. Исходные ответы и условия сохранены в pollinations-classic-schema-replay.json. Одна диагностическая проба не устанавливает общую долю успешности и не доказывает принудительное соблюдение схемы сервером.

Обычный текущий маршрут Classic поэтому объединяет system-инструкции, запрашивает JSON-экземпляр в текстовом режиме и сохраняет строгий разбор и прикладную C#-валидацию до исполнения.

В последующей редакции 60192a9 добавлена ограниченная нормализация только для нестрогого маршрута Pollinations Classic. Если ответ содержит целиком закрытые строки и внутренние контейнеры, но ему

не хватает ровно последней скобки корневого объекта, добавляется одна «}». Незакрытые строки, неправильный порядок скобок и иные повреждения этим способом не исправляются. Для явно распознанной просьбы только объяснить обычный текстовый ответ Classic может быть оформлен как reply с пустым actions; JSON-массивы, кодовые ограждения и повреждённые объекты не превращаются в объяснение. После нормализации сохраняются строгий разбор EditorPlan и все прикладные проверки. Локальный Ollama этот адаптер не использует.

В диагностике раздельно сохраняются RawResponse, при наличии преобразования NormalizedResponse и причина RepairKind; секреты маскируются, а длина записи ограничена. Следующий модельный раунд получает принятый проверенный вариант. Нормализация формы не создаёт скрытых действий и не означает, что содержание ответа стало правильным. Ollama использует отдельный совместимый вариант типизированной схемы декодера. В NPC-пайплайне обе конфигурации проходят одинаковую проверку NpcDecision, а доля валидных ответов измеряется отдельно. Различие способов формирования ответа учитывается при интерпретации каждого протокола.

Использование HTTP-провайдера не означает, что пакет включает production-сервер для коммерческой игры. В распространяемом клиенте нельзя надёжно скрыть ключ облачного аккаунта. Для настоящего многопользовательского продукта понадобится доверенный backend с собственной авторизацией, квотами и наблюдением. Текущая бесплатная demo работает без сохранённого API-ключа.

3.5 Реализация retrieval и индексации

Статические сведения демонстрации описывают деревню Лунный Брод, торговца Мирона, лечебный кристалл, лекаря, дом, колодец, праздник, мост, шахту и архив. Эти сведения являются авторскими тестовыми

данными. Их нельзя считать реальными историческими или медицинскими фактами; они специально созданы для вопросов с известными ответами.

Инструмент create_npc принимает knowledge как текст или список текстов. Исполнитель создаёт действительный ассет KnowledgeBase и назначает каждому документу Scope данного персонажа; ссылка записывается в IntelligentNPC. Благодаря этому персонаж из чата получает индексируемые данные без ручного переноса текста в отдельную вкладку. Обновление через update_npc также меняет реальные поля компонента и при необходимости знания, а не только пояснение в истории беседы.

Дополнительно создан документ стража с отдельным Scope. Он содержит служебный пароль и используется для проверки изоляции. Торговец не получает этот документ в свой индекс. Проверка не ограничивается надеждой на то, что модель проигнорирует чужой текст: отбор выполняется до обращения к генератору.

При первом запросе NpcPipeline вычисляет fingerprint состава KnowledgeBase и при необходимости запускает ReindexAsync. Векторы для chunks получает OllamaEmbeddingProvider. После успешной индексации каждый вопрос преобразуется в query-вектор, а InMemoryVectorStore возвращает ранжированные результаты. В debug сохраняются DocumentId и score, что позволяет проверить происхождение контекста.

Разделение типов памяти позволяет не смешивать установленные факты и недавние события. Статическая база отражает авторский лор; разговорная память содержит реплики; эпизодическая память фиксирует исполненные действия. Ограниченная история не отправляется целиком. Из каждой группы выбираются наиболее близкие фрагменты.

В сравнительном QA-эксперименте накопление разговорной памяти выключено. Иначе ранние ответы могли бы становиться дополнительными знаниями для следующих вопросов, а различия между конфигурациями отражали бы не только статический RAG. Для живой демонстрации память включена и обновляется после обработки диалога.



Рисунок 10 — Просмотр базы знаний отдельного NPC

Переиндексация и запрос используют разные жизненные циклы. Перестроение требуется после изменения текста или scope; обычная реплика получает только один query-вектор и выполняет поиск по готовому индексу. В текущей версии fingerprint вычисляется из сериализованного набора документов персонажа. Это обеспечивает обнаружение изменений в Editor,

но на большом корпусе само вычисление fingerprint следует заменить версией ассета или журналом изменений.

При импорте Markdown разметка хранится как текст. Не выполняется загрузка внешних ссылок и не запускается код из документа. JSON-источник принимается в предусмотренной структуре документов, после чего область видимости назначается активному NPC. Эти правила ограничивают неожиданное расширение доступа при переносе базы из чужого проекта.

3.6 Обработка диалога и структурированного ответа

Создание NPC в редакторском чате формирует видимый объект с телом, головой и аксессуарами роли и назначает IntelligentNPC, Inventory и WorldEntity. Имя, роль, personality, biography, goals, constraints, знания и модельные маршруты записываются в компоненты и ассеты. При стандартном размещении on_terrain исполнитель ищет рядом сухой участок допустимого уклона; для явной специальной позиции можно отключить привязку к Terrain. Иерархия не содержит рига и анимационного персонажа: видимая фигура служит представлением работающего runtime-компонента.

На собственной группе создаётся WorldPlayground — панель проверки в Play Mode. Она находит созданных персонажей сцены, позволяет выбрать одного и отправляет реплику в его настоящий TalkAsync. Если сущность Player отсутствует, создаётся посетитель с Inventory, добавляемый в известные сущности NPC. Панель показывает ответ и фактический инвентарь посетителя. Это даёт возможность проверить нового персонажа без ручной привязки старой демонстрационной панели; наличие диалога само по себе не означает готовность навигации по всему Terrain.

Метод IntelligentNPC.TalkAsync проверяет отсутствие уже активного запроса, переводит NPC в Thinking и создаёт связанный cancellation token. NpcPipeline получает данные мира, выполняет retrieval и формирует CompletionRequest. System-сообщение задаёт правила использования роли,

знаний и разрешённых действий, а пользовательская часть содержит JSON с контекстом и исходной репликой.

Контекст включает инвентарь, состояние NPC, расстояния до известных сущностей, цель, personality и hard_constraints. Число сущностей ограничено 32, дополнительных адаптеров мира — восемью. IWorldContextSource позволяет игровому проекту добавить собственное компактное состояние без изменения provider или router.

После получения CompletionResult вызывается NpcDecision.Parse. Строго проверяются обязательные имена и типы, а повторённое поле считается ошибкой. Markdown-обёртка не извлекается эвристически: если модель вернула текст вне контракта, pipeline фиксирует нарушение. Такое решение облегчает диагностику и не выдаёт частично угаданный объект за правильный ответ.

Корректно разобранное решение передаётся в ActionDispatcher. Принятый результат позволяет показать speech. При отклонении действия интерфейс сообщает причину; обещание модели не отображается как свершившееся изменение инвентаря. Trace содержит raw output и фактический результат проверки, поэтому несоответствие остаётся наблюдаемым.

Отдельный компонент DemoDialogue предоставляет поле ввода, отправку, кнопку автономного решения и переключение local/cloud. Рядом показаны реальные остатки Crystal у NPC и Player. Эти значения читаются из Inventory после исполнения и не являются извлечёнными из речи числами.

Жизненный цикл памяти отделён от уже исполненного действия. Если после commit передача завершена, а обновление embedding-памяти отменено, trace сохраняет исходный результат и предупреждение об индексации. Пользователь получает подтверждённое состояние сцены. Возврат общей ошибки Cancelled после передачи мог бы создать ложное впечатление, что операция не состоялась, и спровоцировать повтор. Для этого случая добавлена отдельная регрессионная проверка с фактическими остатками 9/1.

3.7 Передача предмета и автономное движение

GiveItem является встроенным примером действия с последствиями. Для него проверяются разрешение, зарегистрированная цель, дальность, положительное количество, остаток и вместимость получателя. Общий бюджет решения не позволяет обойти лимит несколькими небольшими командами. В демонстрации у торговца изначально десять кристаллов, а лимит передачи за решение равен одному.

TakeItem представлен тем же TransferAction с обратным направлением, но требует дополнительного AllowTakingItems. Наличие такого обработчика в реестре само по себе не даёт NPC права использовать действие. В стандартной demo оно не входит в список разрешённых операций торговца.

MoveTo, Follow и Flee используют NavMeshAgent. Валидатор сначала проверяет положение агента на NavMesh, затем ищет допустимую конечную точку и полный путь. Для Flee конечная точка вычисляется детерминированно в направлении от угрозы; модель выбирает сам тип действия и цель угрозы, а не промежуточные координаты движения.

Follow обновляет destination с интервалом 0,5 секунды. Для этого не выполняется новый LLM-запрос. Планирование NPC ограничено PlanningCooldown, по умолчанию 45 секунд, и не запускается параллельно активному диалогу. Автономный режим можно включить в Inspector; для демонстрации предусмотрен также явный запрос следующего решения.

Для простых редакторских задач attach_behavior добавляет один из заранее реализованных режимов SceneBehavior: rotate, bob, patrol или followplayer. Эти режимы используют ограниченный набор параметров и обычное runtime-обновление Transform. Они не загружают произвольный тип через reflection и не вызывают LLM каждый кадр. В частности, patrol является движением по заданным точкам, а не автоматическим построением NavMesh-пути. Такой компонент следует отличать от решений IntelligentNPC и проверенного NavMesh-перемещения ниже.

DemoNavigation строит небольшой NavMesh только по назначенной walkable-геометрии рыночной площади [4]. Это позволяет проверить настоящее движение и отделить его от художественного качества рельефа. Проект не объявляет, что автоматически построил навигацию по всем сгенерированным горам и водной поверхности.

Состояния Idle, Thinking, Moving, Following, Waiting и Failed отражают основные этапы работы. Ошибка API переводит NPC в Failed, но не разрушает GameObject и не блокирует новую попытку. Отмена завершает активную операцию и возвращает управление. После достижения MoveTo-цели агент переходит в Idle.

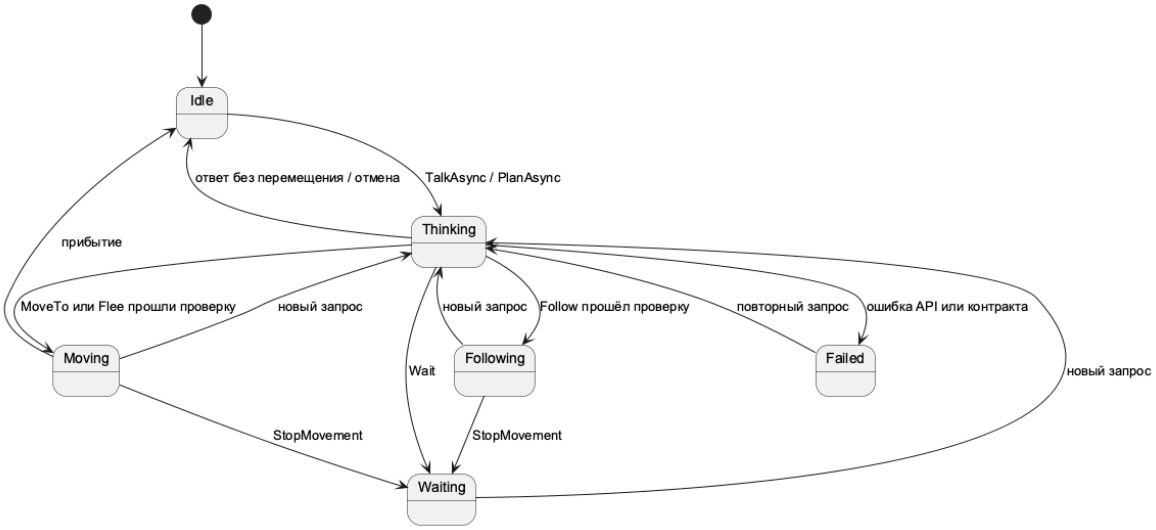


Рисунок 11 — Состояния интеллектуального персонажа

Для автономного Planning предусмотрено отдельное PlanningActions. Итоговый список является пересечением с AllowedActions, поэтому расширение общих возможностей NPC не открывает автоматическую передачу предмета во время периодического планирования. Этот список также включается в enum схемы NpcDecision. Повторная проверка dispatcher обеспечивает правило даже при игнорировании схемы внешним API.

Разговорная память исключается из контекста Planning, поскольку прежняя просьба игрока могла ошибочно восприниматься как новое действие. Эпизодическая запись сохраняется и явно обозначает уже исполненное. Контекст сообщает позицию, расстояния и текущую цель. Такой подход не превращает модель в детерминированный планировщик, но

устраняет обнаруженный путь повторного исполнения диалогового намерения.

3.8 Диагностика и ограничения безопасности

Редакторский чат хранит отдельные попытки получения EditorPlan, ошибки предконтроля и результаты каждого инструмента. Для следующего раунда передаются прочитанное состояние и ограниченный результат; запрос не полагается на одно свободное сообщение «готово». Контекст не более 80 объектов является выборкой, поэтому он не представляет полный сериализованный проект. inspect_object и search_project позволяют уточнять сведения перед изменением; чтение скриптов ограничено Assets, расширением .cs, размером 64 КиБ и проверкой пути.

История сохраняется в UserSettings/WorldWeaverChat отдельными JSON-файлами с ограничением глубины и размера коллекций и атомарной заменой файла. Повреждённый файл не перезаписывается пустой историей при неудачной загрузке. Перед записью и отображением результатов маскируются известные шаблоны ключей. Этот фильтр не является универсальным детектором секретов, поэтому содержимое документов и прикрепляемого контекста также требует осмысленного выбора.

Для нового C# ScriptDrafts сохраняет исходник, описание и версии файла до и после применения. Импорт выполняется только после просмотра, затем Unity сообщает фактические ошибки компиляции. Статус «исходник записан» отделён от «скомпилирован»; отсутствие подтверждённого события после перезагрузки домена не объявляется успешной компиляцией. Подключение компонента является отдельной просмотренной операцией. При восстановлении файл с последующей пользовательской правкой не затирается автоматически. Ни фильтр текста, ни успешная компиляция не создают sandbox и не подтверждают правильность поведения в Play Mode.

PipelineTrace фиксирует время, персонажа, input, route, retrieved chunks, ограниченный контекст, raw JSON, provider/model, токены, результат

валидации и список действий. Журнал в памяти ограничен 30 записями. Экспорт JSON предназначен для технической проверки и исследования, а не для хранения ключей.

Код не журналирует заголовок Authorization и не сохраняет сырые тела provider-ошибок, поскольку такие сообщения иногда включают часть credential. В профиле запрещены credentials внутри URL. Для локального API разрешён HTTP только на loopback, а удалённый endpoint должен использовать HTTPS.

Эти меры не делают вывод модели доверенным. В контекст могут попасть атакующие инструкции из реплики или документа. Prompt отделяет такие данные от правил, но окончательный барьер для действий — список разрешений и проверка текущего состояния. При этом система не гарантирует отсутствие любых нежелательных реплик: контентная политика диалога и формальная допустимость действия являются разными задачами.

Набор жёстких проверок конечен. Текстовые Constraints не преобразуются автоматически в доказуемые правила. Если игровому проекту требуется, например, запрещать торговлю ночью или учитывать принадлежность предмета, разработчик должен добавить соответствующее детерминированное условие или собственный обработчик. Это ограничение отражает область применимости инструментария.

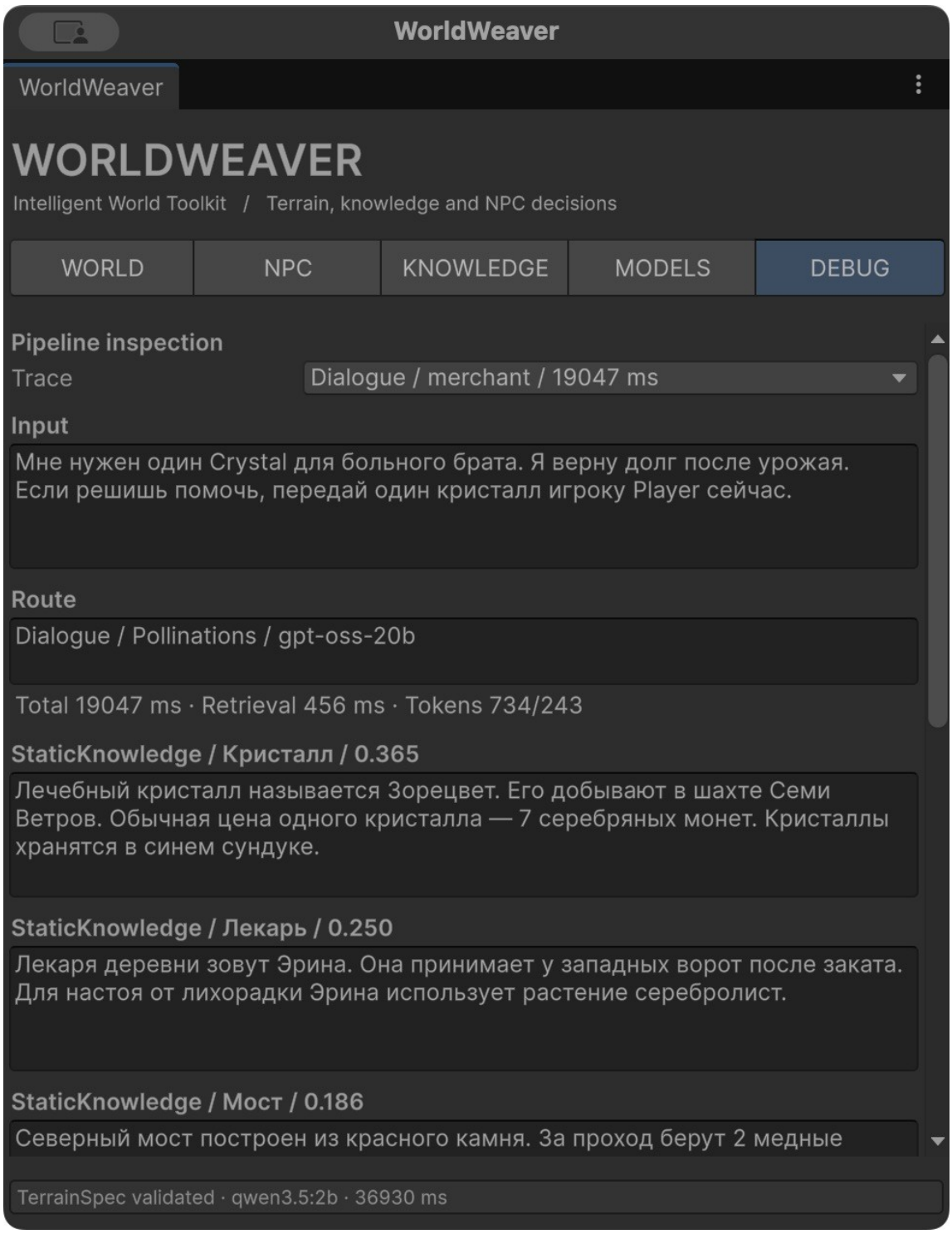


Рисунок 12 — Диагностика маршрута и извлечённых знаний

3.9 Демонстрационный сценарий

Основной сценарий версии 1.1 начинается с активной сцены и открытия чата. После выбора доступной модели пользователь просит создать, например, лесную долину с небольшим поселением и заданным seed. Контроллер получает EditorPlan, исполнитель создаёт собственную группу, Terrain и окружение, а чат показывает фактический результат. Следующая

реплика может уменьшить горный край или изменить свет; цель должна разрешаться в существующий объект, поэтому корректировка не требует создавать вторую копию мира.

Затем пользователь описывает NPC, его личность и знания. Созданные компоненты и ассеты можно проверить в Inspector и через inspect_object. В Play Mode WorldPlayground позволяет обратиться к этому персонажу и наблюдать настоящий результат pipeline. Для простого вращения или патруля используется встроенное поведение; запрос нового C# сначала создаёт просмотренный исходник, затем проходит компиляцию, и только после этого допускается подключение к объекту. Эти шаги являются сценарием работы продукта; их успешность должна подтверждаться отдельным протоколом, а не выводиться из наличия кнопок.

Для воспроизводимого исходного эксперимента сохранён отдельный DemoBuilder. Он сохраняет новую сцену по уникальному пути и добавляет её к существующему списку сборки. При интерактивном запуске предварительно предлагается сохранить изменённую сцену. Демонстрация содержит Terrain, водную зону, рыночную площадь, Player, торговца, стража, дом и колодец. Геометрия намеренно проста: она нужна для различения целей и движения, а не для имитации готовой коммерческой игры. На торговце размещаются IntelligentNPC, Inventory, WorldEntity и NavMeshAgent.

Последовательность базового runtime-сценария начинается с подключения пакета и открытия Advanced Inspector. Разработчик вводит описание рельефа, получает TerrainSpec и генерирует поверхность. Затем он выбирает торговца, показывает личность, базу знаний и маршруты. В Play Mode игрок спрашивает о факте лора, после чего предлагает помощь больному брату и просит один кристалл.

При согласии модели JSON содержит GiveItem, validator проверяет условия, а Inventory показывает изменение остатка. Следующий Planning-запрос предлагает NPC выбрать действие для своей цели. Если выбрано

MoveTo к House, NavMeshAgent физически перемещает объект. В Debug можно сопоставить route, ответ, проверки и выполненную команду.

Смена local/cloud повторяет разговор через другой provider без изменения игрового кода. Для текущего бесплатного cloud заранее учитываются серверный интервал 45 секунд, клиентский запас до 46 секунд и предел max_tokens=1500. Если сервис недоступен, демонстрация показывает контролируемую ошибку. Локальный режим выбирается явно; скрытая подмена провайдера или ответа не используется. Успешный end-to-end результат подтверждается отдельным JSON-протоколом, а не наличием mock-теста.

Реализованный продукт объединяет основные процессы в одном пакете и сохраняет расширяемость на уровне интерфейсов. Его оценка должна различать программные инварианты, фактическое выполнение модельного сценария и субъективную полезность. В следующей главе результаты рассматриваются по отдельным протоколам. Сохранённые метрики базового исследования — автоматические проверки, вопросы с известными ответами и измеренные задержки — не переносятся автоматически на новый редакторский процесс.

4 ТЕСТИРОВАНИЕ И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА

4.1 Цели и организация проверки

Испытания разделены на проверки программных инвариантов, интеграцию с реальными моделями и эксперимент над ответами NPC. Эти группы подтверждают разные свойства. Mock-провайдер позволяет воспроизвести точный ошибочный ответ и проверить отсутствие побочного эффекта. Реальный provider подтверждает сетевую интеграцию, но его единичный успешный ответ не доказывает устойчивость на всех репликах.

Программные испытания выполнены средствами Unity Test Framework [24]. EditMode проверяет чистые алгоритмы, конфигурацию, разбор JSON, retrieval, память и HTTP. PlayMode проверяет изменение Inventory и движение NavMeshAgent. Отдельный сценарий RealScenario вызывает настоящую модель, получает структурированное решение и сохраняет реальные изменения сцены.

Для QA-эксперимента подготовлено 30 фиксированных вопросов с известными ответами. Три конфигурации образуют 90 запросов: получено 89 ответов модели и одна ошибка HTTP 400; ошибочный запрос сохраняется в знаменателе метрик качества. Лор является авторским и содержит конкретные имена, цены, места и правила вымышленной деревни. Такой набор позволяет отделить сведения, доступные из базы, от общих знаний модели. Для каждого вопроса сохранён идентификатор правильного документа и допустимые словесные варианты ответа.

Исследуются три конфигурации: локальная Qwen3.5 2B без RAG, та же модель со статическим RAG и облачная GPT-OSS 20B через бесплатный API с тем же RAG. Embedding-модель и алгоритм retrieval сохраняются одинаковыми. Смешанная оркестрация дополнительно проверяется в демонстрационном сценарии через отдельные Terrain, Dialogue и Planning routes.

Файл сценариев и построчные JSON-протоколы находятся в results. Каждая строка содержит исходный вопрос, конфигурацию, ожидаемые варианты, trace или контролируемую ошибку, время и результат автоматической оценки. Агрегаты строятся из этих файлов; итоговые числа не задаются вручную в графиках.

4.2 Модульные и сетевые испытания

Для исходного ядра инструментария до добавления редакторского чат-ассистента EditMode-набор Unity 6 включает 56 проверок. Последующее расширение набора выделено в подраздел 4.9. Для Terrain проверяются допустимость разрешения и числовых значений, обязательные поля, отклонение дробного integer и неизвестного поля, одинаковый seed и нормированный диапазон высот. Сравнение двух массивов в одной среде выполняется непосредственно по значениям, без визуального предположения о сходстве.

Отдельная регрессионная проверка отменяет индексацию памяти после уже принятого GiveItem: речь и результат исполнения должны сохраниться, а оба остатка составить 9 и 1. Это проверяет границу commit, где отмена вспомогательной операции не должна маскировать состоявшееся изменение мира.

Для Inventory и действий проверяются изменение обоих участников передачи, неправильное количество, неизвестные item и target, запрет операции, дальность, вместимость, повторение идентификаторов сущностей и общий бюджет. Отдельный сценарий проверяет двойное расходование: два действия запрашивают больше доступного количества в сумме. Предварительная проверка должна отклонить весь пакет без частичной передачи.

Разбор NpcDecision проверяется на недостающих и лишних полях, повторённом speech, строковом amount и Markdown-обёртке. Эти случаи

важны для работы с model output: синтаксически похожий ответ не должен попадать в игровой state через неявное преобразование.

RAG-проверки включают перекрытие chunk-окон, устойчивость идентификатора, изменение scope, ранжирование близких векторов, исключение документа другого персонажа и несовместимость размерностей. Ограничение памяти проверяется после добавления большего числа записей, чем разрешено. Отмена переиндексации не должна уничтожать ранее построенный индекс.

Сетевые проверки используют локальный TCP-сервер с заранее заданными ответами. Сервер возвращает 429 с последующим 200, ошибку 400, неправильный JSON либо задерживает ответ. Код проверяет повтор временной ошибки, отсутствие повторов для клиентской ошибки, cancellation и timeout. Использование такого сервера позволяет испытать сбои без намеренного расходования квоты облачного API.

Таблица 6 — Модульные и сетевые испытания

Набор	**Результат**	**Ошибки**	**Длительность или условие**
Unity 6 EditMode, исходное ядро	56/56	0	2,32 с
Unity 6 PlayMode без внешних моделей	2/2	0	1,08 с
Unity 6 PlayMode с реальными моделями	2/2	0	85,16 с
Unity 2022.3 EditMode, исходная редакция	54/54	0	2,34 с
Чистая установка local-tarball и Git	Успешно	0 ошибок компиляции	Два новых проекта и новые Library

Результаты тестов в таблице получены из XML-протоколов Unity, результат установки — из сохранённого JSON-отчёта. Наличие положительного mock pipeline не считается доказательством внешнего inference. В нём проверяется только соединение retrieval, парсера и dispatcher при управляемом ответе.

4.3 Интеграция в Play Mode

Передача Crystal проверяется по обоим инвентарям. Сценарий начинается с десяти предметов у NPC и нулевого количества у Player. После принятого GiveItem(1) ожидаются девять и один соответственно. При отклонённом решении оба остатка должны сохраняться.

Проверка движения строит NavMesh по назначенному walkable-мешу, размещает агент на сетке и отправляет MoveTo. После заданного интервала измеряется фактическое изменение позиции. Проверка Follow дополнительно подтверждает переход в соответствующее состояние и остановку через StopMovement. Это проверка детерминированного исполнительного слоя; необходимость LLM для неё отсутствует.

Реальный сценарий расширяет эту проверку запросом к модели. Сначала NPC получает просьбу помочь больному брату и передать один Crystal. Затем отдельный Planning-запрос предлагает выбрать действие для текущей цели. Протокол содержит исходный и конечный остатки, raw JSON, результат валидации и пройденное расстояние.

Таблица 7 — Интеграция в Play Mode

Маршрут	Ходов диалога	Crystal NPC / Player	Изменение позиции	Итог
Полностью локальный	2	9 / 1	11,6803 м	Пройден
Смешанный с бесплатным облачным Dialogue	1	9 / 1	11,6437 м	Пройден

Для каждого маршрута выполнен один финальный ограниченный переговорный сценарий. Общий протокол содержит до трёх заранее определённых реплик: просьбу помочь, согласие на цену в семь монет с обязательством возврата долга, затем подтверждение условий. После появления одного Crystal у игрока цикл прекращается, затем вызывается Planning. В облачном варианте передача произошла на первом ходе, в локальном — на втором. В обоих случаях Planning выполнила локальная qwen2.5-coder:latest и предложила MoveTo(House). Следовательно, это

проверка завершения двух конкретных сценариев, а не оценка общей вероятности успешных переговоров.

Локальный финальный ответ выявил ограничение: при принятом GiveItem речь сохранила осторожную формулировку о необходимости сначала убедиться в возврате. Изменение обоих инвентарей действительно произошло, но смысловое согласование речи и поступка оказалось неполным. Интерфейс поэтому отображает остатки из Inventory. Структурная валидность и корректность исполнения не гарантируют художественную последовательность текста.

Дополнительный повтор через графический Editor подтвердил ограниченную устойчивость поведения. В облачном GUI-сценарии передача и перемещение вновь состоялись. В локальном GUI-сценарии после двух реплик Inventory также изменился до 9/1, но последующее Planning вернуло пустой actions и речь с отказом; изменение позиции составило 0 м. JSON прошёл проверку, поскольку бездействие допустимо контрактом, однако полный демонстрационный критерий не был выполнен. Эти данные сохранены в gui-scenario-local.json и gui-scenario-cloud.json. Поэтому положительный результат двух интеграционных тестов не интерпретируется как гарантированное планирование при любом повторении.

Ранние прогоны сохранены в каталоге iterations. В них встречались допустимый отказ от подарка, выбор Wait и повторение передачи в Planning. Инженерное исправление ограничило Planning собственным подмножеством действий и enum в JSON Schema; разговорные реплики исключены из planning-контекста, а эпизоды описывают уже исполненное. Это уменьшает возможность повторить просьбу из прошлой реплики. Проверка разрешений в C# остаётся обязательной и после усиления схемы.

Проверка поставки выполнена двумя способами: из local-tarball и из Git-зависимости с закреплённым commit. В обоих случаях создан новый проект с новой Library; число ошибок компиляции равно нулю. Дополнительный consumer-сценарий дважды создал demo, сохранил

исходную сцену и порядок существовавших build settings, подтвердил двух NPC, ссылки KnowledgeBase и маршрутов. Изменение параметров Terrain с последующим Regenerate изменило также alphamap трёх слоёв. Эти результаты сохранены отдельно от модельного inference.

Финальная интеграция выполнена средствами Unity Test Framework: два теста, оба Passed. Чистая установка отдельно подтвердила импорт архива пакета в новый проект и создание демонстрационной сцены без ошибок компиляции. Она не заменяет модельные испытания и не измеряет доступность стороннего API.

Наблюдаемое движение подтверждает исполнение конкретного действия на демонстрационной площади. Оно не доказывает автоматическую проходимость любого сгенерированного Terrain. Если модель выбирает допустимый Wait вместо MoveTo, это валидное решение pipeline, но оно не удовлетворяет отдельному демонстрационному критерию физического перемещения; эти два результата не смешиваются.



Рисунок 13 — Диалог в Play Mode и состояние инвентарей

4.4 Метрики качества ответов и retrieval

Основная метрика QA — доля вопросов, в ответе на которые присутствует допустимая нормализованная формулировка известного факта. Для текста приводятся регистр и буква ё, для числовых вариантов проверяются границы цифровой последовательности. Это автоматический proxy корректности факта, а не полная экспертная оценка смысловой достоверности.

Такой критерий может пропустить противоречие: модель способна назвать ожидаемое имя и одновременно добавить неверное объяснение.

Поэтому результаты нельзя называть абсолютной groundedness. Сырые реплики сохраняются для последующей ручной проверки, а в описании метрики используется формулировка «совпадение с известным ответом».

Retrieval hit@3 показывает долю вопросов, для которых правильный документ присутствует среди трёх статических фрагментов. Дополнительно рассчитывается reciprocal rank первого правильного источника. Precision@3 интерпретируется с учётом разметки: для каждого вопроса задан один целевой документ, поэтому наличие одного нужного фрагмента среди трёх соответствует 1/3, а не полному заполнению выдачи релевантными объектами.

Доля JSON-valid отражает успешность строгого прикладного парсера. Результат ActionDispatcher характеризует принятие предложенного пакета действий. Для вопросов о фактах большинство корректных ответов не требует предметных изменений, поэтому высокий процент принятых пустых actions или Talk не доказывает качество сложного планирования. Надёжность предметных операций оценивается отдельными тестами.

Для каждой конфигурации измеряются полное время запроса, время retrieval и время провайдера, а также доступные счётчики input/output tokens. Полное время включает ожидание квоты, подготовку и первичную индексацию, если она потребовалась. Время провайдера отражает более узкий участок сетевого обращения и не равно чистому времени вычислений модели.

Таблица 8 — Метрики качества ответов и retrieval

Показатель	**Qwen3.5 2B без RAG**	**Qwen3.5 2B с RAG**	**GPT-OSS 20B с RAG**
Факт в исходной речи	2/30	20/30	29/30
Факт в принятой реплике	1/30	16/30	16/30
JSON прошёл парсер	30/30	30/30	29/30
Пакет действий принят	27/30	22/30	16/30
Hit@3	0/30	30/30	30/30

Показатель	Qwen3.5 2B без RAG	Qwen3.5 2B с RAG	GPT-OSS 20B с RAG
MRR@3	0,000	0,983	0,983
Ошибка API	0	0	1

Без RAG локальная модель назвала ожидаемый факт в 2 из 30 исходных реплик. При добавлении одинаковой базы знаний результат составил 20 из 30, то есть прирост на 60 процентных пунктов в этом наборе. Все 30 целевых документов вошли в top 3, а MRR@3 составил 0,983. При этом модель не использовала найденный факт правильно в десяти вопросах. Таким образом, безошибочный hit@3 не означает безошибочную генерацию.

Облачная конфигурация содержала ожидаемый факт в 29 из 30 исходных реплик. На вопросе qa-26 API возвратил HTTP 400; строка сохранена как ошибка и остаётся в знаменателе. Значение 29/30 не следует интерпретировать как отсутствие иных неточностей в этих репликах: метрика проверяет наличие размеченного факта, а не все дополнительные утверждения.

Прикладной результат ниже исходного текста модели. До игрока дошёл ожидаемый факт в 16 из 30 случаев как для локального RAG, так и для облачного. В облачной группе 13 содержательно подходящих ответов были отклонены из-за Talk с target Player, тогда как действовавший контракт требовал пустой target. В локальных группах встречались также перемещения в Editor вне NavMesh. Эти случаи показывают чувствительность конечного результата к договорённости о форме действия и к условиям сцены.

Эксперимент выполнен в Editor, а не в полноценном игровом окружении каждого вопроса. Поэтому доля принятых пакетов не является самостоятельной оценкой навигации. Исполнительный слой обоснованно отклонял MoveTo без активной NavMesh. Отдельные PlayMode-тесты проверяют перемещение в предназначенной для него среде.

После эксперимента уточнены подсказки и схема допустимых действий для Planning. Сохранённые 90 запросов не пересчитывались как результаты исправленной модели. Для доказательства улучшения конечной QA-точности нужен новый прогон с тем же протоколом и новым обозначением версии. Результат работы включает как достигнутые показатели, так и воспроизводимо выявленную проблему согласования семантики model output с контрактом действий.

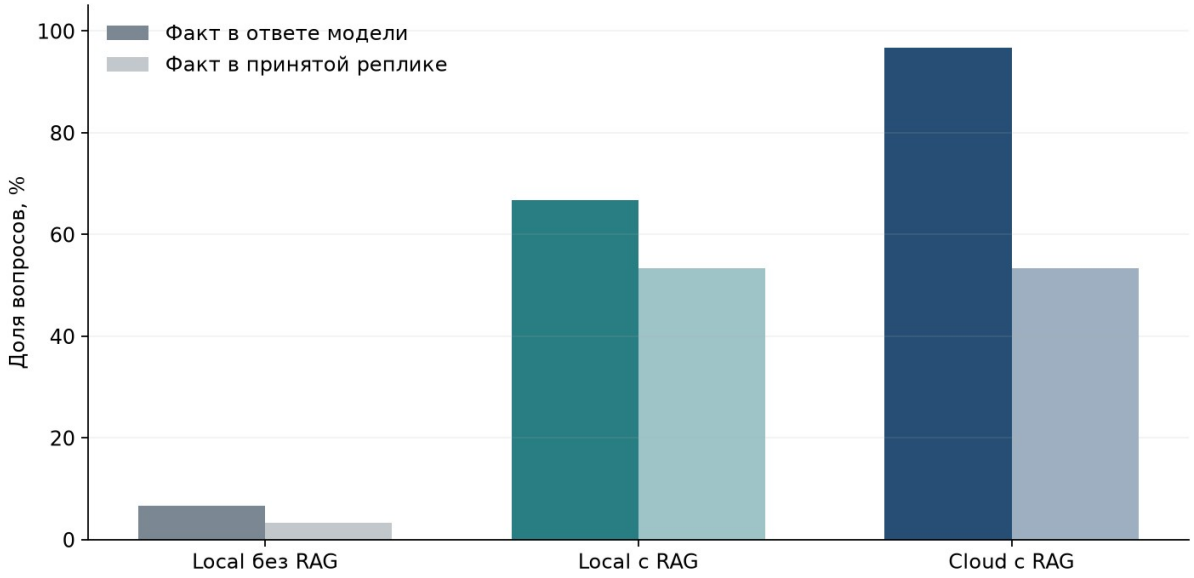


Рисунок 14 — Доля совпадений факта в исходной и принятой реплике

4.5 Локальное и облачное исполнение

Одинаковый прикладной контракт позволяет менять provider, но не делает условия исполнения равными. Локальная модель использует ресурсы компьютера и может конкурировать с Unity за память. Облачная модель имеет другие параметры и серверное оборудование, а бесплатный маршрут дополнительно ограничивает частоту. Поэтому разность времени не следует приписывать только размеру LLM.

Таблица 9 — Локальное и облачное исполнение

Конфигурация	p50 полного запроса, с	p95 полного запроса, с	p50 обращения provider, с
Qwen3.5 2B без RAG	6,33	7,33	6,32
Qwen3.5 2B с RAG	6,95	7,92	6,60
GPT-OSS 20B с RAG	30,95	74,54	10,20

Медиана времени provider рассчитывается по строкам с доступным ненулевым ElapsedMs. Отсутствие этой метрики при ошибке API не интерпретируется как нулевая задержка.

В опыте 1.0 Pollinations вызывался последовательно с интервалом 31 секунда. При оценке пользовательского ожидания этот интервал включается в полную задержку. Исключать его из общего времени означало бы скрыть существенное ограничение бесплатного варианта. Одновременно отдельно приводится время конкретного HTTP-обращения, чтобы различать ожидание слота и получение completion.

Сравнение не требует покупки токенов. Использованные локальные запросы выполняются на оборудовании пользователя, а анонимный cloud-маршрут предоставлялся бесплатно на дату проверки. Нулевая стоимость API-запроса не означает отсутствия затрат электроэнергии, оборудования, разработки или сопровождения. Экономия трудоёмкости относительно ручной настройки не измерялась контролируемым исследованием, поэтому процент сокращения времени не заявляется.

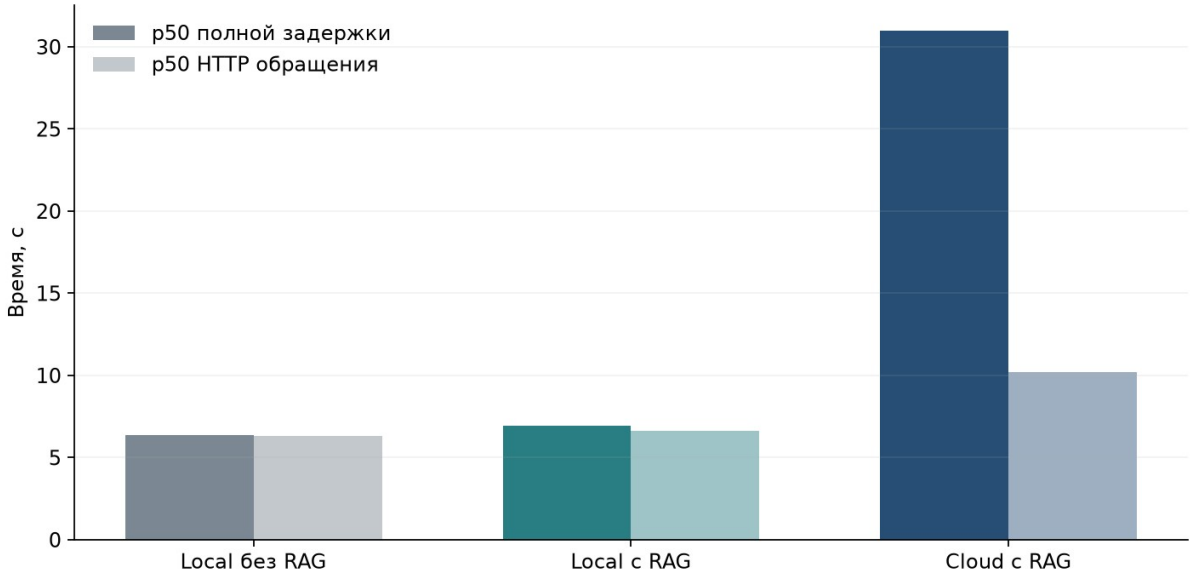


Рисунок 15 — Измеренные задержки модельных конфигураций

4.6 Производительность и воспроизводимость Terrain

Для Terrain выполнена серия из десяти seed на каждом из четырёх разрешений: 65, 129, 257 и 513. Измеряется непосредственно вызов TerrainGenerator.Generate в C#. Сохранение ассетов, работа интерфейса, получение TerrainSpec от LLM и импорт материалов в это время не входят.

Каждая спецификация генерируется дважды, и массивы сравниваются поэлементно. Это позволяет проверить воспроизводимость в конкретной версии Unity и на конкретной машине. Совпадение между разными платформами, компиляторами и режимами вычислений отдельно не исследовалось и не объявляется доказанным.

Таблица 10 — Производительность и воспроизводимость Terrain

Разрешение	Число seed	Медиана Generate, мс	p95 Generate, мс
65	10	2,237	3,846
129	10	8,718	8,756
257	10	34,223	36,445
513	10	136,130	137,208

Рост времени с разрешением согласуется с увеличением числа ячеек. Увеличение стороны сетки примерно вдвое увеличивает их число примерно

вчетверо, поэтому сравнивать результаты только по размеру n без учёта n^2 некорректно. Для интерфейса выбран умеренный размер по умолчанию, который сохраняет читаемость формы и небольшой объём данных.

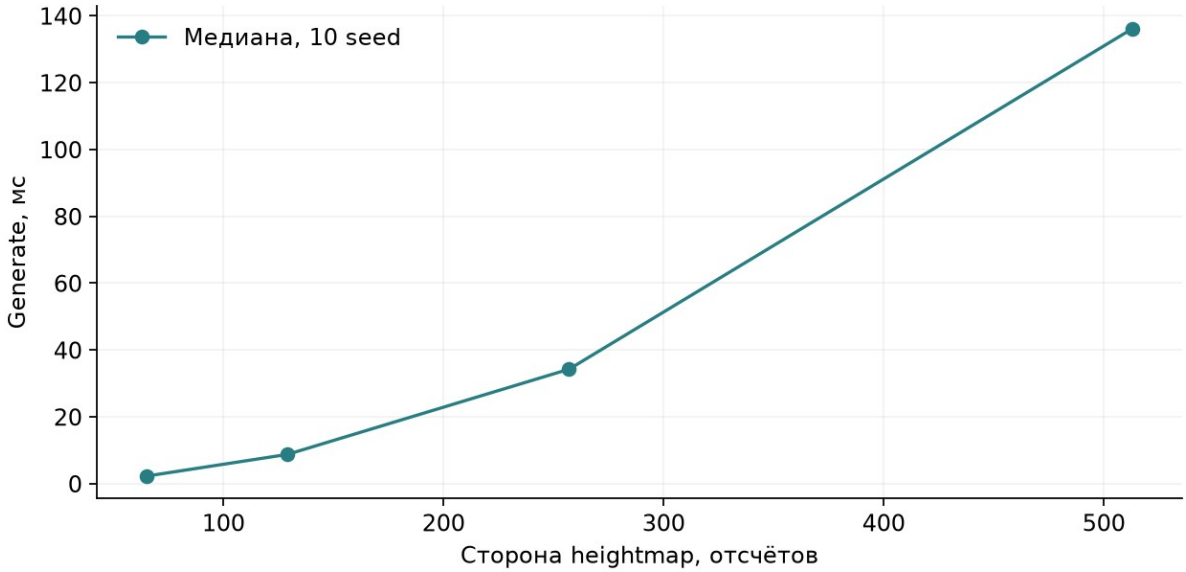


Рисунок 16 — Продолжительность алгоритмической генерации Terrain

4.7 Формализация метрик и проверка воспроизводимости

Для каждого вопроса i задаётся индикатор f_i, равный единице при наличии хотя бы одного допустимого варианта известного ответа в сгенерированной речи. Доля совпадений составляет $F = (\Sigma\ f_i)/n$. Второй индикатор g_i дополнительно требует, чтобы решение прошло прикладной валидатор и соответствующая речь была выдана игроку. Следовательно, $G = (\Sigma\ g_i)/n$ характеризует конечный пользовательский результат. Условие $g_i \leq f_i$ для каждого случая помогает обнаружить ошибочное смешение этих показателей.

Для retrieval индикатор h_i равен единице, если ожидаемый DocumentId найден в первых трёх статических результатах. $Hit@3 = (\Sigma\ h_i)/n$. Если документ находится на позиции r_i, его reciprocal rank равен $1/r_i$; при отсутствии правильного документа принимается ноль. MRR@3 получается усреднением этих величин. В наборе с одним размеченным документом на вопрос эти метрики интерпретируются проще, чем precision, и не требуют считать все остальные фрагменты заведомо нерелевантными.

Для задержки используется медиана и 95-й перцентиль. Отсортированные времена обозначаются $t_0,\ldots,t_{n-1}$. Для доли p вычисляется индекс $j = (n-1)p$ и линейная интерполяция между соседними значениями. Эта конвенция зафиксирована в скрипте анализа, поскольку различные библиотеки могут выдавать разные перцентили на малой выборке. Время ошибки также сохраняется: исключение не удаляет неудобный случай из знаменателя качества.

Доверительные интервалы для доли совпадений рассчитаны методом Уилсона при $z = 1{,}96$. Они показывают статистическую неопределённость при условной модели независимых испытаний и не устраняют зависимость вопросов от общего небольшого корпуса. Поэтому интервалы приводятся как ориентир точности оценки, а не как доказательство превосходства на любых игровых данных.

Таблица 11 — Формализация метрик и проверка воспроизводимости

Конфигурация	**Доля совпадений в исходной речи**	**Интервал Уилсона 95 процентов**
Qwen3.5 2B без RAG	2/30, или 6,7 процента	1,8–21,3 процента
Qwen3.5 2B с RAG	20/30, или 66,7 процента	48,8–80,8 процента
GPT-OSS 20B с RAG	29/30, или 96,7 процента	83,3–99,4 процента

Для повторения исследования сначала фиксируются версии Unity, Ollama, модельные теги и квантование. Затем запускаются тесты алгоритмов и доступности endpoints. QA выполняется в заданном порядке без накопления памяти, а каждая попытка сохраняется сразу после получения результата. После окончания запуск скрипта analyze_results.py формирует таблицу по вопросам, сводку и графики. Повторный запуск анализа над теми же JSONL должен дать те же численные агрегаты; повторный inference может дать другие ответы.

Версия кода, на которой выполнены 90 запросов, отмечена в Git тегом experiment-2026-10-30. После эксперимента внесены исправления политики Planning. Они не используются для переинтерпретации прежних ответов как

новых запросов к модели. Исходные протоколы и история изменений позволяют отличить измеренный исходный вариант от окончательной инженерной редакции.

Для проверки корректности корпуса все вопросы относятся к авторским документам. Ни модель, ни человек не могут получить ответ из внешней энциклопедии о вымышленной деревне. Вместе с тем набор не является скрытым тестом: разработчик видел его при подготовке сценариев. Это ограничивает силу вывода и требует нового независимого корпуса перед заявлением о практической точности в другом проекте.

4.8 Ограничения экспериментального вывода

Корпус состоит из 30 коротких вопросов к одному небольшому миру. Он содержит известные и однозначные ответы, но не охватывает длинные переговоры, неоднозначные намерения, многопользовательские события и изменение лора во время разговора. Для обобщения потребуются независимые корпуса и несколько повторов на вопрос.

Модели различаются размером, обучением и способом исполнения. Эксперимент без RAG и с RAG для одной локальной модели лучше изолирует влияние знаний, чем сопоставление локальной и облачной моделей. Последнее одновременно меняет несколько факторов, поэтому его следует рассматривать как практическое сравнение конфигураций.

Автоматическая оценка по вариантам ответа не является полноценной проверкой галлюцинаций. Отказ сообщить неизвестный факт и уверенный неправильный ответ одинаково могут не совпасть с эталоном, хотя имеют разную природу. Частота галлюцинаций в строгом смысле требует отдельной экспертной разметки утверждений, которая в этом эксперименте не подменяется дополнением до 100 процентов.

Тесты действий покрывают встроенные обработчики и явно сформулированные инварианты. Они не доказывают корректность произвольного пользовательского IGameAction. Особенно важен запрет

побочных эффектов Validate: при расширении registry разработчик обязан сохранять этот контракт.

Свободный диалог не гарантирует соблюдение всех мягких характеристик личности. Ограничения на передачу предметов и доступ к сущностям обеспечиваются C#, но художественная последовательность речи требует более широкого набора оценок с участием пользователей. Текущая работа подтверждает инженерный контур, а не завершённую модель социальной симуляции.

В пределах этих ограничений исходные XML и JSON позволяют ответить на поставленные исследовательские вопросы. Для RQ1 результат ограничен данным корпусом: при одинаковой локальной модели RAG повысил совпадение факта в исходной речи с 6,7 до 66,7 процента, а в принятой реплике — с 3,3 до 53,3 процента. Ответ на RQ2 состоит в измеренных полных медианах 6,33, 6,95 и 30,95 секунды; бесплатная облачная квота существенно входит в пользовательское ожидание. Для RQ3 программные проверки подтвердили сформулированные инварианты на положительных и ошибочных входах, а два реальных прогона продемонстрировали передачу и перемещение. Конечность набора тестов и единичность сквозных прогонов сохраняют ограничения этого вывода.

4.9 Проверка редакторского чат-ассистента

Инженерная редакция 1.1 добавляет диалоговый процесс создания и изменения сцены. Её проверки отделены от QA-эксперимента из 90 запросов и исходных испытаний Unity 2022.3: прежние результаты и тег experiment-2026-10-30 не пересчитывались как испытания нового интерфейса. Наличие интерфейса, корректность исполнения инструментов и способность настоящей модели выполнить просьбу рассматриваются как разные предметы проверки.

Основной пользовательский интерфейс итоговой инженерной редакции — единый чат Tools → WorldWeaver → Чат. В нём доступны история

разговоров, выбор модели, режимы Быстро/Баланс/Глубоко, поле сообщения, прикрепление текстового контекста выделения и просмотр результатов действий. Снимки chat-welcome.png и chat-plus-menu.png фиксируют настоящее окно Unity, выбранную локальную модель и открытое меню начала запроса. Они подтверждают наличие и отображение элементов управления, но не успешное выполнение задачи моделью. Импорт C# требует просмотра плана; восстановление предыдущих файлов вызывается отдельной кнопкой у соответствующего сообщения.

Advanced Inspector с разделами World, Character, Knowledge, Models и Debug используется для ручной настройки и диагностики. Снимки прежнего диалога в Play Mode относятся к runtime-панели NPC, поэтому также не служат проверкой редакторского чата.

Дополнительно выполнен один реальный GUI-диалог с Игнатом в Play Mode сцены, полученной в локальной серии 9B и сохранённой средствами Unity. На вопрос «Во сколько открываются северные ворота?» персонаж ответил: «Северные ворота открываются в шесть утра». Для этого повтора использованы локальная qwen3.5:9b с контекстом 8192 и лимитом 1024 выходных токенов, а также EmbeddingGemma. Панель зарегистрировала принятый ответ и 51902 мс. Рисунок показывает фрагмент настоящей панели диалога; исходный полный снимок сохранён без изменения содержимого. Это одно наблюдение работоспособности runtime и индивидуального знания, а не дополнительный QA-корпус или сравнительный benchmark.

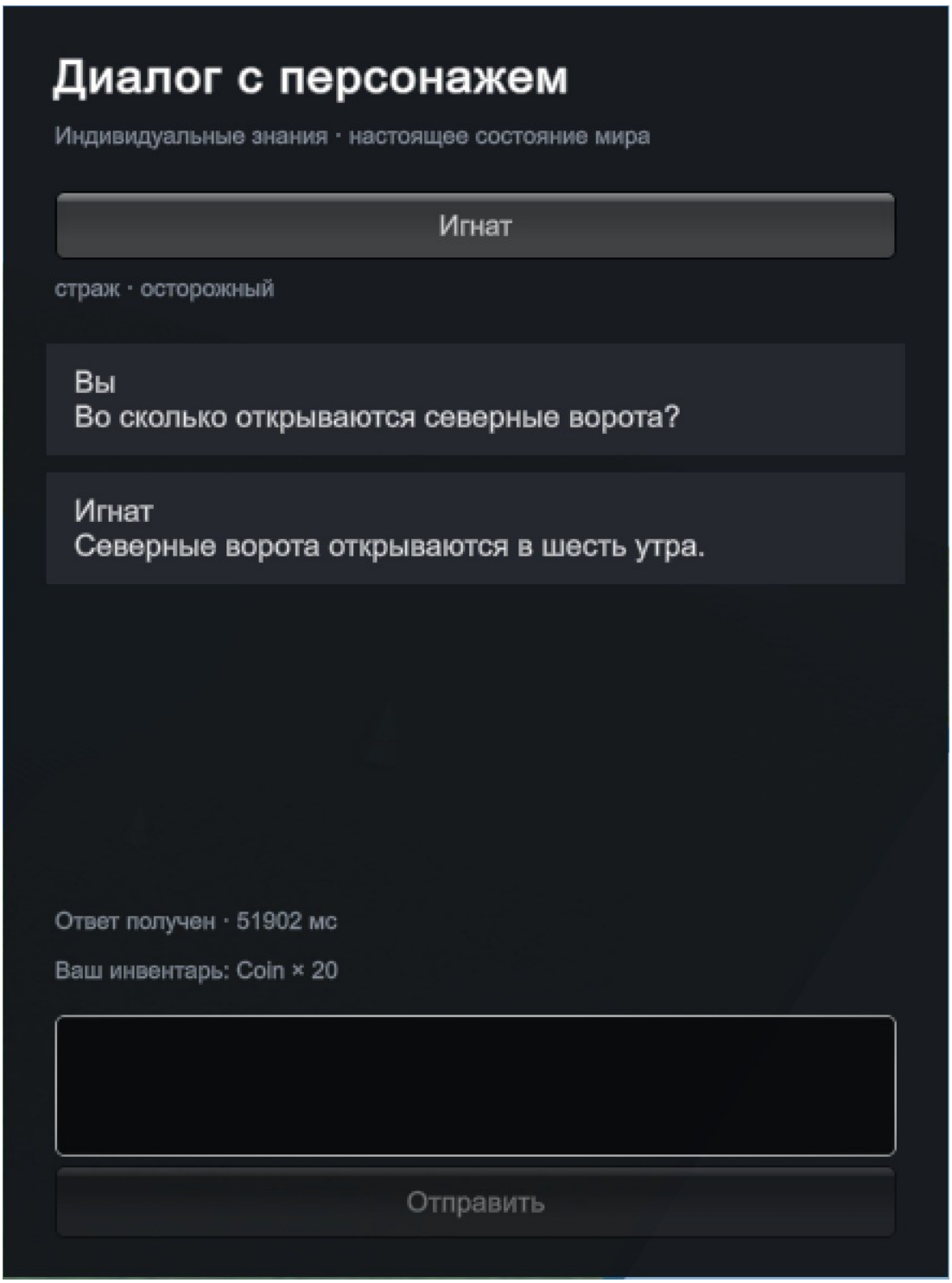


Рисунок 17 — Фрагмент реального GUI-диалога с созданным NPC Игнатом в Play Mode

Программная проверка чата включает инварианты сцены и контроллера, совместимость типизированного ответа с локальным декодером, отдельные импорты рассмотренных LLM-скриптов и два синтетических импорта для проверки журнала C#. В таблице сведены эти группы с условиями интерпретации. Их единицы наблюдения различаются, поэтому доли не объединяются в общую оценку надёжности.

Таблица 12 — Проверка редакторского чат-ассистента

Группа	Сохранённый результат	Условие интерпретации
Unity 6 EditMode редакторской версии	108/108, 0 ошибок, 8,953 с	51 Core, 8 Transport, 27 SceneTool и 22 ChatController; ответы модели контроллера управляемые
PlayMode с регрессией патруля	3/3, 0 ошибок, 1,057 с	Два прежних runtime-сценария и изменение массива точек во время выполнения
Ранний импорт и откат YAxisRotator	11/11 проверок	Исторический отдельный исходник; настоящие компиляция, подключение и Revert
Локальная серия qwen3.5:9b, критерии 2	7/7 заданий	Один последовательный разговор на 0aa9bdf; C# остаётся для просмотра
Облачная серия Classic v4, критерии 2	6/7 формально	Редакция 60192a9; последний отказ связан с лексическим критерием «террейн», исходный JSON сохранён
Импорт и откат WorldWeaverSmoothRotateY	11/11 проверок	Точный новый черновик из локальной серии; отдельный просмотр, компиляция, подключение и Revert
Журнал двух импортов C#	34/34 проверки	Два синтетических исходника одного класса; четыре реальные компиляции и откаты B → A
Полная JSON Schema	33/33 контрактных случаев	Проверки типов, обязательных аргументов и запрещённых полей без inference
Вариант схемы декодера	Те же 33/33 случая	Совпадение контрактной матрицы после строго ограниченного упрощения
Прямые типизированные ответы Ollama	2/2 полных ожидаемых JSON	Заданные JSON-образцы; без исполнения команд в Unity

XML редакторской версии от 31.10.2026, полученный в Unity 6000.3.27f1, фиксирует 108 Passed, 0 Failed, 0 Skipped. Набор включает 56 прежних случаев и 52 дополнительных, поэтому 108 не является числом новых тестов поверх исходного набора. Итоги EditMode и PlayMode

сохранены как chat-agent-editmode.xml и chat-agent-playmode.xml. Последний EditMode выполнялся с 12:47:15 до 12:47:24 UTC на редакции 60192a9. Сохранённый PlayMode выполнялся ранее, с 12:08:09 до 12:08:10 UTC на редакции 0aa9bdf, и не переобозначается как новый прогон после изменения облачного адаптера. Поведение настоящей модели также оценивается отдельно.

Инварианты SceneToolExecutor проверяются настоящими операциями над тестовой сценой. Весь пакет проходит контроль до первой мутации; тесты отклоняют неизвестные поля, ссылку на будущее создание, обращение к удалённому объекту через иной alias, NaN и Infinity в координатах, превышение бюджета геометрии и удаление чужих объектов. При намеренно вызванной ошибке размещения восстановлены Transform существующего объекта и прежний набор GUID ассетов; частичные новые объекты удалены. Для успешного пакета проверена стандартная отмена Unity одной группой. Это подтверждает сформулированные инварианты на данных входах, сохраняя необходимость отдельных проверок при расширении набора инструментов.

На уровне ChatController двадцать две проверки подставляют ILLMProvider, сохраняя настоящий SceneToolExecutor для исполнения планов. В регрессии проверки результата первый ответ ошибочно возвращает complete:true после создания куба вне требуемой позиции. Контроллер запрашивает проверку с действительным instance ID и исходной пользовательской целью, исполняет исправление Transform того же объекта, затем получает подтверждение. Проверены три обращения, два успешных действия и отсутствие дубликата. Другая регрессия вызывает настоящий откат пакета из-за невместимого размещения домов, после которого модельный stub возвращает complete:true с пустыми действиями. Сцена остаётся пустой, а итоговый статус — «Не выполнено», с признаком ошибки. Эти тесты подтверждают механизм проверки и отображения результата;

способность произвольной настоящей модели заметить ошибку ими не измеряется.

Четыре добавленные регрессии освещения проверяют подъём новых Point/Spot над Terrain и водой без снижения высокой позиции, сохранение координат при opt-out, для Directional и вне границ рельефа, явное включение коррекции существующего света с Undo и отказ всему плану при не-boolean on_terrain. Они проверяют однократное размещение, сохраняя отдельную задачу проверки света после последующего изменения высот.

Дополнительные регрессии сохраняют настроенный список KnownEntities при изменении цели NPC, исключают повтор повреждённого assistant-префикса из repair-контекста и проверяют фактическое тело запроса Classic на локальном HTTP-сервере: один объединённый system, пример EditorPlan вместо сырой схемы, без response_format и с пределом 1500 выходных токенов. HTTP 402 с кодом FREE_LIMIT допускает ограниченный повтор, обычная ошибка оплаты не повторяется. Эти проверки программного контракта не измеряют доступность внешнего сервиса.

Отдельная группа проверяет контракт типизированного ответа. Схема содержит 18 инструментов в отдельных ветвях anyOf с одноэлементным enum имени инструмента, четырьмя обязательными полями команды и закрытыми объектами аргументов. Матрица отклоняет строковые resolution и scale рельефа, скаляр вместо трёхмерного transform.scale, недостающий target у scatter и отсутствие точек у patrol. Проверена полнота имён аргументов относительно текущих валидаторов. При этом допустимая схема Draft7 первоначально привела к HTTP 400 при создании grammar в установленной Ollama 0.32.6. Последовательное исключение ограничений показало, что в этой среде достаточно удалить только строковые maxLength от 2000 символов. Полная схема сохранена отдельно; вариант декодера содержит те же типы, enum, required и additionalProperties, но опускает 15 длинных строковых границ. Все исходные ограничения длины и семантики сохраняются в C# валидаторах.

Для сохранённой версии экспорта до изменения порядка свойств create_world выполнены две прямые пробы qwen3.5:2b с think:false, контекстом 8192 и лимитом 256 выходных токенов. По заданным JSON-образцам получены полные create_world с числовыми resolution 129 и scale 0,8, а также create_object с векторными position и scale. Оба ответа завершены штатно и совпали с ожидаемыми типизированными командами; полные времена составили 3134 и 3075 мс. Команды не передавались исполнителю Unity. Эти пробы подтверждают совместимость конкретной схемы и версии декодера, сохраняя отдельную задачу проверки свободного естественно-языкового запроса. Исходные HTTP 400 и ранние ответы только с thinking-токеном сохранены; последние не засчитаны как успешное декодирование команды. Экспорты, хеши и результаты проверки находятся в отдельных JSON-протоколах editor-schema.

Ранний исторический протокол жизненного цикла C# использовал фактический ожидающий draft_script YAxisRotator из ранее сохранённой беседы. До импорта рассмотрен точный исходник и проверено совпадение SHA-256; новые обращения к модели при этой проверке не выполнялись. После явного применения зарегистрированы завершение настоящей компиляции, новый успешный compile_scripts в истории и допустимый тип MonoScript.GetClass. Компонент подключён к специальному тестовому кубу и проверен обратным чтением. Затем куб уничтожен, стандартный Revert удалил отсутствовавший до опыта файл, вторая компиляция завершилась без ошибок, тестовая сцена очищена. Итог chat-code-acceptance.json — 11 из 11, success=true, ручная очистка не требуется. Это положительный путь одного рассмотренного скрипта; корректность любого генерируемого кода, восстановление всех кодировок существующих файлов и все варианты аварийного завершения Editor не исследованы.

Отдельный протокол chat-code-journal-acceptance.json проверяет журнал двух импортов одного класса WorldWeaverJournalRotor. Оба исходника подготовлены вручную для теста и просмотрены по точным

SHA-256: A имеет Revision=1 и скорость 30, B — Revision=2 и скорость 60. После двух реальных импортов получены отдельные сообщения compile_scripts, допустимые типы и фактически подключённые компоненты. В промежутке до компиляции B старый GetClass ещё возвращал A, но стандартный Attach отказал по отсутствию подтверждения текущего хеша, не добавив компонент. Попытка откатить A поверх B также отклонена без мутации. После удаления тестового куба Revert сообщения B восстановил точные байты A и прошёл третью компиляцию; Revert сообщения A удалил отсутствовавшие до опыта исходник и метаданные, после чего прошла четвёртая компиляция. История содержит два успешных restore_compile_scripts, собственная сцена очищена, чужие journal-файлы не изменены. Итог — 34 из 34, success=true, ручная очистка не требуется. Это тест жизненного цикла с двумя синтетическими редакциями, а не ещё два результата генерации модели; аварии во всех фазах и произвольные ошибки компиляции им не покрыты.

Первый сквозной прогон qwen3.5:9b (chat-agent-acceptance-local9b.json, контрольный срез 35fb6c6) получил 7/7 по первоначальным узким критериям, однако этот итог не означает правильного выполнения всех задач. Независимый разбор chat-agent-acceptance-local9b-semantic-review.json обнаружил два из трёх точечных источников света заведомо ниже минимальной отметки всей поверхности Terrain; положение третьего по одной этой границе не оценивалось. Ожидающий просмотра черновик WorldWeaverYRotator также содержал неверную перегрузку Transform.Rotate, перевод в радианы и деление на Time.deltaTime вместо умножения для заданной скорости в градусах в секунду. Проверка формата плана и запрет импорта до просмотра сработали: черновик не импортировался в Unity и не исполнялся. Позднейшая изолированная компиляция точного ожидающего исходника средством mcs со ссылками установленной Unity 6 подтвердила ошибку API кодами CS1502 и CS1503 (exit 1); корректный контрольный вызов с теми же ссылками скомпилирован (exit 0). Это статическая проверка,

а не запуск компонента в Editor или Play Mode. Поэтому сохранённый формальный PASS отделяется от выявленного семантического FAIL. Этот случай обосновывает расширение критериев положения света и содержательного просмотра C#, а исправленная редакция требует нового полного прогона. Исходный JSON сохраняется неизменным; результаты повторения учитываются отдельно.

Повторная локальная серия chat-agent-acceptance-local9b-v2.json завершена 31.10.2026 в 12:33:42 UTC на редакции 0aa9bdf с qwen3.5:9b и режимом «Баланс». В одном последовательном разговоре пройдены 7/7 заданий по критериям версии 2. В отличие от первого узкого протокола свет проверен относительно фактической поверхности Terrain и воды: все три новых Point находились на 3 м выше локальной поверхности (с погрешностью float), в границах Terrain, с range 20 м, достигающим земли. Исходный JSON после завершения зафиксирован SHA-256; команды, ответы, проверки и состояния каждого шага сохранены вместе.

Таблица 13 — Проверка редакторского чат-ассистента

№	Задание пользователя	Фактический результат и граница проверки	Время runner, с
1	Создать лесную долину 160×160, 12 деревьев, 3 дома и 4 камня	Один Terrain 160×160, требуемые количества, свет и одна камера; критерий пройден	132,67
2	Уменьшить height прежнего рельефа до 24 м	Тот же Terrain, height 24 в TerrainData и спецификации; состав декора сохранён	182,68
3	Создать осторожного стража Игната со знанием о воротах	IntelligentNPC и зависимости, роль/личность и документ о шести утра в индивидуальном Scope	188,27
4	Изменить цель Игната, сохранив остальные сведения	Цель существующего NPC изменена; знание, роль и личность сохранены; подключён patrol, движение по сцене этим прогоном не измерено	206,34
5	Добавить три оранжевых Point, сохранив солнце	Три новых источника в границах Terrain, минимум 3 м над поверхностью/водой; range достигает земли, солнце сохранено	230,89
6	Подготовить C# вращения 30°/с без импорта	WorldWeaverSmoothRotateY сохранён как ожидающий просмотра draft; в ходе разговора импорта и подключения нет	164,50

№	Задание пользователя	Фактический результат и граница проверки	Время runner, с
7	Только объяснить модель и состав сцены	Названы qwen3.5:9b и основные объекты; состояние сцены и ожидающий черновик сохранены	109,02

Время в таблице относится к операциям приёмочного runner с возможными несколькими обращениями к модели и проверкой результата; оно не является временем одного токена или одного HTTP-ответа. Проход четвёртого задания подтверждает изменение данных и наличие настроенного пресета, но не нахождение реального маршрута между площадью и воротами. Семь зависимых заданий одного разговора не являются независимой выборкой для оценки общей вероятности успеха.

Точный код шестого запроса этой серии дополнительно прошёл статический просмотр; после завершения разговора он был согласован по SHA-256 для отдельной проверки в Unity. Протокол chat-code-local9b-v2-source-review.json фиксирует точный WorldWeaverSmoothRotateY (SHA-256 сокращённо c734ada7…9026f9, полное значение приведено в протоколе), согласование исходника и успешную изолированную компиляцию mcs со ссылками Unity 6. В нём применяется корректная перегрузка Rotate(Vector3.up, rotationSpeed * Time.deltaTime, Space.Self) при начальной скорости 30. После завершения разговора и отдельного согласования выполнен настоящий Unity-процесс: chat-code-acceptance-local9b-v2.json завершён в 12:35:55 UTC с 11/11 проверок. Прочитаны записанный файл и успешный отчёт компилятора, получен тип, подключён компонент; затем тестовый куб удалён, файл возвращён в исходное отсутствующее состояние, повторная компиляция и очистка тестовой сцены завершены без ошибок. Этот новый протокол учитывается отдельно от раннего YAxisRotator и синтетического журнала 34/34. Покадровая угловая скорость в Play Mode не измерялась; результат подтверждает цепочку работы с одним

просмотренным исходником, сохраняя необходимость проверки любого следующего кода.

Промежуточная облачная серия chat-agent-acceptance-cloud-v3.json с усиленными критериями версии 2 завершилась 5/7: первые пять запросов прошли, включая размещение света. На шестом запросе все три ответа с C# не закрыли корневой JSON-объект последней фигурной скобкой, и допустимый ожидающий план не был получен. На седьмой запрос без изменений модель сначала дала обычное текстовое объяснение, которое строгий разбор отклонил; при исправлении формата содержание сменилось на незапрошенный draft_script. Такой ответ остался ожидающим просмотра и не выполнил исходную просьбу только объяснить сцену. Протокол фиксирует отдельный сбой формата и сохранения намерения при исправлении ответа. Он не является итогом последующей редакции адаптера: оценка её поведения требует нового полного разговора.

Редакция 60192a9 добавила Classic-only восстановление единственной последней скобки корневого объекта и оформление обычного текстового объяснения при явном read-only запросе. Одиннадцать дополнительных регрессий контроллера проверяют сохранение экранированного C# и исходного аудита, передачу принятого нормализованного результата следующему раунду, отказ повреждённым внутренним строкам/скобкам и запрещённым инструментам, запрет такого преобразования для Ollama и сохранение прежнего ожидающего кода при отклонении новой мутации. Также проверены явный запрос без изменений, чтение фактического объекта, слова в кавычках и смешанное задание с настоящим изменением. Все эти случаи входят в 108/108 программных тестов; они используют подставленный ILLMProvider. Локальная серия 7/7 и её кодовый протокол 11/11 относятся к 0aa9bdf: на 60192a9 этот разговор повторно не запускался, а результаты формальной проверки не заменяют нового облачного испытания.

Завершённый новый облачный прогон chat-agent-acceptance-cloud-v4.json на редакции 60192a9 сохранён с формальным результатом 6/7, success=false. Первые шесть заданий прошли; для последнего runner записал единственную причину: объяснение не описало рельеф. Однако сам ответ называет выбранную pollinations:openai-fast, «1 мир, 1 террейн», Игната и фактические количества объектов. Слово «террейн» не входило в лексический список критерия, который искал «долин», «рельеф», terrain или «лес». Поэтому формальный отказ последнего задания не следует трактовать как отсутствие упоминания Terrain. Исходный JSON не исправляется на 7/7: лексический результат и содержательный разбор учитываются раздельно. Независимый разбор chat-agent-acceptance-cloud-v4-semantic-review.json принял семь ограниченных случаев по их заданным критериям, сохранив исходные формальные 6/7. Это не означает достоверности всех фраз: дополнительное утверждение ответа о нахождении всех объектов в заданных координатах не подтверждено; если понимать его как границы Terrain 160×160, ему противоречит обзорная камера с x=88 и z=−92,8 при границах от −80 до 80. Исходник RotatorY из шестого запроса отдельно просмотрен и скомпилирован mcs со ссылками Unity (exit 0) в chat-code-cloud-v4-source-review.json; импорта, подключения и выполнения этого облачного черновика в Unity не было. Эта проверка выявляет ограничение автоматического оценивания и точности объяснения; результат одного разговора не оценивает устойчивость на произвольных формулировках.

Программные 108/108, контрактные 33/33 и два JSON-образца не заменяют сквозной разговор, в котором модель самостоятельно выбирает действия. Её протокол должен связывать исходную просьбу, текст ответа и попытки исправления с фактическим состоянием сцены до и после, указывая причины несоответствия. Количества объектов, знания и цель NPC, число новых источников света и отсутствие импорта до просмотра подтверждаются по компонентам Unity. Число источников само по себе недостаточно для проверки освещения: дополнительно требуется сопоставить их фактическое

положение с поверхностью Terrain и заданной областью сцены. Художественное качество, время обучения начинающего разработчика и удобство по сравнению с Unity Assistant этими протоколами не измеряются.

ЗАКЛЮЧЕНИЕ

В работе разработан WorldWeaver — подключаемый инструментарий Unity для автоматизированного создания динамических игровых миров. Редакция 1.1 включает основной чат на UI Toolkit, UPM-пакет, контракты EditorPlan, TerrainSpec и NpcDecision, процедурный компоновщик окружения, NPC с индивидуальными знаниями, модельные маршруты и проверяемые операции сцены. Прежнее окно с пятью разделами сохранено как Advanced Inspector.

Анализ аналогов определил интеграционный характер вклада. Редакторские чаты, вызовы инструментов и генерация кода уже представлены в современных продуктах и не объявляются изобретёнными в работе. Специализация WorldWeaver состоит в соединении цикла «план — инструмент — фактическое состояние» с воспроизводимым рельефом и готовым runtime-контуром NPC. Возможности Unity Assistant изучены по документации; сравнительное измерение его успешности и скорости не выполнялось.

Для генерации мира реализован переход от естественного языка к ограниченной спецификации и далее к алгоритмической heightmap. WorldComposer добавляет слои поверхности, воду, стилизованные деревья, камни и дома, освещение и камеру. Частичное обновление позволяет менять параметры существующего мира, сохраняя окружение. Геометрия строится из примитивов и программных мешей; генерация ригованного или импортированного художественного ассета этим не подтверждается. Повторение TerrainSpec и seed позволяет исследовать детерминированный результат отдельно от вероятностного вывода LLM.

Редакторский исполнитель предоставляет 16 инструментов сцены и наблюдения; ещё два инструмента обслуживают просмотренный C#. Предконтроль охватывает весь план, цели предыдущих созданий, типы аргументов, диапазоны и бюджеты. Изменения сцены группируются в Undo,

а исполнительная ошибка сопровождается восстановлением сцены и очисткой новых ассетов. После успешного пакета при наличии раундов модель получает актуальное состояние для проверки исходной задачи. Это делает ошибку или неполный результат наблюдаемыми, но не гарантирует, что языковая модель всегда исправит их.

Для NPC выделены статические знания, события и разговорная память. Scope ограничивает документы до ранжирования. Embeddings и vector store отделены интерфейсами, что позволяет заменить backend без изменения логики персонажа. Ограниченный контекст исключает необходимость передавать всю историю разговора при каждом запросе.

Предложенные моделью последствия проходят строгий разбор и предварительную проверку. Инвентарь изменяется после проверки всего набора встроенных действий, а перемещение исполняет NavMeshAgent. Это устраняет прямую зависимость состояния сцены от формулировки speech, хотя не гарантирует художественную или смысловую безошибочность любой реплики.

В сохранённом QA-исследовании версии 1.0 на фиксированном корпусе из 30 вопросов RAG повысил долю совпадений известного факта в исходной речи локальной Qwen3.5 2B с 2/30 до 20/30. Облачная GPT-OSS 20B с RAG дала 29/30, включая один ошибочный API-запрос в общем знаменателе. Для принятой игроком речи обе RAG-конфигурации показали 16/30: проверки действий отсекли часть фактически подходящих ответов. Эти показатели подтверждают пользу внешних знаний и одновременно демонстрируют, что retrieval и качество исходной реплики недостаточны для оценки целого игрового pipeline.

Для версии 1.0 в Unity 6 были пройдены 56 EditMode-проверок и два детерминированных PlayMode-теста. Два отдельных прогона с реальными маршрутами завершили ограниченный переговорный сценарий, передали по одному Crystal и вызвали движение к House. У локального NPC сохранилось смысловое расхождение речи с исполненной передачей. Дополнительный

GUI-повтор завершил передачу, но вернул бездействие вместо движения; устойчивость автономного планирования остаётся ограниченной. Для Terrain все 40 парных генераций совпали; медиана алгоритма составила от 2,237 мс при разрешении 65 до 136,130 мс при 513. Эти измерения и проверка чистой установки базовой версии не переатрибутируются новому чату.

В Unity 6000.3.27f1 для редакции 1.1 пройдены 108 из 108 EditMode-проверок на редакции 60192a9. Сохранённые 3 из 3 PlayMode-проверок относятся к более ранней редакции 0aa9bdf. Новый набор включает реальную работу исполнителя с объектами и AssetDatabase, откат ошибочного пакета, границы владения, частичное обновление Terrain и цикл контроллера с подставленным ILLMProvider. Многошаговая регрессия проверяет, что преждевременное complete:true не завершает цикл до получения фактического состояния и исправления параметра. Дополнительные регрессии проверяют положение новых Point/Spot над Terrain и водой при однократном размещении. Эти результаты подтверждают программные инварианты; они не являются долей успешных запросов настоящей модели.

Повторный локальный разговор qwen3.5:9b на редакции 0aa9bdf в режиме «Баланс» завершил 7/7 заданий по усиленным критериям: создание долины, частичное обновление высот, создание и изменение NPC, размещение света, подготовку C# для просмотра и объяснение сцены без изменений. Для новых Point подтверждены положение в границах Terrain, зазор 3 м над поверхностью/водой и достижение земли заданной дальностью. Это один последовательный разговор на конкретной редакции; изменение цели и подключение patrol не считаются доказательством фактической навигации между площадью и воротами. Ранний локальный формальный результат 7/7 с подземными источниками света и некорректным черновиком сохранён отдельно и не переобозначен как успешный итог исправлений.

Новый WorldWeaverSmoothRotateY из повторного разговора прошёл просмотр точного исходника, изолированную компиляцию со ссылками Unity и затем отдельные 11/11 проверок настоящего импорта, компиляции,

подключения и возврата файлов в Unity. Протокол chat-code-acceptance-local9b-v2.json подтверждает этот конкретный исходник, а не все ответы модели; угловая скорость по кадрам в Play Mode не измерялась.

Ранний исторический протокол chat-code-acceptance.json подтвердил 11 из 11 проверок просмотренного компонента YAxisRotator: чтение записанного файла, настоящую компиляцию Unity, получение типа MonoBehaviour, подключение к объекту и восстановление исходного файлового состояния с повторной компиляцией и очисткой тестовой сцены. Прогон не выполнял новых LLM-запросов и не измерял вращение объекта в Play Mode. Поэтому его результат относится к цепочке импорта, компиляции, подключения и восстановления, а не к общей правильности генерируемого кода или качества модели.

Последующая редакция 60192a9 ограниченно нормализует формат ответов Classic с сохранением исходной и преобразованной записей и блокирует мутации при явно распознанной просьбе только объяснить для всех провайдеров. Новый локальный разговор на этой редакции не выполнялся. Поэтому его прежние 7/7 и 11/11 сохраняют версию и область применимости, а изменённый облачный маршрут имеет собственный протокол v4 с формальными 6/7. Последний отказ связан со словом «террейн», отсутствующим в словаре критерия; независимый разбор принял семь ограниченных случаев, сохранив исходный итог. Дополнительное утверждение модели о нахождении всех объектов в заданных координатах осталось неподтверждённым, поэтому весь текст объяснения не объявляется безошибочным. Для облачного черновика подтверждена только отдельная статическая компиляция, а Unity-импорт 11/11 относится к локальному исходнику. В интерфейсе после применения или возврата остаётся доступный для чтения снимок кода; повторный импорт возможен только из действующего ожидающего плана.

Основные ограничения связаны с малым корпусом базового эксперимента, отсутствием постоянного сохранения игровой памяти NPC,

ограниченной выразительностью TerrainSpec и best-effort характером бесплатного cloud API. Сохраняемая история редакторского чата отделена от этой игровой памяти. Голос, импорт PDF и автоматическая навигация по произвольному новому ландшафту не входят в подтверждённые функции. Результаты ядра 1.0 в Unity 2022 не подтверждают совместимость всего редакторского агента 1.1 с Unity 2022.

Просмотр C# и компилятор не создают sandbox: принятый компонент выполняется с правами Unity, а успешная компиляция не доказывает отсутствие нежелательного поведения. Отмена сцены также не равна удалению всех оставшихся ассетов после успешной генерации. Направления дальнейшей работы включают независимые повторные сценарии редакторского агента, оценку полноты пользовательского задания и интерфейса, формализацию дополнительных игровых политик, долговременную память и расширение корпусов знаний.

Созданный инструментарий предоставляет основу для повторного использования и дальнейших исследований в Unity-проектах. Его существенное свойство заключается в наблюдаемой границе между предложением модели, проверкой встроенной операции и фактически прочитанным результатом. Новый редакторский процесс расширяет эту архитектуру, сохраняя отдельную ответственность разработчика за просмотр программного расширения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Unity Technologies. Unity 6000.3.27f1 Release Notes. — 2026. — URL: https://unity.com/releases/editor/whats-new/6000.3.27f1 (дата обращения: 30.10.2026). — Текст: электронный.

2. Unity Technologies. Git dependencies Unity Manual. — 2022–2026. — URL: https://docs.unity3d.com/2022.3/Documentation/Manual/upm-git.html (дата обращения: 30.10.2026). — Текст: электронный.

3. Unity Technologies. TerrainData SetHeights. — 2026. — URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/TerrainData.SetHeights.html (дата обращения: 30.10.2026). — Текст: электронный.

4. Unity Technologies. NavMeshBuilder BuildNavMeshData. — 2026. — URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/AI.NavMeshBuilder.BuildNavMeshData.html (дата обращения: 30.10.2026). — Текст: электронный.

5. Unity Technologies. Configure AI Gateway Assistant 2.21. — 2026. — URL: https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.21/manual/integration/ai-gateway-get-started.html (дата обращения: 31.10.2026). — Текст: электронный.

6. Convai. Configure character actions. — 2026. — URL: https://github.com/Conv-AI/Convai-Documentation/blob/main/plugins-and-integrations/convai-unity-sdk/features/character-actions/configuring-actions.md (дата обращения: 30.10.2026). — Текст: электронный.

7. Inworld AI. Character Studio deprecation and Agent Runtime. — 2026. — URL: https://docs.inworld.ai/character-studio-deprecation (дата обращения: 31.10.2026). — Текст: электронный.

8. NVIDIA. NVIDIA ACE for Games. — 2026. — URL: https://developer.nvidia.com/ace-for-games (дата обращения: 30.10.2026). — Текст: электронный.

9. Promethean AI. Promethean AI. — 2026. — URL: https://www.prometheanai.com/ (дата обращения: 30.10.2026). — Текст: электронный.

10. Procedural Worlds. Procedural Worlds. — 2026. — URL: https://www.procedural-worlds.com/ (дата обращения: 30.10.2026). — Текст: электронный.

11. UndreamAI. LLMUnity. — 2026. — URL: https://github.com/undreamai/LLMUnity (дата обращения: 30.10.2026). — Текст: электронный.

12. LangChain. LangChain knowledge base. — 2026. — URL: https://github.com/langchain-ai/docs/blob/main/src/oss/langchain/knowledge-base.mdx (дата обращения: 30.10.2026). — Текст: электронный.

13. LlamaIndex. Using VectorStoreIndex. — 2026. — URL: https://developers.llamaindex.ai/python/framework/module_guides/indexing/vector_store_index/ (дата обращения: 30.10.2026). — Текст: электронный.

14. Lewis P., Perez E., Piktus A. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. — 2020. — URL: https://arxiv.org/abs/2005.11401 (дата обращения: 30.10.2026). — Текст: электронный.

15. Park J. S., O'Brien J. C., Cai C. J. et al. Generative Agents Interactive Simulacra of Human Behavior. — 2023. — URL: https://arxiv.org/abs/2304.03442 (дата обращения: 30.10.2026). — Текст: электронный.

16. Yao S., Zhao J., Yu D. et al. ReAct Synergizing Reasoning and Acting in Language Models. — 2022–2023. — URL: https://arxiv.org/abs/2210.03629 (дата обращения: 30.10.2026). — Текст: электронный.

17. Ollama. Structured outputs Ollama. — 2026. — URL: https://docs.ollama.com/capabilities/structured-outputs (дата обращения: 30.10.2026). — Текст: электронный.

18. Ollama and Qwen. Qwen3.5 2B Ollama model card. — 2026. — URL: https://ollama.com/library/qwen3.5:2b (дата обращения: 30.10.2026). — Текст: электронный.

19. Ollama and Google. EmbeddingGemma Ollama model card. — 2025–2026. — URL: https://ollama.com/library/embeddinggemma (дата обращения: 30.10.2026). — Текст: электронный.

20. Google AI for Developers. EmbeddingGemma inference. — 2026. — URL: https://ai.google.dev/gemma/docs/embeddinggemma/inference-embeddinggemma-with-sentence-transformers (дата обращения: 30.10.2026). — Текст: электронный.

21. OpenAI. Chat API reference. — 2026. — URL: https://developers.openai.com/api/reference/resources/chat (дата обращения: 30.10.2026). — Текст: электронный.

22. Groq. Structured Outputs Groq. — 2026. — URL: https://console.groq.com/docs/structured-outputs (дата обращения: 30.10.2026). — Текст: электронный.

23. Pollinations. Pollinations Classic API. — 2026. — URL: https://github.com/pollinations/pollinations/blob/master/APIDOCS.md (дата обращения: 31.10.2026). — Текст: электронный.

24. Unity Technologies. Unity Test Framework introduction Unity 6.3. — 2026. — URL: https://docs.unity3d.com/6000.3/Documentation/Manual/test-framework/test-framework-introduction.html (дата обращения: 31.10.2026). — Текст: электронный.

25. Unity Technologies. Undo RegisterCompleteObjectUndo. — 2026. — URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Undo.RegisterCompleteObjectUndo.html (дата обращения: 30.10.2026). — Текст: электронный.

26. JSON Schema. JSON Schema Draft 2020 12. — 2020. — URL: https://json-schema.org/draft/2020-12 (дата обращения: 30.10.2026). — Текст: электронный.

27. ВолгГТУ. Бланки учебно методического управления ВолгГТУ. — 2019–2026. — URL: https://www.vstu.ru/obrazovanie/umu/blanki/ (дата обращения: 30.10.2026). — Текст: электронный.

28. Росстандарт. ГОСТ 19.201-78 Техническое задание Требования к содержанию и оформлению. — 1978. — URL: https://protect.gost.ru/gost/details/b64061cf-6bb2-4c75-aee9-18bb5e26e554 (дата обращения: 30.10.2026). — Текст: электронный.

29. Росстандарт. ГОСТ 7.32-2017 Отчет о научно исследовательской работе. — 2017. — URL: https://protect.gost.ru/gost/details/7d280e43-7036-4a69-8e6e-d15867028343 (дата обращения: 30.10.2026). — Текст: электронный.

30. Росстандарт. ГОСТ Р 7.0.100-2018 Библиографическая запись Библиографическое описание. — 2018. — URL: https://protect.gost.ru/gost/details/389dfd5a-535a-458a-81c3-14b729b1cee1 (дата обращения: 30.10.2026). — Текст: электронный.

31. CoplayDev. MCP for Unity. — 2026. — URL: https://github.com/CoplayDev/unity-mcp (дата обращения: 30.10.2026). — Текст: электронный.

32. Shaker N., Togelius J., Nelson M. J. Procedural Content Generation in Games. — 2016. — URL: https://pcgbook.com/ (дата обращения: 30.10.2026). — Текст: электронный.

33. LM Studio. LM Studio OpenAI compatibility. — 2026. — URL: https://lmstudio.ai/docs/developer/openai-compat (дата обращения: 30.10.2026). — Текст: электронный.

34. ggml-org. llama.cpp. — 2026. — URL: https://github.com/ggml-org/llama.cpp (дата обращения: 30.10.2026). — Текст: электронный.

35. Kyzr O. Procedural Generation of Outdoor Scenes Bachelor Thesis. — 2023. — URL: https://dcgi.fel.cvut.cz/en/theses/2023/kyzrondr/ (дата обращения: 31.10.2026). — Текст: электронный.

36. Marcher F. Procedural Terrain Generation Tool in Unity Master Thesis DOI 10.3217/rb5eq-fv649. — 2024. — URL: https://repository.tugraz.at/publications/rb5eq-fv649 (дата обращения: 31.10.2026). — Текст: электронный.

37. Wakeford O. R. J. Procedural Terrain Generation with Fixed Points Bachelor Thesis. — 2025. — URL: https://dspace.cuni.cz/handle/20.500.11956/202622 (дата обращения: 31.10.2026). — Текст: электронный.

38. Aslan P. Understanding LLMs for Game NPCs An Exploration of Opportunities Master Thesis. — 2024. — URL: https://kth.diva-portal.org/smash/get/diva2%3A1938971/FULLTEXT01.pdf (дата обращения: 31.10.2026). — Текст: электронный.

39. Pollinations. Pollinations Classic text adapter: limits and request normalization. — 2026. — URL: https://github.com/pollinations/pollinations/blob/73311dad9465c7ddd0ceb9cdc0f2ab8b171f6dbb/text.pollinations.ai/cloudflare-cache/src/origin.js (дата обращения: 31.10.2026). — Текст: электронный.

40. ggml-org. llama.cpp GBNF Guide JSON Schema conversion and known limitations. — 2026. — URL: https://github.com/ggml-org/llama.cpp/blob/master/grammars/README.md (дата обращения: 31.10.2026). — Текст: электронный.

41. ochafik et al. JSON schema to grammar improvements pull request 5978. — 2024. — URL: https://github.com/ggml-org/llama.cpp/pull/5978 (дата обращения: 31.10.2026). — Текст: электронный.

ПРИЛОЖЕНИЕ А Техническое задание

Полная редакция технического задания поставляется отдельным документом «Техническое задание WorldWeaver». Приложение фиксирует связь обязательных требований с реализацией. Приёмка выполняется по согласованным критериям и сохранённым протоколам.

Таблица 14 — ПРИЛОЖЕНИЕ А Техническое задание

Требование	Основные элементы реализации	Источник проверки
Основной чат	WorldWeaverChatWindow, ChatController	Сценарий беседы, история и результаты инструментов
Контракт и проверка сцены	EditorPlan, EditorToolSchema, SceneToolExecutor	EditMode и снимки до/после в приёмочном JSON
Создание и изменение мира	TerrainSpec, TerrainGenerator, WorldComposer	Seed, компоненты окружения и частичное обновление
Модельные маршруты	ModelRouter, OrchestratorProfile	Назначения профилей и фактический endpoint
Индивидуальный RAG	RagIndex, InMemoryVectorStore	Scope-тесты и отдельный QA-протокол 1.0
Проверка игровых действий	NpcDecision, ActionDispatcher	Amount, target, permissions и double spend
Перемещение	WorldAction, NavMeshAgent, SceneBehavior	Раздельные проверки NavMesh и встроенного патруля
Просмотр и компиляция C#	ScriptDrafts, CompilationPipeline	Импорт, события компилятора, подключение и восстановление
Диагностика	Попытки EditorPlan и PipelineTrace	Запрос, схема, ответ, ошибка и прочитанное состояние

Таблица указывает места проверки, а не объявляет все сценарии успешными. Исходное исследование версии 1.0 и новая приёмка редакторского агента имеют отдельные версии кода и протоколы. Полные требования FR-01–FR-24 и критерии AT-01–AT-19 приведены в отдельном ТЗ.

ПРИЛОЖЕНИЕ Б Приемочные сценарии

Таблица 15 — ПРИЛОЖЕНИЕ Б Приемочные сценарии

Код	Начальные условия	Действие	Ожидаемое состояние
TC-01	Unity 6000.3	Подключить UPM и открыть основной чат	Интерфейс, модель и компоненты доступны
TC-02	TerrainSpec seed 42	Дважды вызвать Generate	Все высоты совпадают
TC-03	resolution 128	Разобрать спецификацию	Контролируемый отказ
TC-04	NPC знает village, guard имеет secret	Retrieval от merchant	Secret не попадает в выдачу
TC-05	NPC Crystal 10, Player 0	GiveItem 1	Остатки 9 и 1
TC-06	Лимит передачи 1	GiveItem 2	Остатки не меняются
TC-07	NPC Crystal 10, лимит 100	Два GiveItem по 6	Пакет отклонён целиком
TC-08	Цели Admin нет в KnownEntities	GiveItem Admin	Отказ Unknown target
TC-09	NPC на NavMesh	MoveTo House	Агент перемещается по пути
TC-10	Агент движется	Follow, затем StopMovement	Following, затем остановка
TC-11	HTTP-сервер возвращает 429, затем 200	Запрос с одним повтором	Ровно два обращения
TC-12	HTTP-сервер не отвечает	Cancellation или timeout	Задача завершается без зависания
TC-13	Реальный Ollama доступен	Диалог и Planning	JSON, проверка и реальные последствия
TC-14	Бесплатный API доступен	Смена Dialogue route	Ответ cloud, Planning остаётся local
TC-15	Пустая тестовая сцена	Попросить долину и точное число объектов	Один Terrain, заданные количества, свет и камера
TC-16	Мир из TC-15	Уменьшить height до 24	Изменён прежний Terrain, декор сохранён
TC-17	Мир и доступная модель	Создать Игната с личностью и знанием	IntelligentNPC, Inventory, WorldEntity и scoped KnowledgeBase
TC-18	Игнат создан	Изменить только цель	Новая цель и сохранённые остальные сведения

Код	Начальные условия	Действие	Ожидаемое состояние
TC-19	Солнце уже существует	Добавить три оранжевых Point Light	Три новых света в границах Terrain, минимум 3 м над поверхностью/водой, range достигает земли; солнце сохранено
TC-20	Чат в Edit Mode	Попросить C# без импорта	Непустой PendingPlan, сцена и файлы не изменены
TC-21	Ожидающий черновик	Спросить модель и состав сцены	Объяснение без мутации, черновик сохранён
TC-22	План с ошибкой или исполнительным отказом	Выполнить предконтроль и откат	Чужое состояние сохранено, новые ассеты учтены
TC-23	Просмотренный отдельный C#	Импортировать, подключить и восстановить	Раздельный read-back файлов, компилятора и компонента

TC-02–TC-14 сохраняют проверку базового runtime-контура. TC-15–TC-21 соответствуют семи продолжениям одного редакторского разговора; это зависимая последовательность, а не семь независимых измерений надёжности. Условие «ожидаемое состояние» задаёт критерий, а фактический исход и причина отказа берутся из нового протокола. TC-22 и TC-23 проверяются отдельно от модельной серии.

Исходный набор QA-сценариев содержит 30 строк в results/scenarios.json. В каждом случае зафиксированы вопрос, ожидаемые варианты и source ID. Файл metrics-per-question.csv связывает эти сценарии с наблюдавшимся ответом и агрегатами. При повторном испытании требуется сохранить версии моделей и условия запуска, а не сравнивать только итоговые проценты.

Дополнительная activity-диаграмма показывает обработку ошибки и успешного ответа. Это схема реализованного pipeline, а не обещание автоматического исправления любого ответа модели.

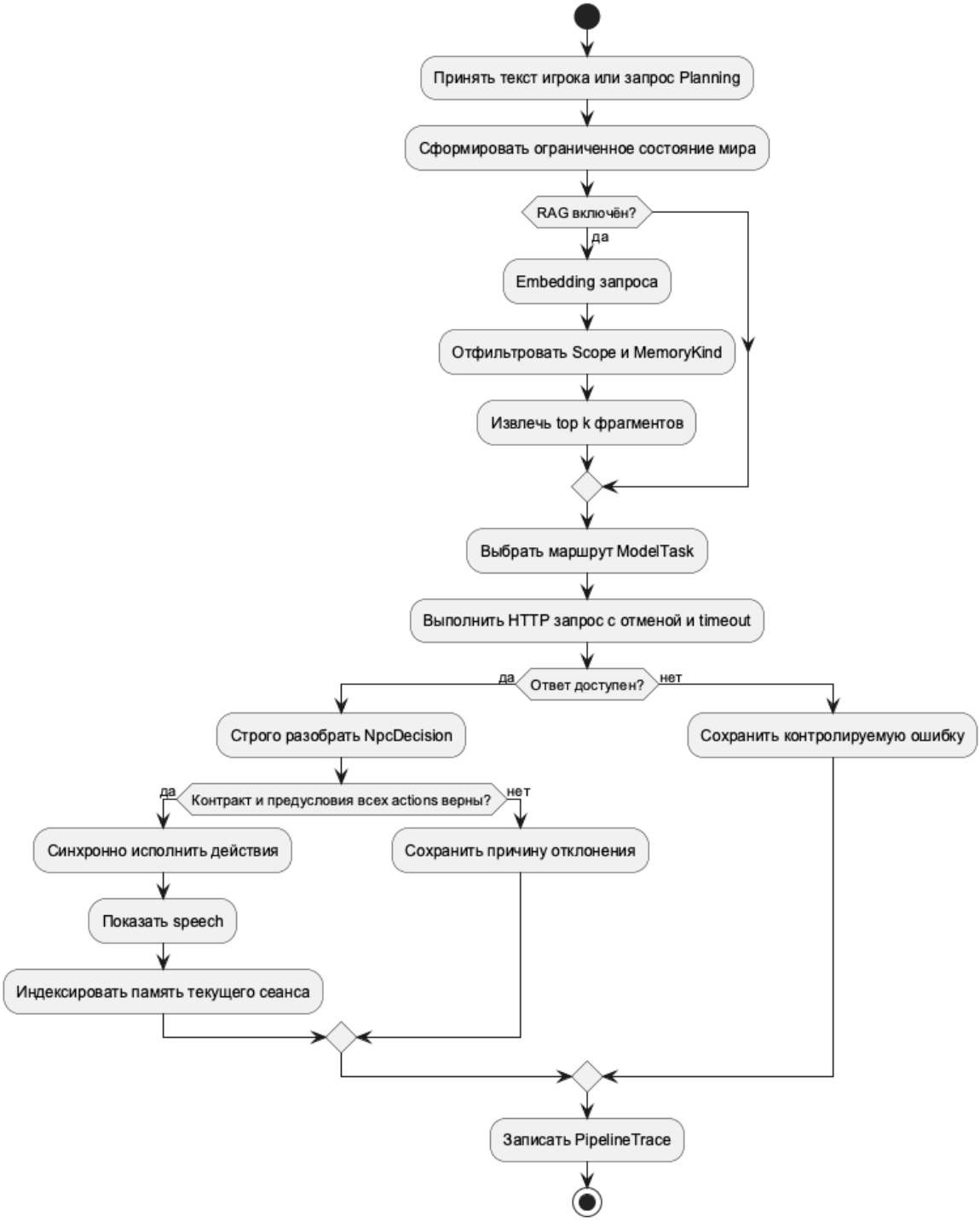


Рисунок 18 — Обработка запроса и отказов pipeline

ПРИЛОЖЕНИЕ В Корпус вопросов к индивидуальным знаниям

Корпус базового исследования 1.0 содержит десять тематических документов и по три вопроса к каждому. Он зафиксирован тегом experiment-2026-10-30 и используется для отдельного сравнения трёх QA-конфигураций. В таблице приведены автоматические варианты совпадений. Частичные основы имени, например «Эрин», позволяют учитывать падежные формы, но не заменяют экспертную проверку смысла.

Таблица 16 — ПРИЛОЖЕНИЕ В Корпус вопросов к индивидуальным знаниям

ID	Вопрос	Source ID и допустимые варианты
qa-01	Как называется наша деревня?	village: Лунный Брод, Лунном Броде
qa-02	Кто основал деревню?	village: Аэлл
qa-03	На какой реке стоит деревня?	village: Тихая, Тихой
qa-04	Как называется лечебный кристалл?	crystal: Зорецвет
qa-05	Где добывают лечебный кристалл?	crystal: Семи Ветров
qa-06	Какова обычная цена одного кристалла?	crystal: 7, семь
qa-07	Как зовут лекаря деревни?	healer: Эрин
qa-08	Где принимает лекарь?	healer: западн
qa-09	Какое растение используют для настоя от лихорадки?	healer: серебролист
qa-10	Кто был твоим учителем?	merchant: Берен
qa-11	Каких животных ты боишься?	merchant: волк
qa-12	Когда открывается твоя лавка?	merchant: рассвет
qa-13	Где стоит твой дом?	house: восточн
qa-14	Что висит над дверью твоего дома?	house: медный полумесяц, медного полумесяца
qa-15	Что ты обычно делаешь после торговли?	house: осматри, осмотр
qa-16	Как называется деревенский колодец?	well: Зеркальн
qa-17	Какова глубина колодца в локтях?	well: 18, восемнадцать
qa-18	Кто проверяет воду колодца?	well: Тарен
qa-19	В какой день проходит праздник Урожая?	festival: 12, двенадцат

ID	Вопрос	Source ID и допустимые варианты
qa-20	Какой символ у праздника Урожая?	festival: белый фонарь, белого фонаря
qa-21	Какую песню поют на празднике?	festival: Семь мостов
qa-22	Из какого материала построен северный мост?	bridge: красного камня, красный камень
qa-23	Сколько стоит проход по северному мосту?	bridge: 2, две
qa-24	До какого времени разрешён проход по мосту после наводнения?	bridge: полудн
qa-25	Какие отметки обозначают вход в шахту?	mine: три сини, тремя сини
qa-26	Как зовут старшего шахтёра?	mine: Дорин
qa-27	Какая штольня закрыта?	mine: южн
qa-28	Где находится деревенский архив?	library: башне Ив, башня Ив
qa-29	Как зовут хранительницу архива?	library: Сели
qa-30	Как называется самая старая книга в архиве?	library: Карта зимнего неба

ПРИЛОЖЕНИЕ Г Порядок установки и повторения проверки

Установка 1.1 и начало работы

Для редакторского чата используется Unity 6000.3.27f1. Можно открыть поставленный UnityProject с локальной file-зависимостью либо установить пакет в отдельный совместимый проект через Package Manager → Install package from tarball, выбрав com.worldweaver.toolkit-1.1.0.tgz. Зависимость Newtonsoft JSON 3.2.1 должна разрешиться через UPM. Для Git-установки используется URL репозитория с path=/Packages/com.worldweaver.toolkit и зафиксированным SHA редакции 1.1. Доступ к закрытому репозиторию предоставляет локальная авторизация Git; credentials в URL не включаются. Проверка чистого импорта 1.0 относится к исходной версии и сама по себе не подтверждает установку 1.1.

Локальный daemon Ollama должен быть доступен по http://127.0.0.1:11434. Начальное назначение чата — qwen3.5:9b; для retrieval NPC требуется embeddinggemma. Исторические qwen3.5:2b и qwen2.5-coder:latest сохраняются для повторения базовых исследований и runtime-сценариев. Модель 9B занимает значительную часть доступных 16 ГБ; выполнение в режиме «Баланс» с проверкой результата может занимать несколько минут. Веса не входят в поставку исходников; имена тегов и фактические digests записываются перед повторением. Изменение embedding-модели требует перестроения индекса знаний.

Основной путь — Tools → WorldWeaver → Чат. В одной беседе выбираются модель и режим, вводится описание мира, затем уточняются его параметры и создаётся NPC. Меню «+» подготавливает обычный запрос или прикрепляет сведения о выделении. История хранится в UserSettings проекта. Advanced Inspector с WORLD/NPC/KNOWLEDGE/MODELS/DEBUG используется для детального контроля и повторения прежнего процесса. Создание Demo in New Scene является отдельной командой демонстрации

ядра; для обычного нового мира чат изменяет активную сцену и не очищает её.

При наличии бюджета после исполнения инструментов модель получает актуальное состояние для проверки исходного задания. Ответ complete:true с непустым actions остаётся намерением; результат следует сверять с Terrain, компонентами и параметрами. Если бюджет закончился после пакета, «Шаг выполнен» не обозначает завершённую дополнительную проверку моделью. Остановка сохраняет уже выполненные шаги. Стандартный Undo сцены и возврат C# являются разными действиями. Во время запроса текущий этап сопровождается прошедшим временем; этот таймер не является прогнозом оставшейся длительности.

Программные проверки и семь запросов агента

Сначала выполняются EditMode и детерминированные PlayMode-проверки. В Test Runner следует выбрать соответствующие сборки и сохранить XML. Скрипт run_unity.py edit запускает EditMode, а play использует фильтр NavigationTests и AssistantBehaviorTests для обеих групп детерминированных проверок версии 1.1. Проверки с подставленным ILLMProvider не требуют настоящей модели и не считаются измерением её успешности. Последний подтверждённый EditMode на 60192a9 содержит 108/108; сохранённый PlayMode 3/3 выполнен на 0aa9bdf. При повторении эти версии и даты записываются отдельно. До batch-запуска закрывается другая копия Editor с тем же projectPath.

Семь запросов ChatAcceptanceRunner выполняются в одном новом чате: создание долины 160×160 с 12 деревьями, 3 домами и 4 камнями; изменение прежнего height до 24; создание Игната со знанием о воротах; изменение его цели; создание ровно трёх оранжевых Point Light; подготовка C# для вращения по Y без импорта; объяснение выбранной модели и фактического состава сцены без мутации. Точные формулировки находятся в исходнике runner и записываются в JSON вместе с ответами. В версии

критериев 2 новые Point дополнительно проверяются внутри границ Terrain, не менее чем на 3 м над поверхностью и сгенерированной водой; их range должен достигать земли. Исходный протокол с узким подсчётом света не заменяет этот повтор.

Явная команда Tools → WorldWeaver → Testing → Chat Acceptance создаёт пустую тестовую сцену. В GUI перед этим предлагается сохранить текущую сцену. Для воспроизведения следует использовать выделенный тестовый проект, а не рабочую сцену потребительского проекта. Обычные инструменты чата не создают и не очищают сцену; создание отдельной сцены относится только к запущенному испытанию.

Для batch-запуска WW_UNITY задаёт абсолютный путь к Unity, WW_PROJECT — к тестовому проекту, WW_AGENT_REPORT — новый абсолютный путь JSON. Эти значения выбираются до команды; опубликованный файл не перезаписывается. Пример запуска из каталога проекта:

```
WW_CHAT_MODEL=ollama:qwen3.5:9b \
WW_CHAT_OUTPUT="$WW_AGENT_REPORT" \
"$WW_UNITY" -batchmode -nographics \
-projectPath "$WW_PROJECT" \
-executeMethod \
WorldWeaver.Editor.Assistant.ChatAcceptanceRunner.Run
```

Флаг -quit не добавляется: асинхронный runner завершает batch-процесс после записи результата. В JSON сохраняются выбранная модель, условия запуска, prompt, попытки с запросом и схемой, результаты инструментов, состояние до и после, время и причины неуспеха. Нужно проверить completed, passed, success и причины каждого запроса, а также файлы истории по historyFolder. Для каждой попытки сопоставляются RawResponse, NormalizedResponse, RepairKind и Accepted: узкая нормализация Classic может сохранить исходный повреждённый ответ и отдельный принятый вариант. Это не ручная замена команд; локальный Ollama этот адаптер не использует. Прерванная серия и невыполненные зависимые задания не объявляются успешными. Один последовательный разговор не оценивает общую вероятность выполнения произвольной задачи. Сохранённые local9b-

v2 7/7 и его отдельный кодовый прогон 11/11 относятся к 0aa9bdf; они не считаются новым повторением на 60192a9.

Ожидаемый исход шестого запроса — draft_script, сохранённый для просмотра. ChatAcceptanceRunner не нажимает «Применить» и не импортирует C#. Седьмой запрос должен сохранить этот план и не менять сцену. Выполнение этих условий проверяется независимо от правильности самого текста программы. Формальный проход draft/review-gate не означает допустимость Unity API или правильную зависимость скорости от Time.deltaTime; эти свойства проверяются по точному исходнику до импорта.

Просмотр C# и отдельная проверка компиляции

В пользовательском процессе исходник читается в сообщении чата и импортируется явной кнопкой применения. Затем проверяются реальные сообщения Unity compiler. После успешной компиляции подключение компонента выполняется отдельным просмотренным планом. В сообщении доступного сохранённого импорта кнопка «Вернуть предыдущий код» восстанавливает прежний файл или удаляет новый и инициирует повторную компиляцию. Последующие ручные правки не перезаписываются. Возврат исходника не заменяет Undo других изменений сцены. После применения и возврата в сообщении остаётся снимок принятого исходника для чтения и копирования; он не обязан совпадать с текущим файлом. Для повторного импорта нужен новый действующий PendingPlan, а не нажатие на исторический снимок.

Для воспроизводимого испытания предусмотрен ChatCodeAcceptanceRunner. Он использует уже подготовленный pending-план из сохранённой изолированной истории этого же проекта, а не запрашивает новую программу у модели. До запуска необходимо содержательно проверить точный args.code и записать SHA-256 его UTF-8-строки. Контрольная сумма файла с добавленным служебным маркером или изменёнными переводами строк не заменяет hash исходной строки. Входной

JSON должен сохранять sessionId, historyFolder и неизменённый capturedReviewOnlyPlans; одного перенесённого JSON без истории недостаточно.

Для испытания требуется новое уникальное имя C#-класса и файла. Runner исполняется в выделенном batch-проекте, проверяет отсутствие текущей компиляции и не применяет код, если сохранённый backup занят другим импортом. При несоответствии проверенного hash автоматического применения не происходит. WW_REVIEW_HASH задаётся только после просмотра конкретного исходника, WW_CODE_REPORT — новым путём протокола. Команда выполняется из WW_PROJECT без -quit:

```
WW_CHAT_INPUT="$WW_AGENT_REPORT" \
WW_CODE_REVIEW_SHA256="$WW_REVIEW_HASH" \
WW_CODE_OUTPUT="$WW_CODE_REPORT" \
"$WW_UNITY" -batchmode -nographics \
-projectPath "$WW_PROJECT" \
-executeMethod \
WorldWeaver.Editor.Assistant.ChatCodeAcceptanceRunner.RunCLI
```

Проверка охватывает чтение импортированного файла, события настоящей компиляции, полученный тип MonoBehaviour, действительное подключение к собственной тестовой фигуре, восстановление исходного файлового состояния, повторную компиляцию и очистку временной сцены. По выходу изучаются checks, errors, sourceRestoredAtExit, manualCleanupRequired и manualCleanupPaths. Компиляция и наличие компонента не подтверждают его поведение в Play Mode: например, вращение надо наблюдать отдельно. Принятый C# выполняется с правами Unity; hash, просмотр и компилятор не создают sandbox.

Повторение исходного QA-эксперимента 1.0

Отдельная копия репозитория на теге experiment-2026-10-30 используется для воспроизведения базового QA. Его корпус содержит 30 вопросов для каждой из трёх конфигураций: локальная Qwen3.5 2B без RAG, та же модель с RAG и облачная GPT-OSS 20B с RAG. Всего сохранены 90 обращений: 89 текстовых ответов и один HTTP 400. Эти наблюдения

относятся к исходному контракту и не пересчитываются как ответы редакторского агента 1.1.

ResearchRunner запускается с WW_EVAL_PROVIDER=local либо cloud. Вопросы читаются из scenarios.json; для local выполняются конфигурации без RAG и с RAG, для cloud — с RAG. Накопление памяти выключается. До нового запуска исходные JSONL архивируются или для эксперимента используется отдельная копия: повторное выполнение перезаписывает соответствующие данные.

Для исходной бесплатной конфигурации CloudRoutes оставляет Terrain и Planning локальными, а Dialogue использует Pollinations Classic и openai-fast. В trace проверяется фактическая модель. На исходном теге установлен интервал обращений 31 секунда; он сохраняется как исторический параметр опыта. В текущей редакции от 31.10.2026 подтверждён серверный бесплатный интервал 45 секунд, клиент использует запас до 46 секунд и предел max_tokens=1500. Перенос новой квоты в старый протокол был бы изменением условий, поэтому повторный inference оформляется отдельно. Текущая доступность стороннего сервиса проверяется заново; одинаковые параметры прошлого API не обещают идентичный ответ.

Скрипт analyze_results.py читает сохранённые JSONL и terrain.json и записывает metrics-per-question.csv, summary.json и графики. Повторение агрегатов требует Python с matplotlib и numpy, но не запуска модели. Сначала сохраняются прежние аналитические артефакты; новый анализ не должен незаметно заменить опубликованный протокол. Совпадение факта в сырой речи и факт в принятом игроком ответе учитываются раздельно.

Исходные сквозные runtime-сценарии запускаются отдельно с WW_REAL_TESTS=1 и фильтром RealProviderIntegration. Без флага они пропускаются; это не подтверждает доступность inference. Движение проверяется по NavMeshAgent и изменению позиции в подготовленной области, передача — по обоим Inventory. При отказе локального соединения проверяются daemon, модели и embedding endpoint. Исторический транспорт

учитывал HTTP 429 и 402 с Retry-After. В текущем адаптере среди HTTP 402 ограниченно повторяется только распознанный FREE_LIMIT; обычная платёжная ошибка не запускает повтор или оплату. HTTP 400 и ошибочный JSON не превращаются в действие вручную.